

1. Проект по ТПКС. Команда 3	3
1.1 Бизнес-требования	4
1.1.1 Глоссарий	4
1.1.2 Системные требования	6
1.1.3 Анализ предметной области	20
1.2 [Deprecated] Архитектура системы	24
1.3 Пользовательские сценарии	27
1.3.1 Сценарии в виде диаграммы последовательностей	27
1.3.2 Сценарии в виде UML-диаграммы	29
1.3.3 Сценарии в виде диаграммы кооперации	30
1.4 Разработка	32
1.4.1 Серверная часть	33
1.4.2 Пользовательская часть	33
1.4.3 Структура работы над data science-проектом	33
1.4.4 Диаграмма классов, участвующих в приложении	35
1.4.5 Функциональная модель приложения	36
1.4.6 Требования к ПО	39
1.4.6.1 Требования к процессу модерирования заявок для добавления событий	40
1.4.6.2 Требования к процессу обработки репортов пользователей менеджером	42
1.4.6.3 Требования к процессу регистрации организатора	47
1.4.6.4 Требования к базовым сценариям	52
1.4.6.5 Требования к пользовательскому профилю	54
1.4.6.6 Требования к процессу регистрации новых менеджеров	55
1.4.6.7 Требования к процессу написания отзывов	57
1.4.6.8 Требования к процессу написания репортов	58
1.4.6.9 Требования к представлению основной ленты событий	59
1.4.6.10 Требования к процессу поиска событий по тегам или фильтру	61
1.4.6.11 Требования к представлению персонализированной ленты событий	62
1.4.7 Проектирование	65
1.4.7.1 Архитектура системы	65
1.4.7.2 Mobile Gateway	70
1.4.7.3 CRM Gateway	75
1.4.7.4 Events Backend. Сервис с данными о мероприятиях и информацией о пользователях	78
1.4.7.5 CRM Backend. Сервис с данными об организаторах и информацией о репортах.	81

1.4.7.6 Мобильное приложение	82
1.4.7.7 DWH	84
1.5 Документация	88
1.5.1 Глоссарий терминов	89
1.5.2 Ответственность участников разработки	91
1.5.3 План разработки	93
1.5.3.1 Модель жизненного цикла	94
1.5.3.2 Использование Jira	95
1.5.3.3 Процесс разработки требований к ПО	97
1.5.4 План управления конфигурацией	99
1.5.4.1 Базовые версии	99
1.5.4.2 Идентификация единиц конфигурации	104
1.5.4.3 Инструменты управления конфигурацией ...	106
1.5.5 Стандарты	107
1.5.5.1 Стандарт на кодирование backend-составляющей	108
1.5.5.2 Стандарт на кодирование data science-составляющей	109
1.5.5.3 Стандарт написания пользовательских сценариев	113
1.5.5.4 Стандарт на проектирование баз данных ...	113
1.5.5.5 Стандарт на кодирование frontend-составляющей	119
1.5.5.6 Стандарт на представление аналитики	122
1.5.5.7 Стандарт на разработку требований	126
1.5.6 План верификации	129
1.5.6.1 Формальная инспекция	130
1.5.6.2 Тестирование ПО	133
1.5.6.3 Анализ покрытия	137
1.6 Тестирование	137
1.6.1 gonkey. Функциональное тестирование backend .	137

Проект по ТПКС. Команда 3

Общая информация

Проект: разработка программной системы для студентов.

В данном пространстве Confluence расположены все документы, написанные в рамках работы над проектом в курсе Технология программирования кибернетических систем (управление программными проектами)

Идея проекта

Создать приложение для студента, которое бы позволяло:

- Отслеживать информацию об актуальных студенческих мероприятиях
- Получать информацию о релевантных и актуальных акциях/скидках и пр.
- Смотреть новости города и ВУЗа
- Производить управление информацией для студентов со стороны города, ВУЗа,

Целевая аудитория: студенты, ВУЗы, работники городских администраций.

Сутью является агрегация информации, которая является релевантной для студентов. Для обеспечения этого требуются разные части

Более подробное описание отражено в бизнес-требованиях.

Состав команды

Участник	Роль
Лев Мардер	Лидер команды
Павел Арсентьев	Data Science
Александр Тихонов	Backend, проектирование БД
Георгий Чистов	Backend, серверная архитектура
Кирилл Воронин	Frontend
Николай Пак	Дизайн
Мария Морозевич	Тестирование, проектирование пользовательских сценариев

Андрей Кузьмин	Аналитика
----------------	-----------

Используемые средства

- Управление разработкой - Jira
- База знаний - Confluence
- Система версионирования - Gitlab

Бизнес-требования

Назначение раздела

Данный раздел предназначен для описания функциональных бизнес-требований к приложению для студентов. Эти требования описывают основное поведение приложения со стороны “заказчика”, которое будет перенесено на пользовательскую часть.

Данный раздел будет использован членами проекта в дальнейшей разработке.

Содержание

- [Глоссарий](#)
- [Анализ предметной области](#)

Глоссарий

Назначение подраздела

Глоссарий необходим для свободного ориентирования в проекте. В данном подразделе представлены основные понятия, характерные для всего проекта.

Список понятий

	Понятие	Определение	Аналоги
1	Пользователь	Человек, который использует мобильное приложение	Юзер, клиент
2	Менеджер	Человек, который использует администраторскую панель в роли менеджера	

3	Администратор	Человек, использующий администраторскую панель в роли администратора	
4	Главный экран	Основной экран, который видит пользователь при входе в приложение. Включает в себя ленту событий, ленту новостей, профиль пользователя и	
5	Событие	Основная информационная единица в приложении. Является собирательным понятием, включающим в себя мероприятия, скидки	
6	Требование	Спецификация того, что должно быть реализовано. В них описано поведение системы или ее атрибуты. Они могут служить ограничениями в процессе разработки системы	
7	Бизнес-требование	Описывают, почему организации нужна такая система, то есть цели, которые она планирует достичь с ее помощью	
8	Пользовательское требование	Описывает цели и задачи, которые пользователь должен иметь возможность выполнять с помощью продукта. Оно описывает то, что пользователь должен иметь возможность делать с системой	
9	Функциональное требование	Определяет, каким должно быть поведение продукта в тех или иных условиях	

10	Нефункциональное требование	Описание свойства или особенности, которыми должна обладать система, или ограничение, которое должно соблюдаться	
11	Системное требование	Описывает требование к продукту, которое содержит многие компоненты или подсистемы	
12	Бизнес-правило	Политика, предписание, стандарт или правило, определяющее или ограничивающее некоторые стороны бизнес-процессов. Это не требование к ПО, но оно служит источником нескольких типов требований к ПО	
13	Ограничение	Ограничение на выбор вариантов, доступных разработчику при проектировании и разработке продукта.	
14	Атрибут качества	Вид нефункционального требования, описывающего характеристику сервиса или производительности продукта	

Системные требования

Описание

В данном подразделе содержатся сценарии использования системы, предоставленные заказчиком.

0. Базовые сценарии

Действие	Результат	Комментарий
----------	-----------	-------------

1	Вход в приложение в первый раз/после скачивания	Открываются приветственные “слайдеры” с подсказками по работе и ориентации в приложении. Предложение включить уведомления о новых событиях. Предложение авторизоваться /зарегистрироваться в системе	Как и в любом приложении, quick start guide после первого запуска приложения. Там несколько слайдов, каждый с какой то своей инфой, а ля где можно найти свой профиль, где поиск, где еще что-то.
2	Вход в приложение	Открываются актуальные новости по новым популярным событиям	По сути основная лента, но новые события в ней выше
3	Вход в приложение а авторизованным пользователем	Открываются актуальные новости по новым популярным событиям + персональные рекомендации	Аналогично с предыдущим + рекомендации по интересам
4	Повторный вход в приложение в течение определенного интервала времени, после закрытия его в первый раз	Появление рор-уп окна “Предложить оценить приложение” в магазине приложений (не чаще, чем раз в сутки)	Например, закрыл пользователь приложение и через минут 10 открыл снова.

5	Регистрация в приложении	Открывается форма для заполнения данных: логин, возраст, электронная почта, пароль и повторение пароля, капча. На след странице подтверждение эл почты. После подтверждения открывается заполнение персональных интересов пользователя: вид события ...	Можно еще добавить Имя
6	Авторизация в приложении	Открывается форма для заполнения данных: электронная почта + пароль или авторизоваться используя соц сети /гос услуги	
7	Открытие профиля после регистрации с полями для заполнения	Открывается полный профиль пользователя с возможностью ввода доп. информации и интересов	

1. Пользовательский профиль

Действие	Результат	
----------	-----------	--

1	Открытие профиля	Открывается полная информация о пользователе, что он заполнял при регистрации + численная информация о посещенных событиях, активности в приложении	Вынести как основную кнопку в приложении
2	Просмотр подробной /дополнительной информации в профиле	Открывается информация о пользователе в профиле	Возможно разделить на несколько вкладок (как в вк) инфу, заполняемую в профиле
3	Нажатие кнопки “редактировать”	Открытие профиля с полями и возможностью их редактирования	Редактировать можно все, кроме логина
4	Изменение полей профиля с последующим нажатием кнопки “сохранить” для завершения редактирования	Открытие полного профиля, передача измененных данных в базу данных	

2. Просмотр основной ленты событий

	Действие	Результат	
1	Открытие общей ленты	Открывается лента новостей, мероприятий и скидок и их краткое описание	Вынести как основную кнопку в приложении
2	Листание ленты	Отображение объявлений	Вертикальная лента

3	Сортирование ленты	Выбор из вариантов: по новизне, по популярности, по количеству голосов “я пойду”, по рейтингу	
4	Открытие фильтра	Предлагается фильтровать по типу объявления: событие, скидка, новость; по типу события: выставка, театр и тп; по дате; по возрасту; по направлению: наука, что-то художественное и тп; по цене билета. Все это в виде чекбоксов /выпадающих меню /интервалов(в случае для чисел). В конце две кнопки: убрать фильтр и применить.	
5	Применение фильтра к ленте	Отображение подходящих под условия фильтра объявлений в ленте, отображение самого фильтра вверху ленты с возможностью отключить любое из условий отдельно или все сразу	
6	Нажатие на любое из объявлений в ленте	Открывается окно с полной информацией этого объявления	

3. Просмотр персонализированной ленты событий

	Действие	Результат	
1	Открытие персонализированной ленты (вторая основная кнопка в приложении)	Открывается лента новостей, мероприятий и скидок и их краткое описание, актуальных для конкретного пользователя	или можно сделать как доп функцию (переключатель) для обычной ленты
2	Листание ленты	Отображение объявлений	Вертикальная лента
3	Сортирование ленты	Выбор из вариантов: по новизне, по популярности, по количеству голосов “я пойду”, по рейтингу	
4	Открытие фильтра	Предлагается фильтровать по типу объявления: событие, скидка, новость; по типу события: выставка, театр и тп; по дате; по возрасту; по направлению: наука, что-то художественное и тп; по цене билета. Все это в виде чекбоксов /выпадающих меню /интервалов(в случае для чисел). В конце две кнопки: убрать фильтр и применить.	

5	Применение фильтра к ленте	Отображение подходящих под условия фильтра объявлений в ленте, отображение самого фильтра вверху ленты с возможностью отключить любое из условий отдельно или все сразу	
6	Нажатие на любое из объявлений в ленте	Открывается окно с полной информацией этого объявления	

4. Поиск события по тегу(-ам) и/или фильтру

	Действие	Результат	
1	Открытие поиска	Открывается строка поиска сверху, под ней полоска с фильтром, по середине часто искомые запросы по событиям, скидкам, тегам (должны обновляться регулярно)	третья основная кнопка в приложении
2	Нажатие на полоску с фильтрами	Открывается новое окно со всеми возможными фильтрами, внизу кнопка применить. После закрывается окно и срабатывает поиск с применением фильтров.	

3	Нажатие на любой из предложенных вариантов	Если это конкретное событие/скидка, то открывается карточка с общей инфой по ним. Если это ключевые слова для поиска или тег, то поиск с применением фильтра с использованием ключевого слова /тега соответственно.	
---	--	---	--

5. Написание отзыва

	Действие	Результат	
1	Открытие блока с отзывами о событии на странице события	Отображается как на али экспresse: количество оценок, качество оценок, последние фотографии и комментарии, отсортированные по новизне (чем ниже, тем старше). Так же есть кнопки лайк /дизлайк на отзыв и флажок с функцией репорта комментария.	На странице с основной информацией о событии после блока с текстом идут отзывы от людей. Там же можно оставлять фотографии, помимо текста и оценки.

2	Есть кнопка “оставить отзыв” по желанию	Открывается стандартная форма с полем для заполнения текста, внизу 5 звездочек для оценки, возможность прикрепить фото или видео и кнопка отправить. Оставлять отзывы могут только авторизованные пользователи.	Любой может оставить отзыв, даже если не было его на событии (ну не проверять же человека что он был там или нет), но при условии что он интересовался событием более 1 раза (открывал карточку), причем оставить отзыв можно не ранее чем через N промежуток времени после первого открытия события в приложении
3	Открытие окна с предложением пройти небольшой опрос, рассказав подробнее о мероприятии/скидке	Открывается форма с несколькими оценками по разным критериям	После того как был оставлен отзыв предлагается пользователю более подробно оценить событие, ответив на какие то основные вопросы (больше не надо, так как скорее всего в отзыве он и так это все написал, а нам чисто для статистики)

4	Репорт на отзыв другого пользователя	Открывается стандартная форма с причиной жалобы в выпадающем меню. Ниже нее поле для текста для подробного описания жалобы. В конце кнопка пожаловаться /отправить жалобу.	
---	--------------------------------------	--	--

6. Написание репорта

	Действие	Результат	
1	Репорт на событие	У любого события на странице с информацией должна быть кнопка “сообщить об ошибке”. Открывается стандартная форма с причиной жалобы в выпадающем меню. Ниже нее поле для текста для подробного описания жалобы. В конце кнопка пожаловаться /отправить жалобу.	В основном жалобы на неправильную информацию о событии

2	Репорт на скидку	У любой скидки на странице с информацией должна быть кнопка “сообщить об ошибке”. Открывается стандартная форма с причиной жалобы в выпадающем меню. Ниже нее поле для текста для подробного описания жалобы. В конце кнопка пожаловаться /отправить жалобу.	Например некорректное /неактуальное предложение
---	------------------	---	---

7. Регистрация и добавление новых организаторов

	Действие	Результат	
1	Вход в CRM и последующая регистрация	Отправка сообщения на почту об оставленной заявке, находящейся на рассмотрении Получение заявки на нового организатора, подбор и/или одобрение его тегов Передача на одобрение главному менеджеру	Зарегистрироваться могут только при создании события (во избежании мертвых орг аккаунтов).
2	Получения уведомления по почте о решении по заявке	Отправка шаблона письма с решением	

3	Генерирование пары логина-пароля для входа в систему организатором	После положительного решения о регистрации в системе нового организатора запускается генератор логина-пароля с последующей отправкой credentials организатору	
4	Предоставление расширенного доступа к функционалу приложения	Автоматически или вручную выдаются доступы к различному функционалу в приложении	Для разных организаторов разные возможности по добавлению событий/скидок

8. Регистрация и добавление новых менеджеров

	Действие	Результат	
1	Добавление главным менеджером данных о новом сотруднике	Выделение ему доступа к рабочим компонентам, обновление базы данных сотрудников	
2	Генерирование пары логина-пароля для входа в систему менеджером	После регистрации главным менеджером генерируются credentials и отправляются на почту зарегистрированного менеджера	

3	Предоставление расширенного доступа к функционалу приложения	Автоматически или вручную выдаются доступы к различному функционалу в приложении	Для разных менеджеров разные возможности для модерирования (или чем они там занимаются)
---	--	--	---

9. Добавление событий/скидок и их модерирование

	Действие	Результат	
1	Открытие списка заявок CRM	Открытие интерфейса заявок CRM	
2	Принятие заявки мероприятия/скидки	Добавление события в базу данных событий, отправка уведомления по почте организатору	
3	Редактирование заявки	Отправка уведомления о внесенных изменениях в заявку организатору и ожидание новой заявки по данному мероприятию/скидке	
4	Отклонение заявки по критериям	Написание комментария "почему", отправка уведомления по почте организатору	

10. Обработка репортов пользователей менеджером

	Действие	Результат	
1	Открытие списка репортов	Открытие интерфейса репортов CRM	

2	Чтение репорта и вынесение решения после его рассмотрения	Возможность добавления менеджером организаторов для рассмотрения в черный список Написание оповещения для пользователя Добавление событий данного организатора в теневой бан	
3	Добавление организатора /события/скидки в черный список /удаление	Автоматически или вручную удаляется или добавляется в черный список организатор /событие/скидка при нарушении правил пользования	
4	Открытие черного списка организаторов и вынесение решения главным менеджером	Изменение статуса организатора Отправка по шаблону уведомления по почте о решении организатору	

11. Заполнение заявок организаторами

Действие	Результат	
----------	-----------	--

1	Создание заявки на добавление мероприятия/скидки	Открытие интерфейса, в котором есть стандартные поля с названием, информацией, сроками проведения, дополнительными ссылками и контактами (для обратной связи с организатором).	
2	Отправка заявки на добавление мероприятия/скидки	После нажатия кнопки “отправить заявку”, организатору показывается текст об отправленной заявке	
3	Автоматическая проверка на корректность заполняемых данных для мероприятия/скидки организатором	Мгновенное уведомление организатора о существующих проблемах в заявке на добавление мероприятия/скидки	
4	Сохранение заявки “как черновик”, при ее закрытии во время редактирования	Сохраняется в профиле организатора карточка с неотправленной заявкой.	По типу неотправленного письма на яндекс почте

12. Просмотр дашбордов по пользовательской активности

13. Получение апдейтов сервиса

Анализ предметной области

Общая характеристика предметной области

Разрабатываемое приложение “Город - студенту” предоставляет муниципальной администрации города Москва, администрациям учебных

заведений и частным организаторам услуги централизованной агрегации информации о событиях в сфере образования, мероприятиях, услугах и акциях для обучающихся ВУЗов и колледжей. Предметная область оперирования приложения охватывает частично или полностью большое количество различных сфер жизни и деятельности, таких как:

- сфера высшего и среднего профессионального образования (целевая аудитория - студенты ВУЗов и колледжей, осуществляется агрегация информации о новостях и мероприятиях ВУЗов и колледжей);
- сфера муниципального управления образованием (муниципальная администрация осуществляет управление приложением, распространяет через него новости и прочую важную информацию, модерирует заявки на размещение рекламы/акций/новостей со стороны независимых организаторов и учебных заведений в соответствии с внутренними стандартами);
- информационно-новостная сфера (происходит агрегация актуальных новостей от муниципальной администрации, администраций учебных заведений);
- сфера рекламы и маркетинга (происходит агрегация рекламы мероприятий, услуг и акций от независимых организаторов /предпринимателей, при этом используются алгоритмы таргетирования рекламы);
- сфера развлечений (среди рекламируемых мероприятий, услуг и акций имеются входящие в развлекательную сферу - концерты, фестивали и т.д.);
- сфера дополнительного образования (среди рекламируемых мероприятий, услуг и акций имеются входящие в сферу дополнительного образования - курсы, мастер-классы, интеллектуальные состязания и т.д.);

Пользователей приложения можно разделить на три группы:

- модераторы, управляющие приложением от лица муниципальной администрации;
- организаторы, частные организаторы и администрации ВУЗов, использующие приложение для распространения информации и рекламы;
- студенты, использующие приложение для получения информации

Предметная область данного проекта представляет из себя совокупность потребностей этих групп, связанные с вопросом, связанных с удовлетворением этих потребностей процессов и потока информации между этими группами. Задача приложения - удовлетворить все потребности этих групп, связанные с централизованной агрегацией информации, автоматизировав связанные с ними процессы.

Анализ потребностей муниципальной администрации и связанных с ними процессов

Одной из групп, которые можно выделить в данной предметной области, является администрация города, которая совместно с его учебными заведениями запускает приложение “Город - студенту”. В задачи представителей муниципальной администрации входят контроль над агрегируемой приложением информацией и над тем, какие пользователи имеют доступ к использованию приложения. С точки зрения рассматриваемой предметной области можно выделить следующие потребности муниципальной администрации:

- возможность устанавливать стандарты и требования для распространяемой информации и применять их;
- полный контроль над всей распространяемой информацией;
- полная личностная идентификация всех использующих сервис пользователей;
- полный доступ к личным данным пользователей и собранной о них информации;
- полный контроль над тем, какие пользователи имеют доступ к сервису;
- возможность собирать статистику о взаимодействиях пользователей с сервисом;

С этими потребностями связаны следующие процессы:

- рассмотрение и одобрение поступающих запросов на публикацию информации;
- поиск и исключение информации из цикла публикации;
- регистрация пользователей с помощью личных документов;
- поиск пользователей и получение их личных данных, статистики их взаимодействий с сайтом;

Анализ потребностей пользователей-организаторов и связанных с ними процессов

Другой группой, которую можно выделить в данной предметной области, являются пользователи-организаторы, которые размещают информацию в приложении. Их можно разделить на две группы - представители учебных заведений и частные организаторы.

Публикуемая заказчиками информация может представлять из себя:

- объявления и новости от администраций учебных заведений;
- рекламу разовых мероприятий развлекательного и/или образовательного характера (примеры: фестивали, концерты, хакатоны, публичные лекции)
- рекламу услуг, предоставляемых заказчиком (примеры: курсы, секции, каршеринги, экскурсии, показы фильмов)
- рекламу заведений (примеры: кафе, тренажерные залы, барбершопы)
- объявления о акциях и скидках

Для этого класса пользователей можно выделить следующие потребности:

- наличие четких, понятных и доступных стандартов и требований к формату и содержанию публикуемой информации;
- возможность представлять публикуемую информацию в удобном и доступном формате, содержащем все важные заказчику и целевой аудитории детали (ссылки, адреса, контактная информация, изображения и т.д.);
- представление публикуемой информации всем потенциально заинтересованным в ней пользователем (“таргетирование”) на основании их интересов, места проживания, места учебы и прочих факторов;
- получение исчерпывающей информации о взаимодействиях пользователей с опубликованными объявлениями;

С этими потребностями связаны следующие процессы:

- формирование заявки на публикацию информации;
- редактирование заявки на публикацию в соответствии с существующими стандартами и требованиями;
- сбор информации о интересах пользователей;
- выбор информации для демонстрации пользователю на основании собранных о нём данных;
- сбор метрик взаимодействия пользователей с опубликованной информацией, получение этих метрик организаторами.

Анализ потребностей пользователей-студентов и связанных с ними процессов

Последней из выделяемых в предметной области групп являются студенты ВУЗов и колледжей, использующие приложение для получения информации о событиях в сфере образования, объявлений, сообщений о акциях и прочего. Для них можно выделить следующие потребности:

- возможность указывать подробную личную информацию и информацию о интересах для оптимизации подбора получаемых объявлений;
- получение актуальной информации на основе своих интересов и личных данных;
- возможность отмечать свою заинтересованность или незаинтересованность в том или ином объявлении с дальнейшим влиянием этих действий на содержание получаемых объявлений;
- возможность оценивать мероприятия, услуги и акции после их посещения/использования с дальнейшей корректировкой получаемого набора объявлений в соответствии с этими оценками;

С этими потребностями связаны следующие процессы:

- создание учетной записи пользователя, содержащей его личную информацию;
- оценка предпочтений пользователя на основе его взаимодействий с публикуемыми объявлениями;

Анализ потока информации в предметной области

Поток информации в предметной области можно рассмотреть с точки зрения трёх выделенных ранее групп.

Поток информации для администрации включает в себя:

- получение заявок на публикацию информации от организаторов;
- отправку одобренных объявлений, содержащих новости, рекламу и прочую информацию, для публикации;
- отправку сообщений с описанием требуемых изменений для запросов, не удовлетворяющих установленным стандартам;
- получение личных данных студентов и организаторов по требованию;
- получение метрик о взаимодействиях пользователей с опубликованной информацией по требованию;
- получение жалоб от пользователей в отношении объявлений или других пользователей;

Поток информации для пользователей-организаторов включает в себя:

- отправку личной информации для идентификации личности;
- отправку заявок на публикацию информации администрации;
- получение сообщений с описанием требуемых изменений для запросов, не удовлетворяющих установленным стандартам;
- получение метрик о взаимодействиях пользователей с опубликованной информацией по требованию;

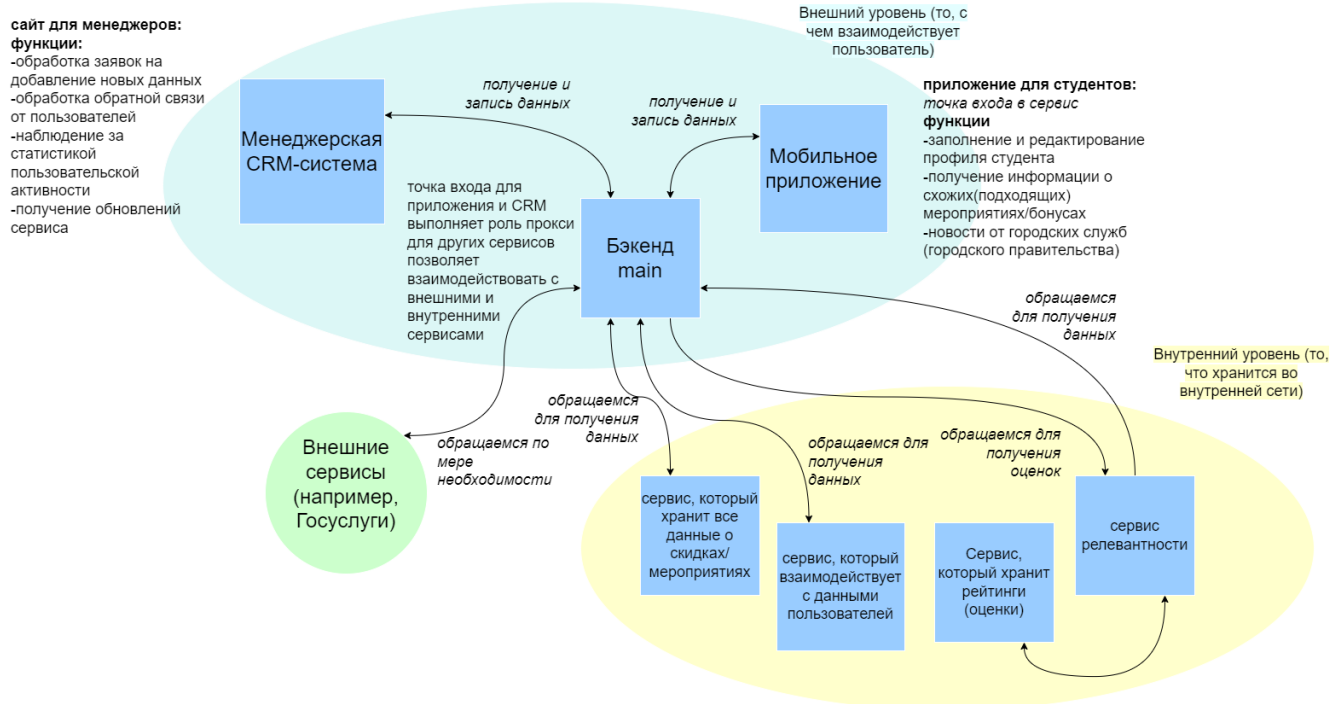
Поток информации для пользователей-студентов включает в себя:

- отправку личной информации для идентификации личности;
- отправку информации о интересах и метрик взаимодействия;
- получение актуальных опубликованных объявлений;
- отправка отзывов о объявлениях и организаторах;

[Deprecated] Архитектура системы

Графическое представление архитектуры системы:

Муниципальное приложение для студентов "Город - студентам"



Концептуально структуру приложения можно разделить на 2 уровня в зависимости от представления данных о системе пользователю:

- **внешний уровень** (та часть системы, с которой непосредственно взаимодействует пользователь);
- **внутренний уровень** (часть системы, недоступная пользователю, но хранящаяся во внутренней сети и необходимая системе для предоставления данных).

Также отдельным блоком, не являющимся частью внешнего или внутреннего уровня системы, является блок **внешних сервисов**, обращение к которым происходит при необходимости получения данных из внешней среды (примером такого внешнего сервиса может служить портал Госуслуги).

Состав внешнего уровня системы:

1. Взаимодействие пользователя-студента с системой происходит через **мобильное приложение**, которое является точкой входа в сервис.

Пользователю предоставляются следующие функции:

- заполнение и редактирование профиля студента;
- получение информации об интересующих пользователя мероприятиях /бонусах;
- просмотр новостей от городских служб (городского правительства).

2. Менеджеры имеют возможность взаимодействовать с системой с помощью **менеджерской CRM-системы**. Она предоставляет пользователю-менеджеру такие функции, как:

- обработка заявок на добавление новых данных;
- обработка обратной связи от пользователей;
- наблюдение за статистикой пользовательской активности;
- получение обновлений сервиса.

3. Точкой входа для мобильного приложения и CRM-системы является **программно-аппаратная часть сервиса (Бэкенд)**. Она выполняет роль прокси-сервера для других сервисов, выполняющего роль посредника между пользователем и целевым сервером и позволяющего клиентам как выполнять косвенные запросы к другим внешним и внутренним серверам, так и получать ответы.

Состав внутреннего уровня системы:

1. Сервис, который хранит все **данные о скидках/мероприятиях**.

Для формирования карточек событий со стороны пользователя, с помощью которых он имеет возможность просматривать, оставлять оценку или производить другие действия, необходим сервис, предоставляющий системе данные об этих событиях. Помимо основной информации о мероприятиях (описание, дата, место и т.д.), которые сервис предоставляет системе при обращении, он также имеет предоставляет возможность менеджерам менять статус событий.

2. Сервис, который взаимодействует с **данными пользователей**.

Использование специализированного сервиса для хранения и взаимодействия с пользовательскими данными необходим для возможности пользователя сохранять и обновлять в случае изменения информацию в профиле. Возможности данного сервиса позволяют пользователю также выбирать те категории мероприятий и новостей, которые ему подходят наибольшим образом. Обращение к сервису происходит также в случае формирования персонализированной ленты событий и новостей - благодаря информации, которую предоставляет сервис, настроенная лента содержит только потенциально интересные пользователю карточки новостей и мероприятий.

3. Сервис релевантности.

Взаимодействие внешнего уровня системы с сервисом релевантности происходит в двух направлениях. Обращение системы к сервису позволяет получать данные об оценках для различных целей: сбора статистики, поиска наиболее востребованных мероприятий, контроля качества со стороны менеджеров. Сервис релевантности в свою очередь обращается к системе для получения данных, необходимых для формирования оценок.

4. Сервис, который хранит **рейтинги (оценки)**.

Вспомогательным сервисом для сервиса релевантности служит сервис, позволяющий хранить оценки. Обращение к нему необходимо для получения актуальных данных о рейтинге мероприятий, с которыми в дальнейшем работает сервис релевантности.

Пользовательские сценарии

Назначение раздела

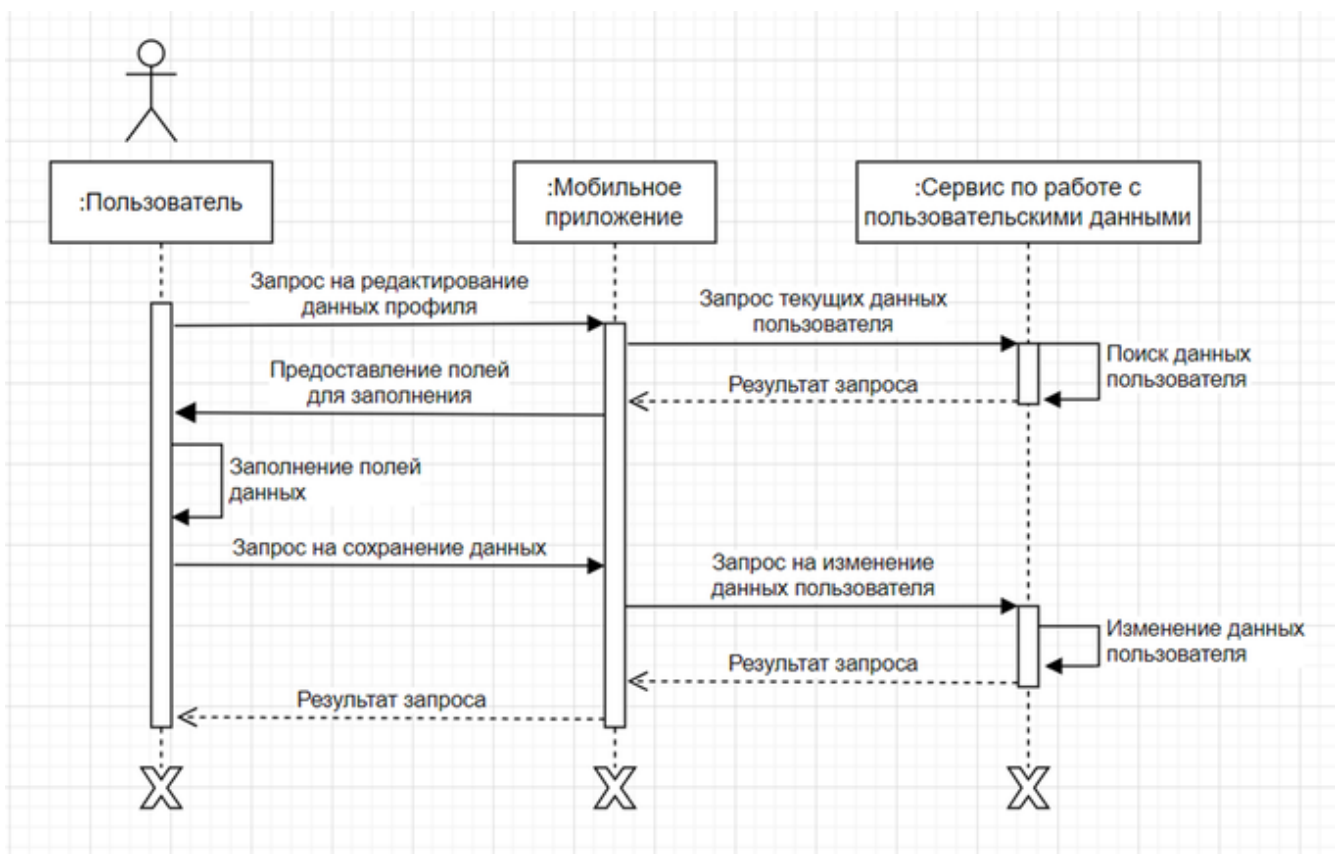
В данном разделе представлены сценарии использования приложения.

Содержание

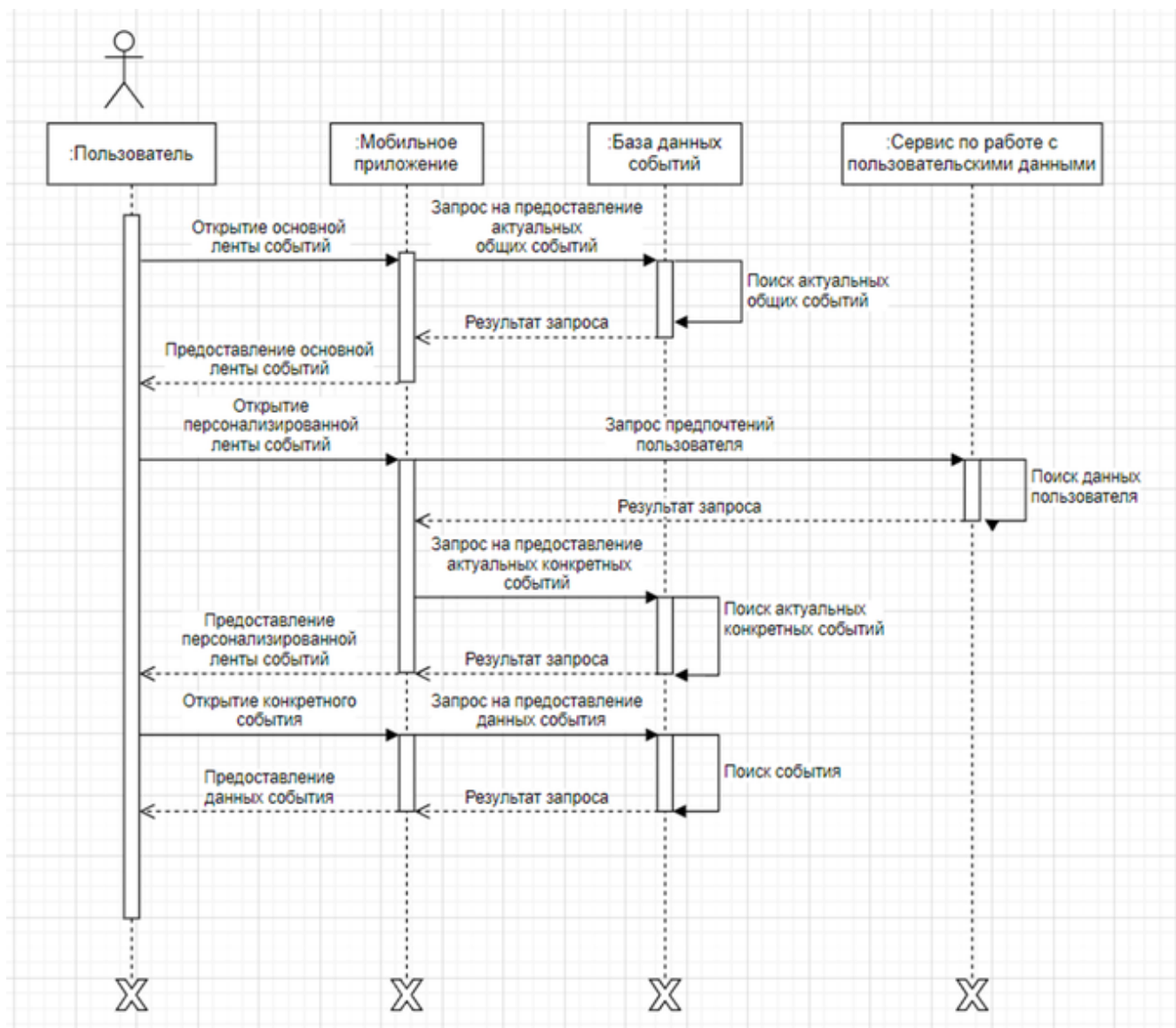
- [Сценарии использования мобильного приложения](#)
- [Сценарии в виде диаграммы последовательностей](#)
- [Сценарии в виде UML-диаграммы](#)
- [Сценарии в виде диаграммы кооперации](#)

Сценарии в виде диаграммы последовательностей

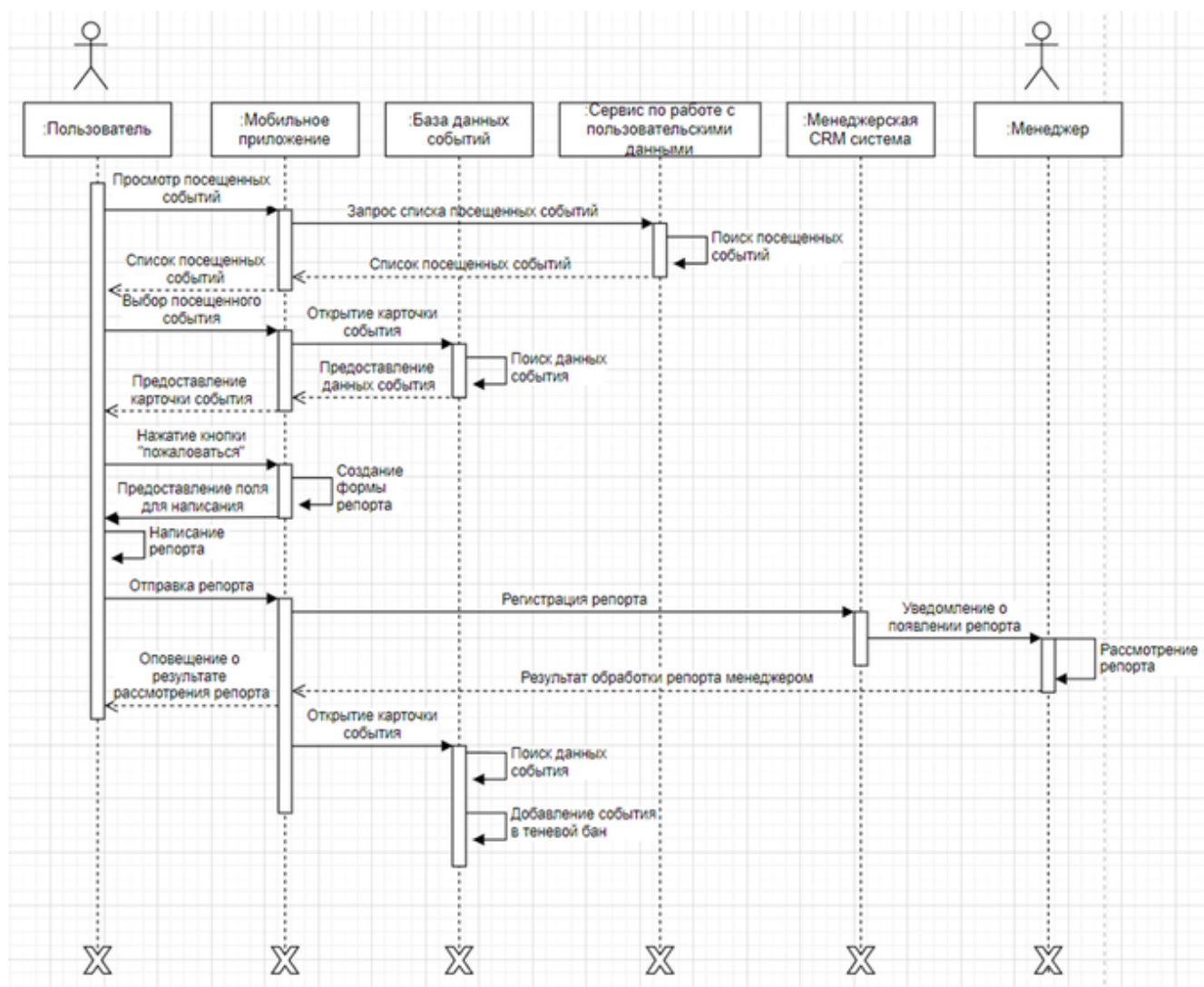
Редактирование профиля



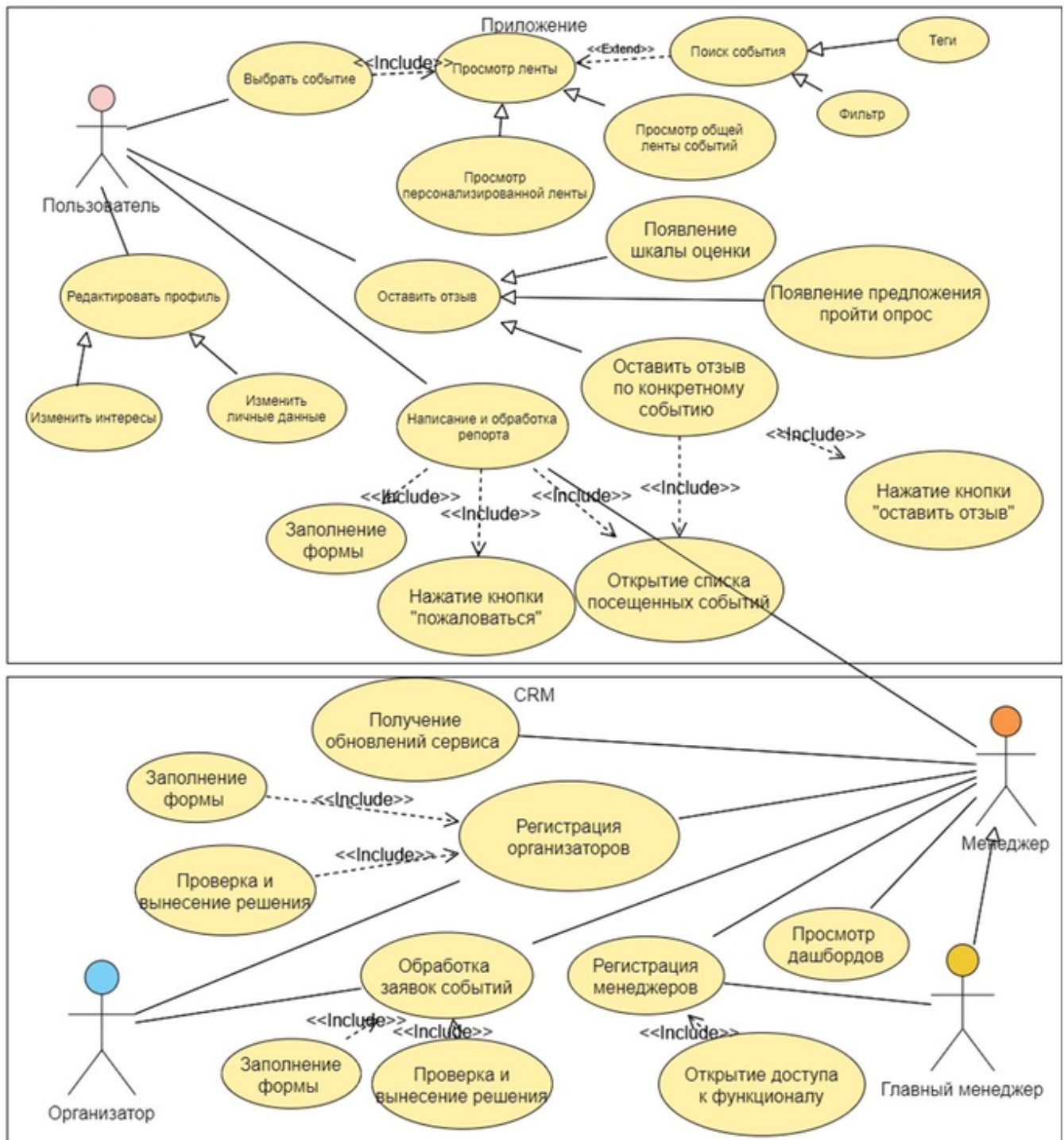
Работа с лентой событий



Работа с репортажами

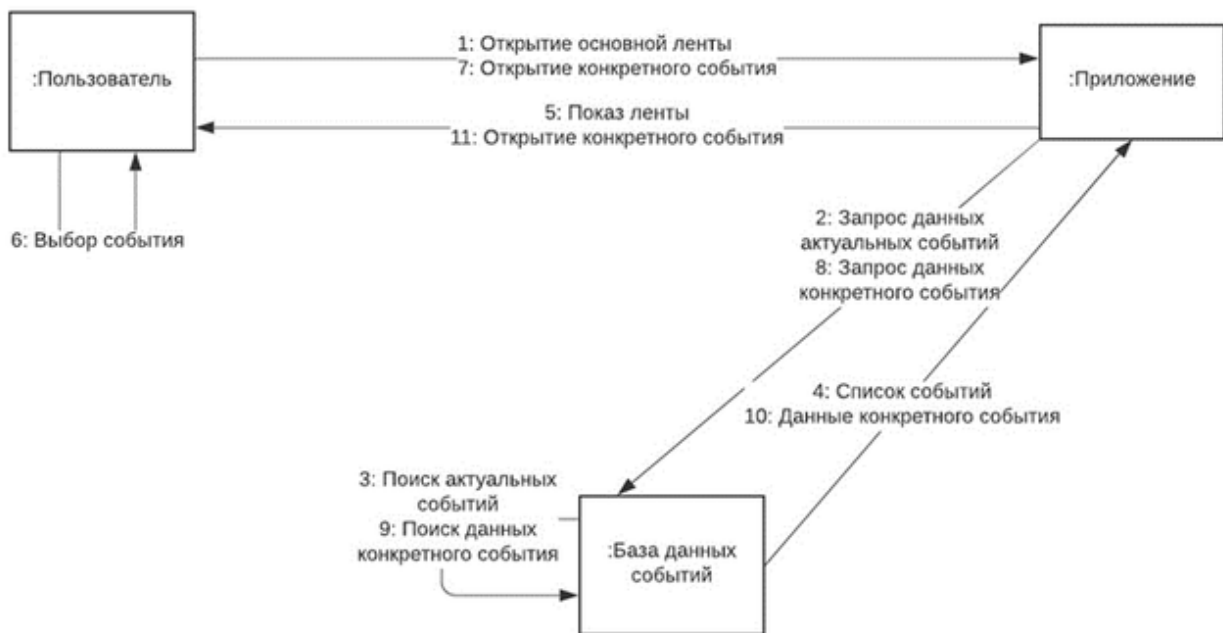


Сценарии в виде UML-диаграммы



Сценарии в виде диаграммы кооперации

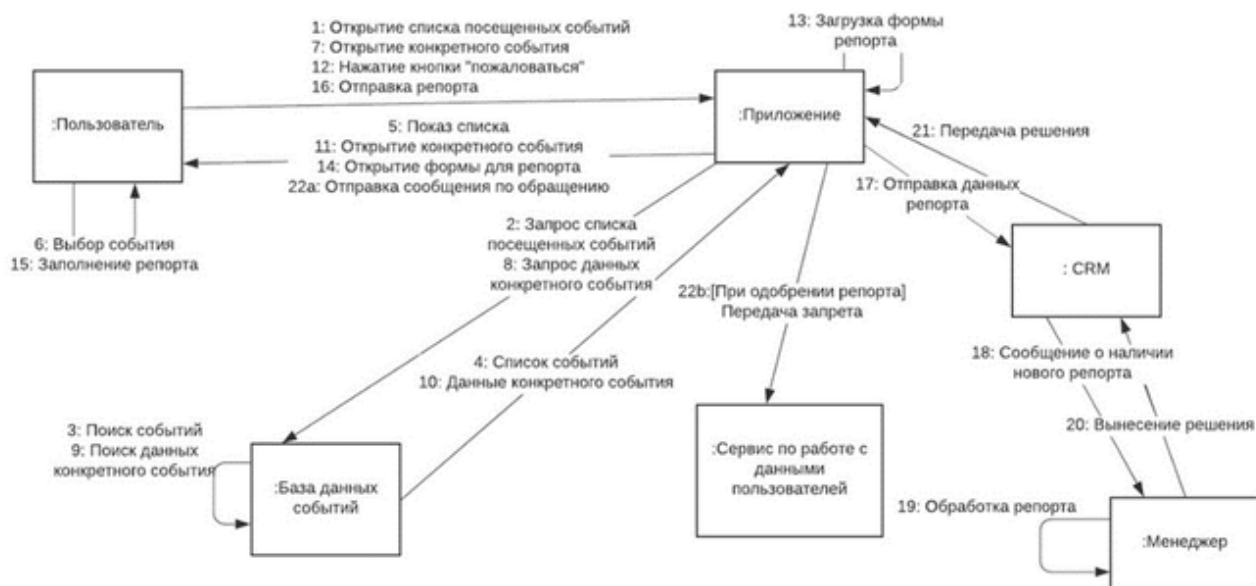
Открытие основной ленты новостей



Открытие персонализированной ленты новостей



Обработка репорта



Разработка

Назначение раздела

В данном разделе представлены требования к программной разработке системы. Здесь агрегированы знания о серверной и клиентской части приложения.

Дорожная карта

	Задача	Цель	Ссылка на Jira
1	Разработка архитектуры серверной части системы	Разработать способ взаимодействия микросервисов между собой и с проxy сервером	https://b18-504-cspt.atlassian.net/jira/software/projects/CSPT/boards/1?selectedIssue=CSPT-2
2	Анализ средств разработки пользовательского интерфейса	Проанализировать возможные средства разработки пользовательского интерфейса мобильного приложения и серверной части, которые бы отвечали самым начальным требованиям	https://b18-504-cspt.atlassian.net/jira/software/projects/CSPT/boards/1?selectedIssue=CSPT-6

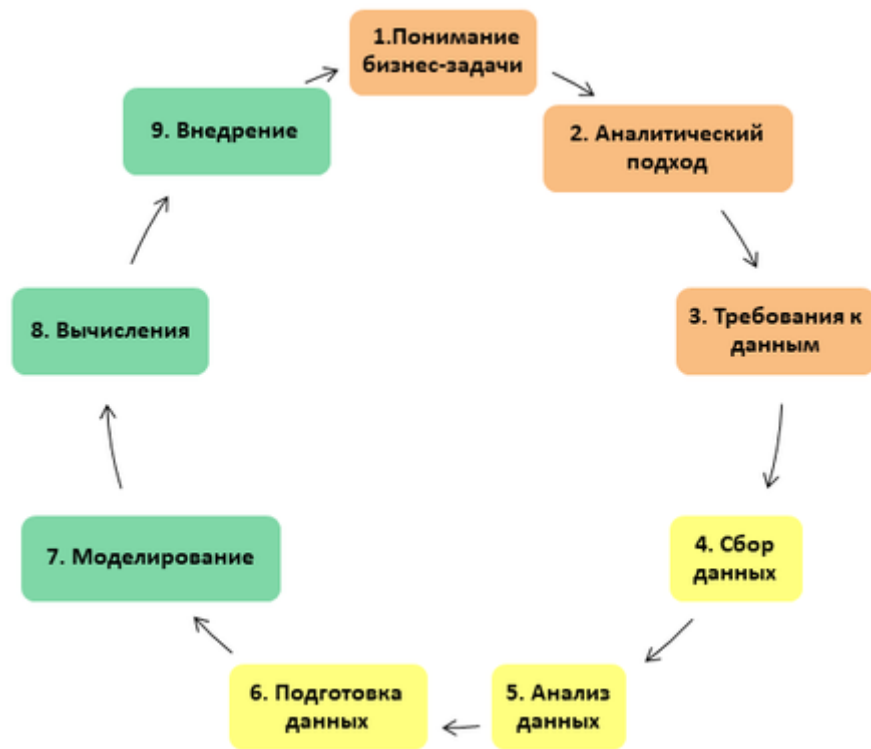
3	Написание технических требований к пользовательскому интерфейсу	Фиксирование технологического стека, средств разработки, вспомогательных компонентов разработки	https://b18-504-cspt.atlassian.net/jira/software/projects/CSPT/boards/1?selectedIssue=CSPT-18
4	Написание технических требований к серверной части	Фиксирование технологического стека, средств разработки, вспомогательных компонентов разработки	https://b18-504-cspt.atlassian.net/jira/software/projects/CSPT/boards/1?selectedIssue=CSPT-21
5	Разработка бизнес-требований к микросервисам		
6	Разработка бизнес-требований к мобильному приложению		
7	Проработка сущностей, принимающих участие в бизнес-процессах	Необходимо составить сущности, принимающие участие в бизнес-процессах, описать их и привести к виду, пригодному для использования в БД	https://b18-504-cspt.atlassian.net/jira/software/projects/CSPT/boards/1?selectedIssue=CSPT-14

Серверная часть

Пользовательская часть

Структура работы над data science-проектом

В разработке data science-проекта выделяется 3 блока по 3 этапа в каждом, всего 9 этапов.



Первый блок: работа с бизнес-требованиями

1. Понимание бизнес-задачи

Необходимо четко выявить цель заказчика.

Определение цели исследования: что является проблемой? Почему проблема должна быть решена? Кого затрагивает проблема? Какие есть альтернативы? По каким метрикам будет оцениваться успешность проекта?

2. Аналитический подход

Выбор аналитического подхода для решения поставленной бизнес-задачи.

Например если ответ должен быть вида да/нет, подойдет наивный байесовский классификатор. Если нужен ответ в виде численного признака, то подойдут регрессионные модели.

3. Требования к данным

Подготовка требований к данным: контент, форматы и источники, которые будут использованы на следующей стадии проекта.

Второй блок: работа с данными

4. Сбор данных

Данные тянутся из внутренних баз данных проекта, внешних источников. После получения данных происходит верификация данных пересмотрение требований к ним: принятие решения о необходимости

дополнительных данных, могут быть выявлены лакуны в данных и составлен план, как их закрыть или найти замену.

5. Анализ данных

Строится описательная статистика: исследуется центральное положение (среднее, медиана, мода), ищутся выбросы и выполняется оценка вариабельности (размах, дисперсия и стандартное отклонение). Строятся гистограммы переменных, ящики с усами и другие инструменты визуализации. Выполняется попарное сравнение: вычисляются корреляции между переменными, чтобы определить, какие из них связаны и насколько.

Принимается решение по поводу репрезентативности собранных данных относительно поставленной задачи

6. Подготовка данных

Удаляются дубликаты, обрабатываются отсутствующие или неверные данные, проверяются и при необходимости исправляются ошибки форматирования.

Конструируется набор факторов, с которым на следующих этапах будет работать машинное обучение: проводим извлечение и отбор признаков, которые потенциально помогут решить бизнес-задачу.

Третий блок: разработка и внедрение

7. Построение модели

Разрабатывается и проверяется модель, выбранная на этапе 2, на выборке данных, полученных на этапе 6.

8. Применение модели

На этом этапе ищется ответ на вопрос: отвечает ли построенная модель бизнес-задаче?

Проводятся диагностические измерения, которые помогают понять, работает ли модель, так как задумано.

9. Внедрение

Внедрение может происходить поэтапно, например, на ограниченную группу пользователей или в тестовом окружении. Налаживается система фидбэка, чтобы отслеживать, насколько успешно разработанная модель справляется с поставленной задачей. Также могут появиться новые источники данных, новые заинтересованные лица, не говоря уже о том, что сама бизнес-задача может быть уточнена.

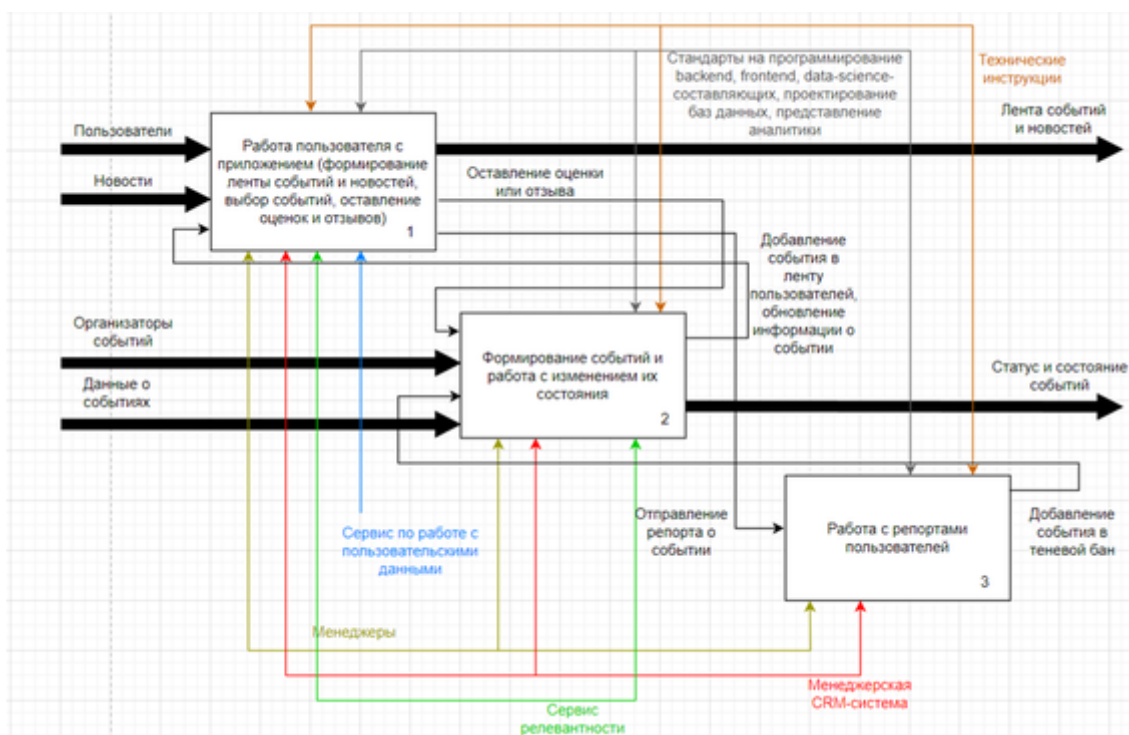
Диаграмма классов, участвующих в приложении

Контекстная диаграмма представляет собой общее описание системы и ее взаимодействие с внешней средой.

- В качестве *входа* выступают лица, взаимодействующие с приложением (пользователи и организаторы событий), а также информация, используемая приложением (данные о событиях и новостях);
- В качестве *выхода* предоставляется лента новостей и событий и информация о событиях, динамика которой зависит от рейтинга, отзывов и состояния;
- В качестве *управления* используются технические инструкции и стандарты;
- В качестве *механизма* выступает команда менеджеров, а также вспомогательные сервисы и CRM-система для менеджеров.

Диаграммы декомпозиции

Диаграмма декомпозиции 1



В качестве функциональных блоков выступают основные процессы, которые затрагивают разные стороны работы с приложением.

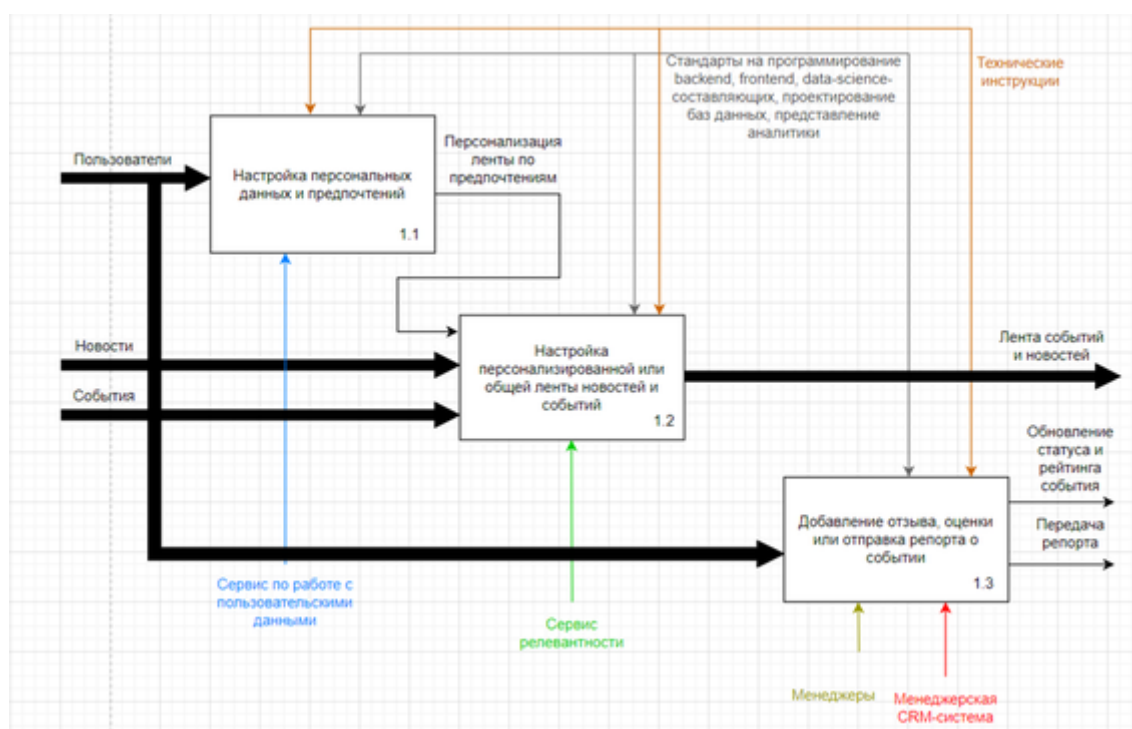
Один из процессов направлен на формирование ленты новостей и событий конкретного пользователя. Он работает с входными данными, которые предоставляет пользователь, а также с данными о новостях и событиях, и, как результат, пользователь получает персонализированную или общую ленту.

Второй процесс затрагивает формирование событий в системе – он включает в себя модерирование заявок на добавление событий, в случае одобрения менеджерами которых события попадают в ленту

пользователей. Также в рамках этого совокупного процесса происходит изменение данных события, обновление рейтинга и отзывов и добавление данных события в базу данных событий.

Также другим важным процессом, осуществляемым с помощью менеджерской CRM, является обработка репортов о событиях, получаемых от пользователей, а также изменение статусов событий после обработки менеджерами репортов.

Диаграмма декомпозиции 2



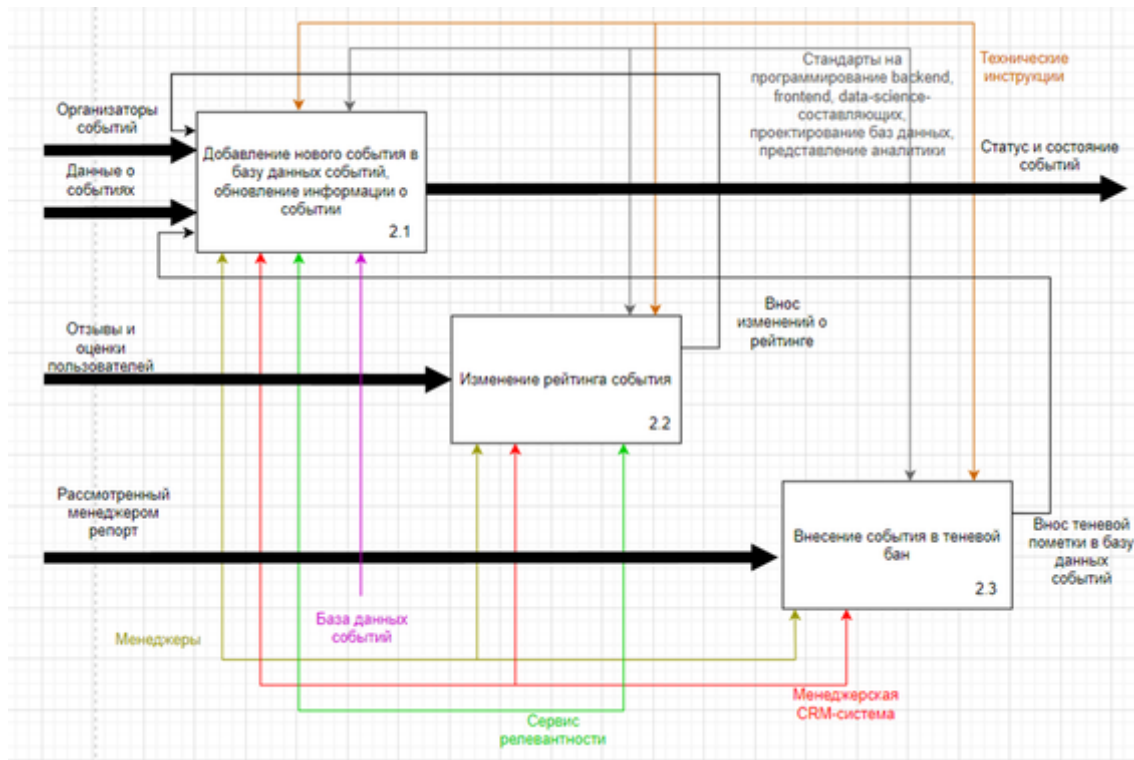
На диаграмме более подробно представлена работа пользователя с приложением, в качестве основных функциональных блоков были выделены настройка персональных данных, настройка ленты и добавление отзыва или оценки события.

В качестве входных данных для двух из этих процессов выступает сам пользователь, так как вручную изменяет данные или оставляет отзыв.

Для настройки общей ленты необходимы входные данные о новостях и событиях, а для настройки персонализированной – еще данные пользовательских предпочтений, которые хранятся и обрабатываются через сервис по работе с пользовательскими данными.

В работе с отзывами подключается менеджерская CRM и менеджеры для обработки отзывов, а также для дальнейшей работы с репортами в случае их появления.

Диаграмма декомпозиции 3



В рамках более детального рассмотрения процессов, связанных с регистрацией событий и изменением их данных и статуса, в качестве функциональных блоков были выделены добавление и изменение события в базу данных событий, изменение рейтинга и попадание события в особый статус теневого бана.

Последние два процесса работают с поступающими на вход из других, рассмотренных ранее, процессов отзывами и репортами пользователей, и в качестве выхода вносят соответствующие изменения в базу данных событий.

Требования к ПО

- Требования к процессу модерирования заявок для добавления событий
- Требования к процессу обработки репортов пользователей менеджером
- Требования к процессу регистрации организатора
- Требования к базовым сценариям
- Требования к пользовательскому профилю
- Требования к процессу регистрации новых менеджеров
- Требования к процессу написания отзывов
- Требования к процессу написания репортов
- Требования к представлению основной ленты событий
- Требования к процессу поиска событий по тегам или фильтру

- Требования к представлению персонализированной ленты событий

Требования к процессу модерирования заявок для добавления событий

Входные данные:

- описание сценариев для модерирования заявок для добавления событий
- описание архитектуры системы
- «План разработки ПО»
- «Стандарты на разработку требований к ПО»

Идентификационный номер	Требование
9-1-BF-00001	Список заявок CRM должен содержать поля “Дата”, “Номер”, “Отправитель” и “Тема”: • Поле “Дата” - поле, содержащее дату отправления заявки, имеет формат ДД.ММ.ГГГГ ЧЧ.ММ.СС • Поле “Номер” - уникальный идентификационный номер заявки • Поле “Отправитель” - текстовое поле, содержащее имя пользователя, который отправил заявку • Поле “Тема” - текстовое поле с описанием темы заявки
9-1-BF-00002	Заявки в списке должны сортироваться автоматически по полю “Дата”

9-1-BF-00003	Интерфейс должен позволять опцию поиска и фильтр заявок по полям: • “Дата” - поиск определённой даты/временного промежутка. Результатом применения фильтра будет являться список заявок, отправленных в указанную дату/за указанный временной промежуток • “Отправитель” - поиск всех заявок, отправленных определённым пользователем. Результатом применения фильтра будет являться список заявок, отправленных указанным пользователем • “Номер” - поиск определенной заявки по ее идентификационному номеру. Результатом поиска будет являться единственная заявка (при успешном результате поиска)
9-2-BF-00004	Чтобы менеджер имел возможность принять заявку, интерфейс должен содержать соответствующую кнопку “Принять”
9-2-BB-00005	В случае принятия заявки обработчик регистрирует принятие: данные о мероприятии поступают в базу данных событий, происходит валидация данных по критериям корректности и полноты. Заявка получает в CRM системе статус “принято”
9-2-BF-00006	В случае принятия заявки менеджером система создает карточку события

9-2-CXX-00007	В случае принятия заявки уведомление о результате рассмотрения поступает в виде сообщения на почту организатору
9-3-BB-00008	Должен быть реализован обработчик изменений в заявке, который формирует шаблонный текст с изменениями в заявке и ставит задачу на отправку письма организатору
9-3-CXX-00009	В случае появления изменений письмо поступает на почту организатору
9-4-BF-00010	Чтобы менеджер имел возможность отклонить заявку, интерфейс должен содержать соответствующую кнопку “Отклонить”
9-4-BB-00011	В случае отклонения заявки обработчик регистрирует отклонение: Данные о мероприятии не поступают в базу данных событий, карточка события не создаётся. Заявка получает в CRM системе статус “отклонено”
9-4-BF-00012	В случае отклонения заявки менеджером интерфейс обратной связи должен содержать поле для написания комментария под заголовком “Почему” для объяснения причин
9-4-CXX-00013	В случае отклонения заявки уведомление о результате рассмотрения поступает в виде сообщения на почту организатору

Требования к процессу обработки репортов пользователей менеджером

Входные данные:

- описание сценариев для обработки репортов пользователей менеджером
- описание архитектуры системы
- «План разработки ПО»
- «Стандарты на разработку требований к ПО»

Идентификационный номер требования	Требование
10-1-BF-00001	Список репортов CRM должен содержать поля “Дата”, “Номер”, “Отправитель”, “Событие” и “Тема”: • Поле “Дата” - поле, содержащее дату отправления репорта, имеет формат ДД.ММ.ГГГГ ЧЧ.ММ.СС • Поле “Номер” - уникальный идентификационный номер репорта • Поле “Отправитель” - текстовое поле, содержащее имя пользователя, который отправил репорт • Поле “Событие” - текстовое поле, содержащее название события, о котором написан репорт • Поле “Тема” - текстовое поле с описанием темы репорта
10-1-BF-00002	Репорты в списке должны сортироваться автоматически по полю “Дата”

10-1-BF-00003	Интерфейс должен позволять опцию поиска и фильтр репортов по полям: • “Дата” - поиск определённой даты/временного промежутка. Результатом применения фильтра будет являться список репортов, отправленных в указанную дату/за указанный временной промежуток • “Отправитель” - поиск всех репортов, отправленных определённым пользователем. Результатом применения фильтра будет являться список репортов, отправленных указанным пользователем • “Номер” - поиск определенного репорта по его идентификационному номеру. Результатом поиска будет являться единственный репорт (при успешном результате поиска) • “Событие” - поиск среди всех отправленных репортов тех, которые содержат в поле “Событие” указанные ключевые слова. Результатом применения фильтра будет являться список репортов, содержащих указанные ключевые слова в поле “Событие”
10-2-BB-00004	Должен быть реализован обработчик рассмотрения репорта, который формирует шаблонный текст с уведомлением о рассмотрении и ставит задачу на отправку письма пользователю

10-2-CXX-00005	После рассмотрения репорта на почту пользователя-отправителя репорта отправляется уведомление о рассмотрении репорта
10-2-BB-00006	В CRM системе менеджеров должен быть реализован раздел “Черный список”. Менеджер должен иметь возможность добавлять организатора в данный раздел, а также иметь возможность просматривать список организаторов, помещенных в “Черный список”
10-2-BF-00007	Раздел “Чёрный список” должен содержать следующие поля: • Поле “Организатор” - текстовое поле, содержащее имя пользователя, который является организатором события • Поле “Дата” - поле, содержащее дату добавления организатора в черный список, имеет формат ДД.ММ.ГГГГ ЧЧ.ММ.СС • Поле “Причина” - текстовое поле, содержащие объяснение причины добавления организатора в черный список

10-2-BB-00008	Пользователь CRM системы с правами главного менеджера должен иметь функционал работы с разделом “Черный список”: • Изменять статус организатора • Добавлять события организатора в теневой бан • Удалять события • Убирать организатора из черного списка
10-2-BF-00009	Для добавления организатора в раздел “Черный список” в интерфейсе должна быть реализована кнопка добавления, при нажатии на которую менеджеру предоставляется текстовое поле для написания причины добавления
10-2-BXD-00010	При добавлении организатора менеджером в раздел “Черный список” данные об организаторе поступают в базу данных пользователей.
10-2-BF-00011	При добавлении организатора менеджером в черный список данные об организаторе появляются в разделе “Черный список”. Поле “Дата” формируется автоматически исходя из времени сервера, в поле “Причина” заносятся текст, указанный ранее менеджером

10-2-BF-00012	Интерфейс раздела “Черный список” должен позволять опцию поиска и фильтрации по полям: • “Дата” - поиск определённой даты/временного промежутка. Результатом применения фильтра будет являться список организаторов, добавленных в черный список в указанную дату /за указанный временной промежуток • “Организатор” - поиск организатора по имени пользователя в черном списке. Результатом поиска будет являться единственный организатор (при успешном результате поиска)
10-3-BF-000013	Менеджер должен иметь возможность удалять события организатора для случаев нарушения правил пользования
10-4-BB-00014	Должен существовать шаблонный текст сообщения для организатора, содержащий результат рассмотрения репорта (изменение статуса организатора, удаление события)
10-4-CXX-00015	После рассмотрения репорта главным менеджером на почту организатора должно отправляться уведомление о результатах рассмотрения

Требования к процессу регистрации организатора

Входные данные:

- описание сценариев для обработки репортов со стороны пользователей
- описание архитектуры системы

- «План разработки ПО»
- «Стандарты на разработку требований к ПО»

Идентификационный номер	Требование
7-1-BF-00001	При переходе на страницу CRM без авторизации одной из опций, предоставляемой интерфейсом, должна быть опция “Подать заявку организатора”.
7-1-BF-00002	Выбор опции “Подать заявку организатора” должен открывать форму для заполнения профиля организатора.
7-1-BF-00003	Форма заявки должна содержать следующие поля/наборы полей, необходимые для формирования профиля организатора: • Наименование юридического лица/имя физического лица, являющегося организатором • Контактная информация организатора • Адрес проживания (для ФЛ)/Адрес регистрации (для ЮЛ) • Род деятельности организатора • Поле выбора тегов организатора • Поле для загрузки копий документов организатора • Поле для примечаний
7-1-BF-00004	Форма заявки должна быть оснащена инструментами для автоматической проверки соответствия введенной информации установленным условиям, таким как: • Отсутствие в наименовании/имени запрещенных символов; • Соответствие контактной информации ожидаемому формату (количество цифр в телефонном номере, формат адреса электронной почты и т.д.) • Корректность введенного адреса • Правильный формат загружаемых файлов

7-1-BF-00005	Форма заявки должна содержать блок reCAPTCHA для противодействия спаму.
7-1-BF-00006	На странице формы должна находиться кнопка “Подать заявку”.
7-1-BF-00024	При нажатии кнопки “Подать заявку”, если данные введены корректно, форма закрывается, и на экран организатора выводится сообщение о успешной отправке его заявки с просьбой ожидать контакта со стороны менеджмента сервиса, что воспользуется для этого предоставленной в контактной информацией.
7-1-BF-00025	При нажатии кнопки “Подать заявку”, если данные в заявке введены некорректно, система должна вывести сообщение о ошибке и указать на проблемные поля формы.
7-1-CFB-00007	После успешного заполнения организатором формы запрос, содержащий всю введенную им информацию, отправляется на серверную часть сервиса для последующей обработки.
7-1-CBD-00008	Информация из полученных заявок должна храниться сервером в базе рассматриваемых заявок. Каждой заявке должен присваиваться уникальный идентификатор.
7-1-BF-00009	CRM должна предоставлять менеджерам и администраторам сервиса меню рассмотрения заявок на регистрацию организаторов, что должно представлять из себя список ожидающих на данный момент рассмотрения заявок с датами их подачи.
7-1-BF-00026	При выборе одной из заявок в меню перед менеджером должна выводиться содержащаяся в заявке информация.

7-1-BF-00010	После рассмотрения заявки менеджеру должны предоставляться 4 опции: • Принять заявку • Отклонить заявку • Отправить заявку на редактирование • Отправить заявку на доп. рассмотрение
7-1-BF-00011	В случае, если менеджер выбирает опцию “Отправить заявку на доп. рассмотрение”, заявка остается в списке ожидающих рассмотрения.
7-2-BF-00012	В случае, если менеджер выбирает опцию “Отправить заявку на редактирование”, перед ним должно выводиться текстовое окно и предложение ввести комментарий, который должен быть отправлен организатору.
7-2-CFB-00013	В случае, если менеджер выбирает опцию “Принять заявку” или “Отклонить заявку”, запрос, содержащий принятое решение и идентификатор заявки должен быть отправлен на сервер.
7-2-CFB-00014	В случае, если менеджер выбирает опцию “Отправить заявку на редактирование”, запрос, содержащий принятое решение, идентификатор заявки и комментарий, должен быть отправлен на сервер.
7-2-CBX-00015	Получив сообщение о отклонении заявки, сервер должен поставить задачу отправки на указанный в заявке адрес электронной почты автоматического уведомления о отклонении заявки.
7-2-CBD-00021	После отправки уведомления об отклонении заявки информация о заявке должна быть перенесена из базы рассматриваемых заявок в регулярно очищаемую базу отклонённых заявок.

7-3-BB-00016	Получив сообщение о принятии заявки, сервер должен сгенерировать уникальные логин и пароль для учетной записи организатора.
7-2-CBD-00022	Получив сообщение о принятии заявки, сервер перенести информацию из заявки и данные для авторизации в базу учетных записей и удалить заявку из базы рассматриваемых заявок.
7-2-CBX-00017	Получив сообщение о принятии заявки, сервер должен поставить задачу отправки на указанный в заявке адрес электронной почты автоматического уведомления о принятии заявки, содержащего сгенерированные логин и пароль.
7-2-CBX-00018	Получив сообщение о отправке заявки на редактирование, сервер должен сформировать уникальную ссылку редактирования и отправить на указанный в заявке адрес электронной почты автоматическое уведомление о необходимости внесения изменений в заявку, что должно содержать в себе сформированную ссылку и комментарий менеджера.
7-2-CBD-00023	Получив сообщение о отправке заявки на редактирование, сервер должен перенести заявку в базу заявок, ожидающих редактирования.
7-2-BF-00019	В случае перехода по ссылке редактирования перед пользователем должна открываться форма заполнения новой заявки, все поля в которой автоматически заполняются информацией из предыдущей версии заявки. Эта заявка должна рассматриваться как новая заявка; после загрузки необходимой информации копия заявки должна удаляться из базы заявок, ожидающих редактирования.

7-4-A-00020	При вводе организатором логина и пароля, соответствующего логину и паролю, привязанному к созданному по одобренной заявке аккаунту, ему должен быть предоставлен расширенный функционал CRM для размещения своих объявлений в приложении.
-------------	---

Требования к базовым сценариям

Идентификационный номер	Требование
0-5-BF-00001	Форма регистрации должна содержать поля “логин”, “имя”, “возраст”, “электронная почта”, “пароль”, “повторение пароля”, а также поле Captcha.
0-5-BB-00001	Таблица, хранящая записи о пользователе в БД должна содержать столбцы “логин”, “электронная почта”, “пароль”, “возраст”, “почта подтверждена”, “имя”, “фамилия”, “город”, “университет” связь с таблицей виды событий для отображения интересов пользователя.
0-5-BB-00002	Данные, отправляемые пользователем должны валидироваться: правильность написания почты, совпадение паролей.
0-5-BF-00002	Страница регистрации должна содержать кнопки “зарегистрироваться” и “авторизация через сторонний сервис”, которые соответственно иницируют процедуру регистрации и авторизации через другой сервис.

0-7-CFF-00001	После регистрации пользователю открывается страница с редактированием профиля: выбором интересов, указанием более подробной информации о месте обучения и пр.
0-6-BF-00001	Форма авторизации должна содержать поля “логин”, “пароль”).
0-6-BX-00001	Должна быть интегрирована авторизация через портал госуслуг и соцсетей (“Вконтакте”, “Facebook”).
0-6-BF-00002	Страница авторизации должна содержать кнопки “войти” и “авторизация через сторонний сервис”, которые соответственно инициируют процедуру авторизации при помощи данных из бд и авторизации через другой сервис.
0-1-BF-00001	При первом открытии приложения появляется onboarding: несколько картинок из приложения с основными страницами и текстом-описанием разделов.
0-1-A-00001	Необходимо хранить куки пользователей для того, чтобы запоминать, кому уже был показан onboarding, а также для хранения настроек.
0-2-A-00001	При входе в приложение открывается основная страница - лента событий, отранжированная по новизне.

0-3-A-00001	При входе в приложение авторизованным пользователем открывается основная страница - лента событий, отранжированная по новизне и интересам конкретного пользователя.
0-4-A-00001	Не чаще, чем раз в сутки, пользователю при повторном входе должно показываться рорир-окно с предложением оценить работу предложения.
0-4-A-00002	В файлах куки необходимо хранить информацию о том, оценил пользователь приложение или нет, и когда последний раз было показано рорир-окно.

Требования к пользовательскому профилю

Идентификационный номер	Требование
1-1-BF-00001	В интерфейса приложения должна быть стандартная кнопка перехода на страницу профиля.
1-2-BF-00001	Интерфейс должен быть разделен на вкладки “персональная информация” и “статистика”.
1-2-BF-00002	Страница профиля пользователя должна показывать персональную информацию (“персональная информация”) пользователя (см. 1-4-BF-00001), а также информацию о посещенных событиях и активности в приложении (“статистика”).
1-3-BF-00001	Страница редактирования профиля должна содержать кнопку “редактировать”.

1-3-BF-00002	Кнопка “редактировать” открывает форму для заполнения /редактирования персональных данных пользователя
1-4-BF-00003	Кнопка “сохранить” отправляет данные в базу и закрывает форму редактирования.
1-4-BF-00001	Страница с профилем пользователя должна предоставлять возможность заполнения/редактирования полей о себе: интересы, имя, фамилия, город, университет.
1-4-BB-00001	Отправленные со страницы редактирования профиля данные должны добавляться/обновляться в таблице с пользователями

Требования к процессу регистрации новых менеджеров

Входные данные:

- описание сценариев для модерирования заявок для добавления событий
- описание архитектуры системы
- «План разработки ПО»
- «Стандарты на разработку требований к ПО»

Идентификационный номер	Требование
8-1-BF-00001	В CRM при авторизации на аккаунте, которому предоставлена роль администратора, должна существовать опция “Добавить менеджера”.
8-1-BF-00002	Выбор опции “Добавить менеджера” должен открывать форму для заполнения профиля менеджера.

8-1-BF-00003	Форма должна содержать следующие поля для ввода данных: • ФИО менеджера • Уровень доступа (менеджер/администратор) • Адрес электронной почты
8-1-BF-00004	Форма должна быть оснащена инструментами для автоматической проверки соответствия введённой информации установленным условиям, таким как: • Отсутствие запрещённых символов в ФИО (только кириллица) • Формат адреса электронной почты
8-1-BF-00005	На странице формы должна находиться кнопка “Добавить менеджера”.
8-1-BF-00012	При нажатии кнопки “Добавить менеджера”, если данные введены корректно, форма закрывается, и на экран администратора выводится сообщение о успешном завершении регистрации нового менеджера.
8-1-BF-00013	Если данные в заявке введены некорректно, при нажатии кнопки “Добавить менеджера” система должна вывести сообщение о ошибке и указать на проблемные поля формы.
8-1-CFB-00007	После успешного заполнения администратором формы запрос, содержащий всю введенную им информацию, отправляется на серверную часть сервиса для последующей обработки.
8-2-BB-00008	Для каждого запроса на добавление менеджера на сервере должна случайным образом генерироваться пара логин-пароль.
8-2-CBD-00009	После создания пары логин-пароль она вместе с информацией из запроса должна быть сохранена в базе учетных записей менеджеров.

8-2-CBX-00010	После успешного создания учетной записи сервером должна ставиться задача по отправке на указанный в ней адрес электронной почты автоматического уведомления, содержащего в себе логин и пароль для входа в эту учетную запись.
8-3-A-00011	Если введенная при входе в CRM пара логин-пароль соответствует учетной записи менеджера, пользователю должен быть предоставлен расширенный функционал CRM для осуществления управления сервисом.

Требования к процессу написания отзывов

Входные данные:

- описание сценариев для обработки репортов со стороны пользователей
- описание архитектуры системы
- «План разработки ПО»
- «Стандарты на разработку требований к ПО»

Идентификационный номер	Требование
5-1-BF-00001	В окне события после информации о нем идет блок «Отзывы». В его начале справа указана усредненная оценка по 5тибальной шкале
5-1-BF-00002	В блоке показаны отзывы, отсортированные по дате по новизне. Для каждого из них есть • оценка по 5тибальной шкале • логин пользователя • текст отзыва • (прикрепленные материалы - фото и/или видео)
5-1-BF-00003	Для каждого отзыва есть кнопки «Нравится», «Не нравится» и «Пожаловаться»
5-2-BF-00004	После открытия события есть кнопка «Оставить отзыв»

5-2-BF-00005	Открывается окно, в котором находится • текстовое поле для подробного описания с возможностью прикрепить фото/видео • 5тибальная шкала для оценки • кнопка «Отправить отзыв»
5-3-BF-00006	Открывается окно с анкетой для заполнения в виде формы с вопросами и списком возможных ответов, в конце расположена кнопка «Отправить ответ»
5-4-BF-00007	После нажатия кнопки «Пожаловаться» открывается окно со списком возможных причин жалобы («Спам», «Оскорбление», «Враждебное высказывание», «Обман /Провокация») и кнопкой «Отправить жалобу»
5-1-BB-00008	После открытия события в базу данных отправляется запрос на отображение отзывов данного события
5-2-BB-00009	После нажатия кнопки «Отправить отзыв» его данные отправляются в соответствующую базу данных
5-2-BB-00010	В сервис релевантности отправляется оценка из отзыва и связанные с событием данные (теги)
5-3-BB-00011	В сервис релевантности отправляется оценка из анкеты и связанные с событием данные (теги)

Требования к процессу написания репортов

Входные данные:

- описание сценариев для обработки репортов со стороны пользователей
- описание архитектуры системы
- «План разработки ПО»
- «Стандарты на разработку требований к ПО»

Идентификационный номер	Требование
6-A-BF-00001	После открытия события есть кнопка «Сообщить об ошибке»
6-A-BF-00002	Открывается окно с формой со списком возможных причин жалобы («Некорректная информация», «Плохая организация», «Некорректное/неактуальное предложение» «Манипуляция/провокация», «Другое»), текстовым полем для подробного описания проблемы и кнопкой «Отправить жалобу»
6-A-BB-00003	Отправка в список репортов CRM с полями «Дата», «Номер», «Отправитель», «Событие», «Тема», «Сообщение»
6-A-BB-00004	Поля «Отправитель» и «Событие» соответствуют идентификационным номерам пользователя и события
6-A-BB-00005	Поле «Дата» имеет формат ДД.ММ.ГГГГ ЧЧ:ММ:СС
6-A-BB-00006	Поле «Тема» представляет собой номер закодированной причины жалобы

Требования к представлению основной ленты событий

Идентификационный номер	Требование
2-1-BB-00001	При открытии ленты новостей должно запускаться автообновление
2-1-BF-00002	События общей ленты должны сортироваться автоматически по дате

2-2-CFB-00003	Если пользователь находится в начале ленты и “оттягивает” ленту вниз , должна устанавливаться связь с БД событий и получение актуализированных данных
2-2-BF-00004	При пролистывании ленты вверх должна появляться подсказка-кнопка - «переместиться в начало»
2-2-BB-00005	При отображении ленты должен быть лимит загрузки, если ленту прокручивают больше лимита - получение дополнительных данных из БД событий
2-2-BF-00006	Когда доходит до конца ленты, должна отображаться загрузка данных, и после окончания - отображение
2-3-BB-00007	При сортировке - достижение лимита/загрузка более старых/новых объявлений - необходимо сохранить выбранную сортировку
2-3-BB-00008	Сортировка должна сохраняться временно, и очищаться долгосрочно
2-3-BF-00009	Сортировку необходимо проводить только по одному из параметров(радиогруппа)
2-3-BB-00010	При выборе сортировки должно приниматься одно значение, которое характеризует по какому полю будет произведена сортировка

2-3-BF-00011	При выборе «сортировке по» необходимо закрывать окно выбора и образить обновленную персонализированную ленту
2-3-BB-00012	При выборе «сортировке по» необходимо актуализировать персонализированную ленту с учетом сортировки
2-4-BB-00013	При выборе фильтра /нескольких фильтров должны передаваться бинарные значения для каждого из возможного фильтра
2-4-CBF-00014	При открытие фильтра необходимо обеспечить множественный выбор фильтров (чекбоксы)
2-4-BF-00015	Кнопки “убрать” / ”применить” необходимо применять ко всем выбранным фильтрам
2-4-BF-00016	После нажатия кнопки «применить» открывается лента
2-5-BB-00017	После нажатия кнопки «применить» запускается обновление ленты с учетом фильтров
2-6-BF-00018	При нажатие на любое объявление - должно открываться карточка события на весь экран

Требования к процессу поиска событий по тегам или фильтру

Идентификационный номер	Требование
-------------------------	------------

4-1-BB-00001	Необходимо своевременная актуализация часто вводимых запросов по событиям, скидкам, тегам из БД событий
4-2-BB-00002	Фильтры должны иметь множественный выбор
4-2-CBF-00003	Если после выбора фильтров и поиск не дает результатов — отобразить то , что выдает по введенному поиску. Если по нему ничего нет - отобразить по фильтрам.
4-3-BF-00004	При нажатии на любой из предложенных часто вводимого варианта поиск должен закрываться и открываться результат поиска
4-3-BB-00005	При выборе предложенного часто вводимого запроса происходит запрос в БД событий для формирования результата поиска
4-3-BF-00006	При выборе конкретного события /скидки открывается на весь экран пользователя
4-3-BB-00007	При выборе конкретного события /скидки должен производиться запрос в БД событий для получения информации о данном событии

Требования к представлению персонализированной ленты событий

Идентификационный номер	Требование
-------------------------	------------

3-1-BB-00001	При открытии персонализированной ленты новостей должно запускаться связь с БД событий и получение актуализированных данных
3-1-BF-00002	События персонализированной ленты должны сортироваться автоматически по дате
3-2-CFB-00003	Если пользователь находится в начале ленты и “оттягивает” ленту вниз , должна устанавливаться связь с БД событий и получение актуализированных данных
3-2-BF-00004	При пролистывании персонализированной ленты вверх должна появляться подсказка-кнопка - «переместиться в начало»
3-2-BB-00005	При отображении персонализированной ленты должен быть лимит загрузки, если ленту прокручивают больше лимита - получение дополнительных данных из БД событий
3-2-BF-00006	Когда доходит до конца персонализированной ленты, должна отображаться загрузка данных, и после окончания - отображение
3-3-BB-00007	При сортировке - достижение лимита/загрузка более старых/новых объявлений - необходимо сохранить выбранную сортировку

3-3-BB-00008	Сортировка должна сохраняться во время одного временного периода сессии, и очищаться после окончания
3-3-BF-00009	Сортировку необходимо проводить только по одному из параметров(радиогруппа)
3-3-BB-00010	При выборе сортировки должно приниматься одно значение, которое характеризует по какому полю будет произведена сортировка
3-3-BF-00011	При выборе «сортировке по» необходимо закрывать окно выбора и отобразить обновленную персонализированную ленту
3-3-BB-00012	При выборе «сортировке по» необходимо актуализировать персонализированную ленту с учетом сортировки
3-4-BB-00013	При выборе фильтра /нескольких фильтров должны передаваться бинарные значения для каждого из возможного фильтра
3-4-BF-00014	При открытие фильтра необходимо обеспечить множественный выбор фильтров (чекбоксы)
3-4-BF-00015	Кнопки “убрать” / ”применить” необходимо применять ко всем выбранным фильтрам
3-4-BF-00016	После нажатия кнопки «применить» открывается персонализированная лента

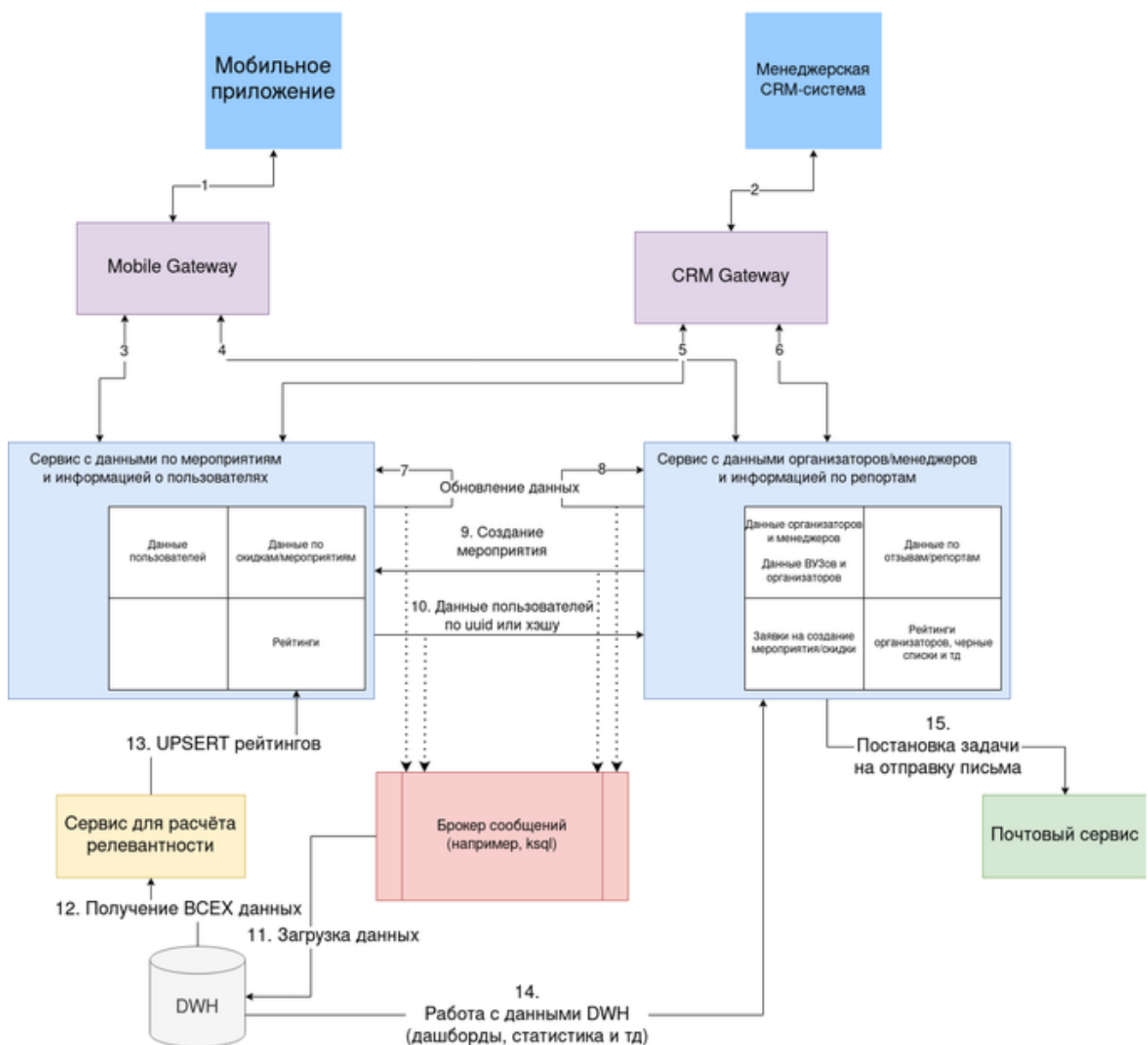
3-5-BB-00017	После нажатия кнопки «применить» запускается обновление персонализированной ленты с учетом фильтров
3-6-BF-00018	При нажатие на любое объявление - должно открываться карточка события на весь экран

Проектирование

Архитектура системы

В данном разделе описана спроектированная по [требованиям к ПО](#) архитектура системы.

Муниципальное приложение для студентов "Город - студентам"



Концепт

Концептуально вся система может быть разделена на три уровня:

- **Уровень пользователя.** Определяет сервисы и компоненты системы, предназначенные для использования пользователем.
- **Уровень публичного API.** Определяет сервисы, предназначенные для предоставления доступа ко внутреннему уровню системы, являющиеся входной точкой для пользовательских приложений.
- **Уровень приватного API и служебных сервисов.** Определяет сервисы, используемые для хранения данных, выполнения бизнес-логики, недоступные обычному пользователю "извне".

Сервисы системы

Мобильное приложение

Мобильное приложение представляет собой основную точку входа в систему для пользователей. Именно оно является главным разрабатываемым продуктом, все остальные сервисы служат для его поддержки.

Через мобильное приложение пользователь работает с лентой событий /скидок, осуществляет запись на мероприятия, просматривает связанную информацию, а также получает функционал по оценке/осуждению предоставляемого контента.

Схема работы приложения.

Менеджерская CRM-система

Менеджерская CRM-система представляет собой продукт для менеджеров и организаторов, который позволяет управлять данными по мероприятиям/скидкам, создавать новые события в системе, администрировать отзывы и репорты пользователей, а также получать аналитику в текстовом и графическом виде.

Схема работы системы.

Mobile Gateway

Mobile Gateway - это сервис-шлюз для предоставления доступа к данным мобильному приложению. Реализует публичный API для работами с такими сущностями как, например, пользователи, события и проч.

Mobile Gateway не хранит состояния и какие-либо данные, а лишь осуществляет предобработку запросов, занимается маршрутизацией запросов по получению данных в нужные сервисы, а также реализует аутентификацию и авторизацию пользователей.

Схема работы сервиса описана в разделе [Mobile Gateway](#).

CRM Gateway

CRM Gateway - это сервис-шлюз для предоставления доступа к данным менеджерской CRM-системе. Реализует публичный API для работами с такими сущностями как, например, отзывы, заявки на создание событий и проч.

CRM Gateway не хранит состояния и какие-либо данные, а лишь осуществляет предобработку запросов, занимается маршрутизацией запросов по получению данных в нужные сервисы, а также реализует аутентификацию и авторизацию пользователей.

Схема работы сервиса описана в разделе [CRM Gateway](#).

Уровень приватного API и служебных сервисов

Сервис с данными по мероприятиям и информацией о пользователях

Данный сервис представляет собой мастер данных по мероприятиям и пользовательской информации. Он включает в себя:

- Данные пользователей, такие как личная информация, пароли (в зашифрованном виде), списки интересов. Осуществляется создание, чтение, изменение и удаление таких данных.
- Данные по скидкам/мероприятиям, например, название, описание, тип, место проведение и прочее. Осуществляется создание, чтение, изменение и удаление таких данных.
- Рейтинги мероприятий. Рейтинги являются зависимым параметром, поэтому осуществляется только чтение.

Схема работы сервиса [Events Backend](#).

Сервис с данными организаторов/менеджеров и информацией по репортам

Данный сервис представляет собой мастер данных по данным организаторов/менеджеров, а также всем связанным с мероприятиями помимо самих мероприятий. Он включает в себя:

- Данные организаторов, такие как имя юридического/физического лица, связь с компаниями/ВУЗами, адрес и т.д. Каждому организатору соответствует одна или более учётных записей для работы с системой, данные о которой также хранятся в сервисе.
- Данные менеджеров, такие как ФИО, логины и пароли в зашифрованном виде, уровни доступа и прочее.
- Данные по отзывам/репортам, такие как содержание отзыва и репорта, ссылки на статическую информацию (фото, видео), численные характеристики (оценки, рейтинги), а также связь с автором по uuid или hash.
- Рейтинги организаторов, чёрные и теневые списки. Черный и теневой списки содержат время и продолжительность блокировки.
- Заявки на создание мероприятий. Сущности, отправляемые организаторами, необходимые для создания мероприятия внутри системы. Содержат, например, название, описание, тип, место проведение и прочее.

Схема работы сервиса *вставить раздел*.

Брокер сообщений

Сервис, обрабатывающий сообщения об изменении данных сервисов (более подробно в описании переходов 7-10), трансформирующий их и перенаправляющий в общее хранилище данных - DWH.

Схема работы сервиса описана в разделе *вставить раздел*.

DWH

DWH - Data Warehouse - хранилище данных, служит для аккумуляции всех данных системы в одном месте. Используется, в первую очередь, для аналитических целей, а также собирает все данные в одном месте, что позволяет использовать их для различных расчётов рейтингов и т.д.

Схема работы хранилища описана в разделе *вставить раздел*.

Сервис для расчёта релевантности

Данный сервис производит расчёты релевантности скидок/событий на основе данных в DWH. Внутри используются аналитические модели, способные, на основе входных параметров, предсказать интерес пользователя к тому или иному мероприятию.

Выходным параметром сервиса является рейтинг каждого отдельного мероприятия, который влияет на формирование пользовательской персонализированной ленты.

Схема работы сервиса описана в разделе *вставить раздел*.

Почтовый сервис

Данный служебный сервис предназначен для неблокирующей отправки почтовых писем с различными уведомлениями пользователей и организаторов.

Межсервисное взаимодействие

Взаимодействие между сервисами на диаграмме отмечено стрелками-переходами от одного компонента в другую. Каждый переход имеет свой номер.

Переход 1. Общение между Мобильным приложением и Mobile Gateway. Цель: осуществление корректной работы мобильного приложения, непрерывный обмен данными.

Переход 2. Общение между Менеджерской CRM-системой и CRM Gateway. Цель: осуществление корректной работы менеджерской CRM системы, непрерывный обмен данными.

Переходы 3-6. Общение Mobile Gateway и CRM Gateway с сервисами приватного уровня. Цель: запись, чтение, изменение и удаление данных, маршрутизация запросов, агрегация данных.

Переходы 7-8. Обновление данных внутри сервиса с данными по мероприятиям и информацией о пользователях и сервиса с данными организаторов/менеджеров и информацией по репортам. Происходит, например, в случае создания нового пользователя или мероприятия - независимые данные обновляются (зависимые, такие как рейтинги, остаются нетронутыми, т.к. пересчитываются извне). Каждое обновление влечёт за собой отправку сообщения об обновлении в брокер сообщений. Цель: запись, чтение, изменение и удаление сущностей, поддержка данных в DWH в актуальном состоянии.

Переходы 9-10. Общение между сервисом с данными по мероприятиям и информацией о пользователях и сервисом с данными организаторов/менеджеров и информацией по репортам. Происходит, например, в случае недостатка данных в одном из сервисов и их наличии в другом. В случае, если такое общение предполагает обновление данных в одном из сервисов, то отправляется сообщение об обновлении в брокер сообщений. Цель: запись, чтение, изменение и удаление сущностей, поддержка данных в DWH в актуальном состоянии.

Переход 11. Загрузка агрегированных данных из сообщений об обновлении данных в DWH. Цель: поддержка данных в DWH актуальном состоянии.

Переход 12. Открытие данных в DWH для чтения сервисом для расчёта релевантности. Цель: бесперерывная работа моделей по расчёту релевантности событий, работа персонализированной ленты в приложении.

Переход 13. Обновление рейтингов внутри сервиса с данными по мероприятиям и информацией о пользователях. Цель: бесперерывная работа моделей по расчёту релевантности событий, работа персонализированной ленты в приложении.

Переход 14. Открытие данных в DWH для чтения сервисом с данными организаторов/менеджеров и информацией по репортам. Данные не “задерживаются” в сервисе, а лишь проходят через него, агрегируются с существующей информацией в самом сервисе и затем передаются в Менеджерскую CRM систему. Цель: предоставление менеджерской CRM системе доступа к данным в DWH.

Переход 15. Служебный переход, определяющий процедуру постановки задачи на отправку письма.

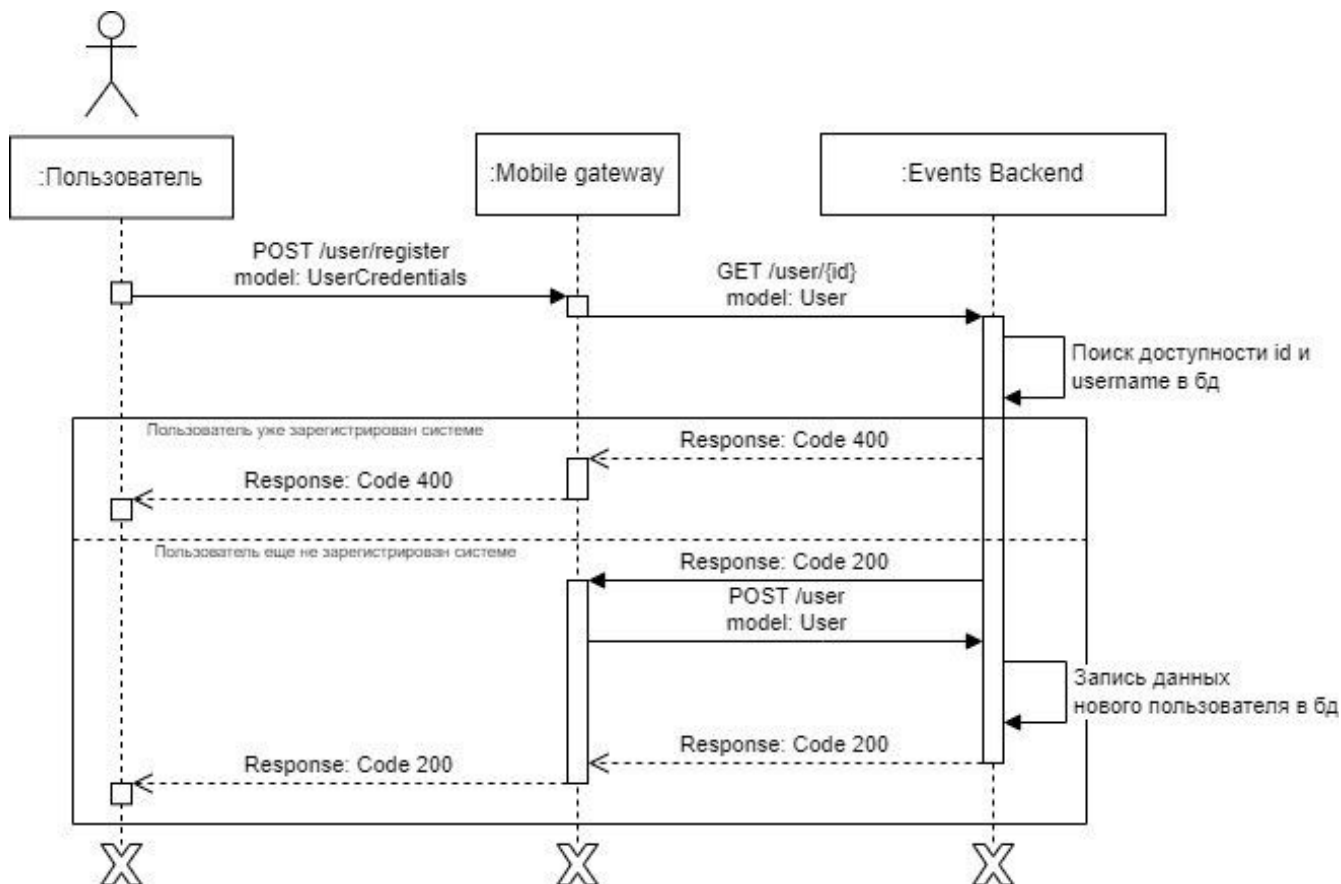
Mobile Gateway

Шлюз для взаимодействия сервиса мобильного приложения с сервисом мероприятий/пользователей.

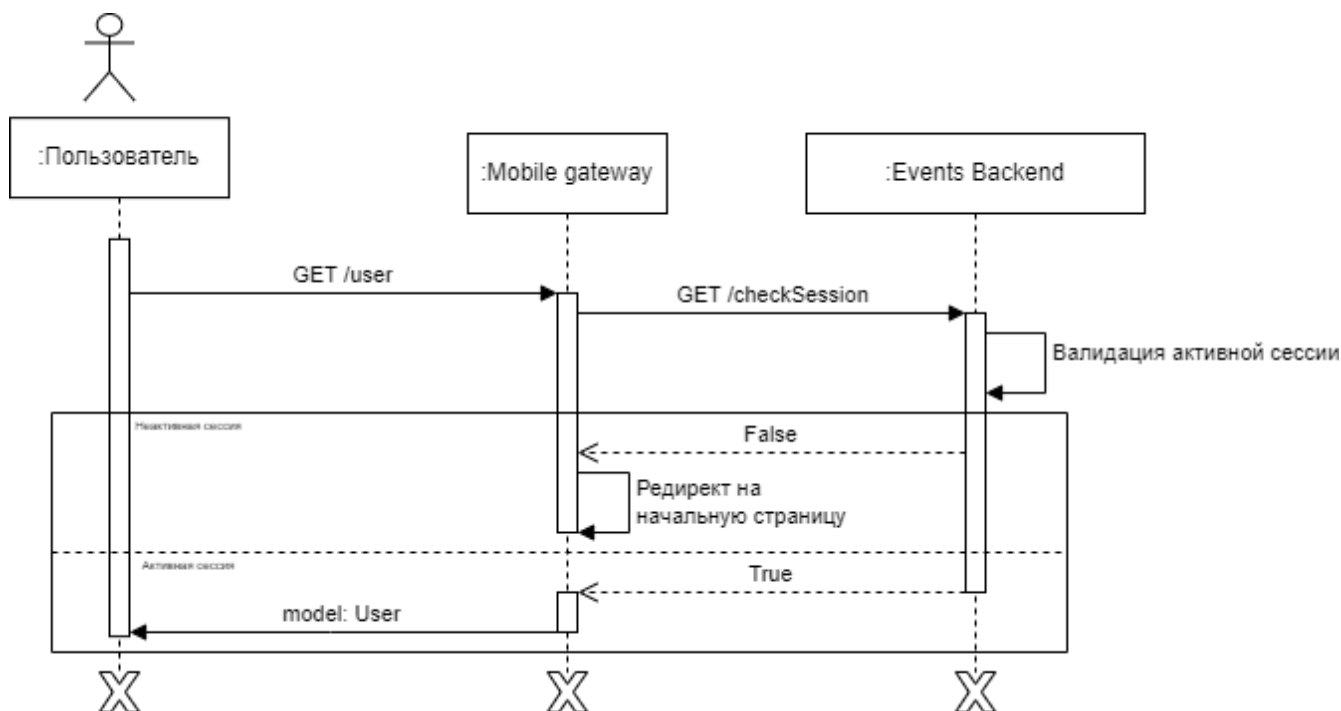
Спецификация API

Функциональные схемы процессов

Запрос на регистрацию пользователя в системе

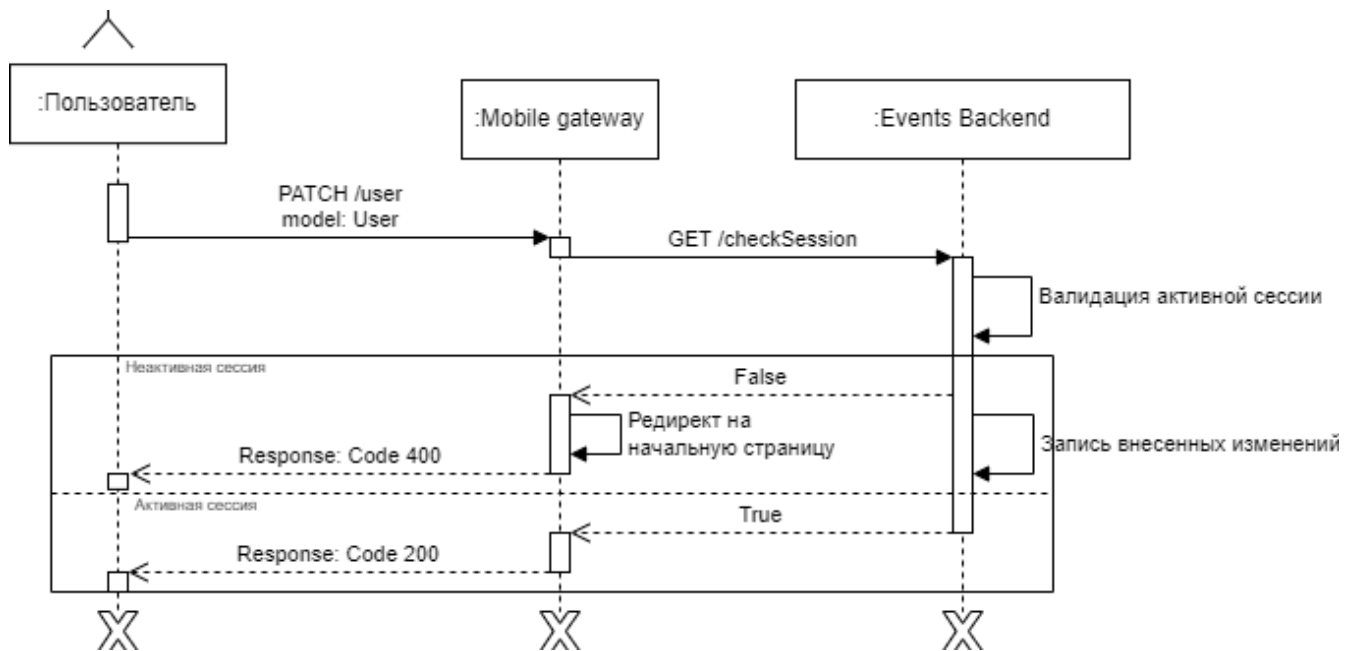


Запрос на получение данных пользователя в системе

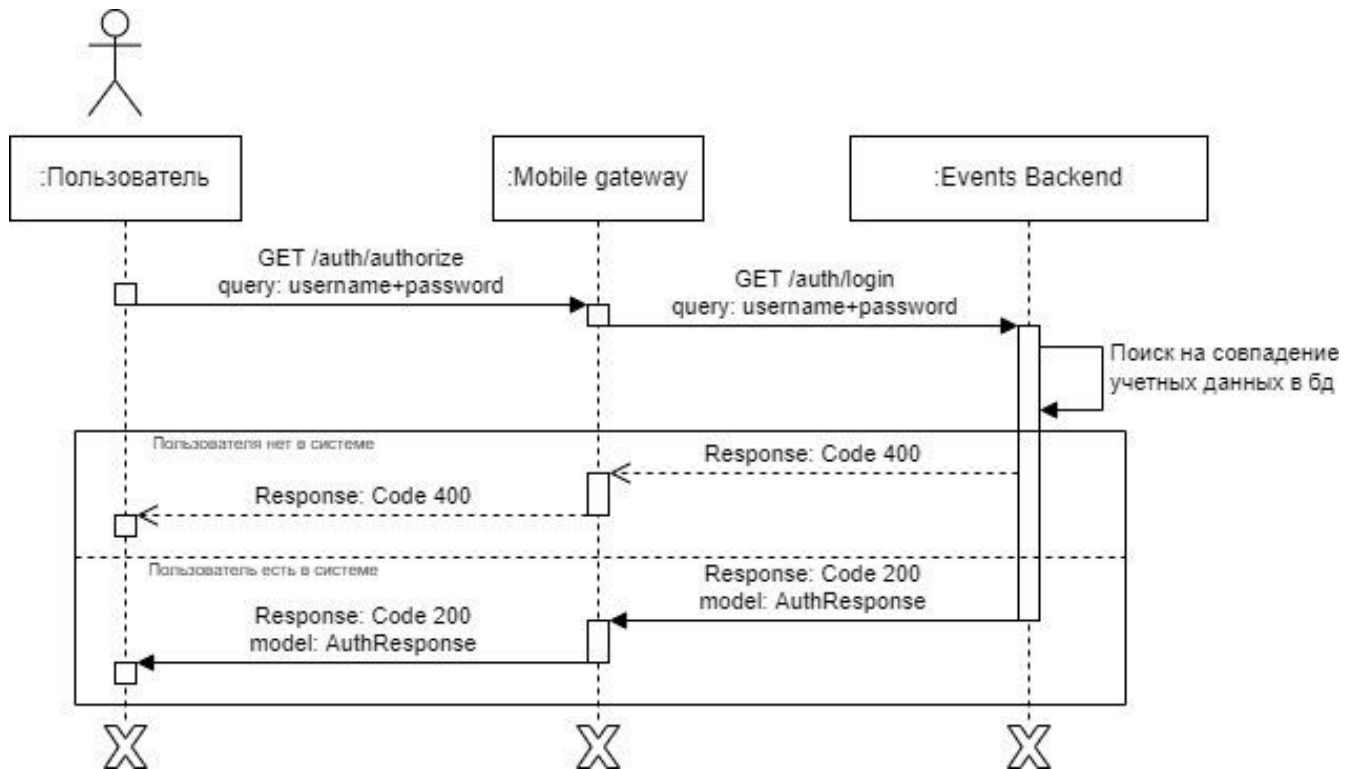


Запрос на внесение измененных данных пользователя в систему

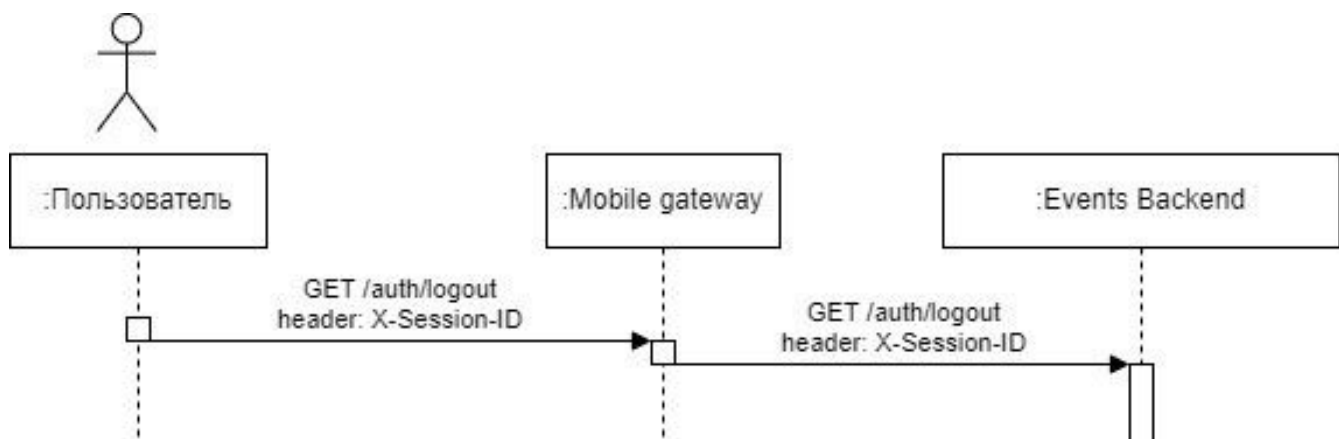


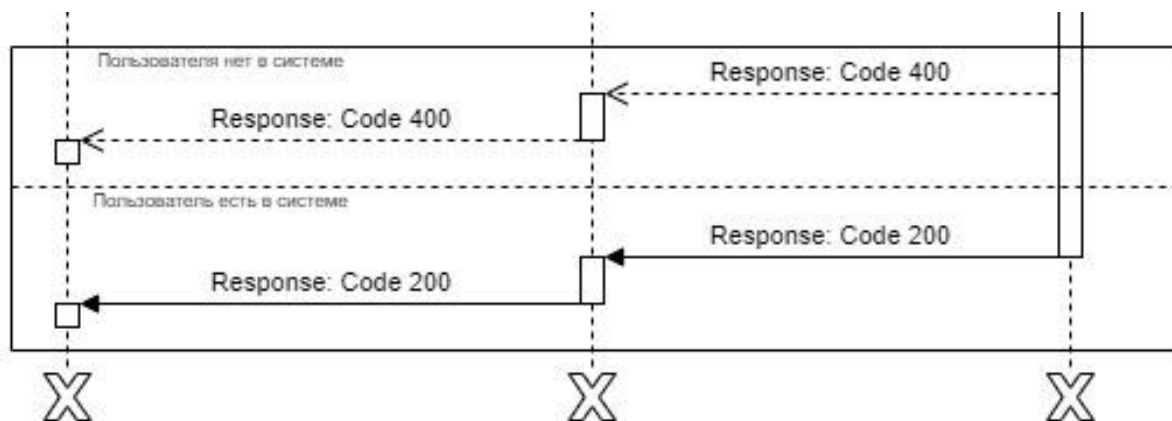


Запрос на авторизацию пользователя в системе

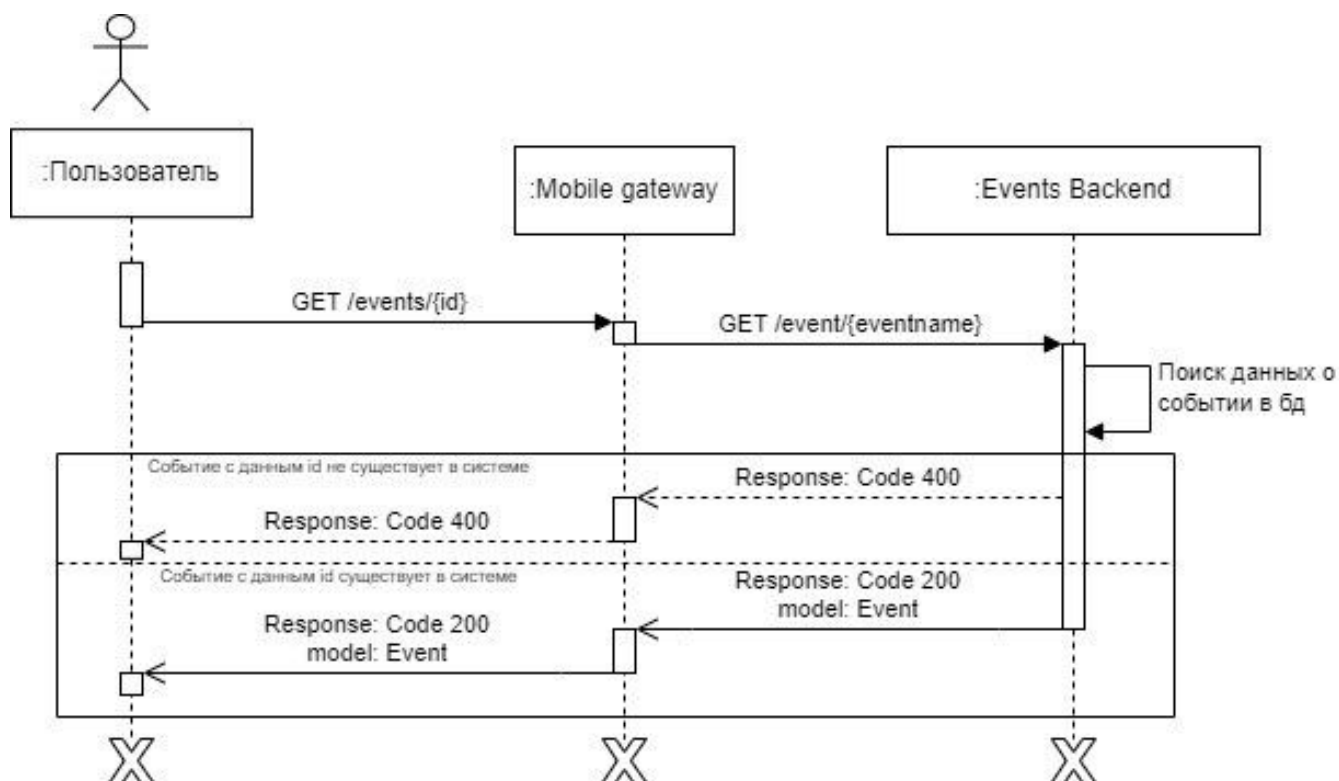


Запрос на выход пользователя из системы





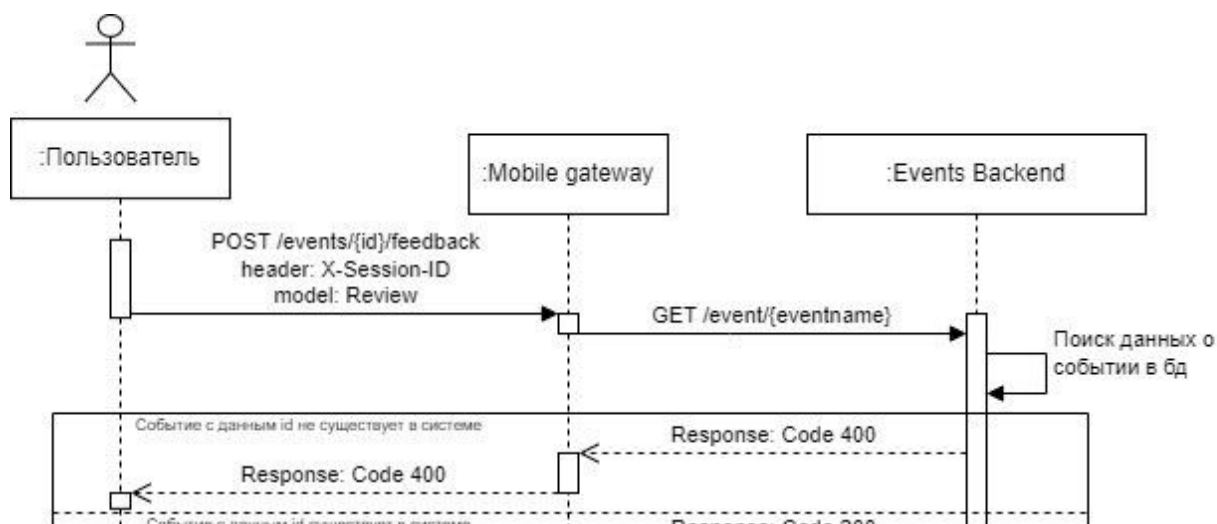
Запрос на получение информации о событии

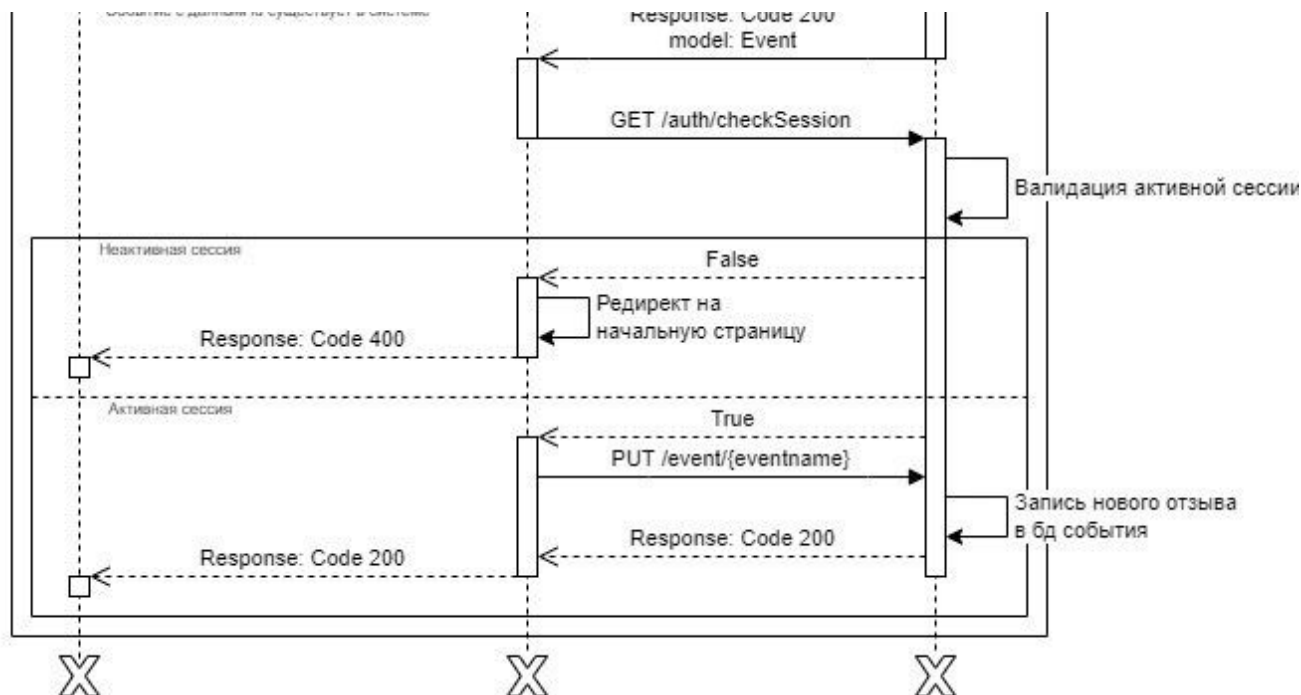


Запрос на получение списка событий

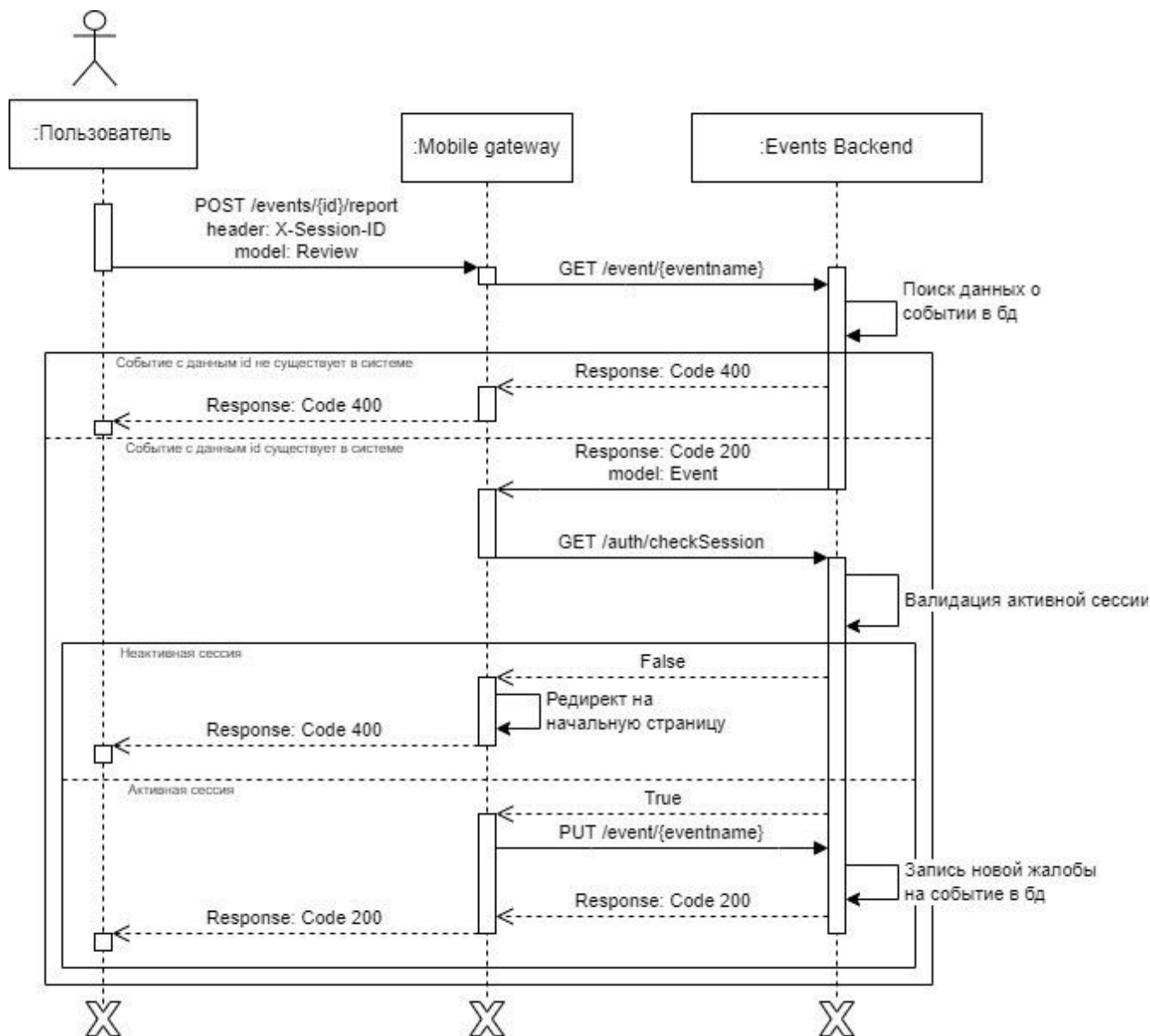
Запрос на получение персонализированного списка событий

Запрос на отправку отзыва о событии





Запрос на отправку жалобы на событие



Запрос на получение информации об организаторе

Запрос на отправку отзыва об организаторе

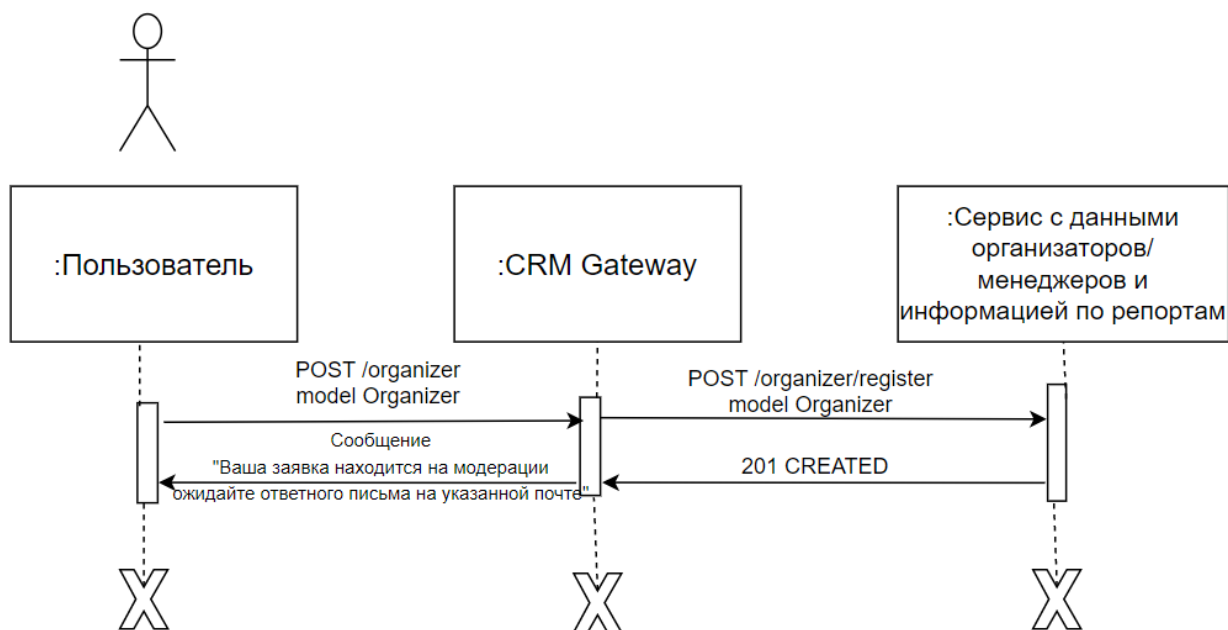
Запрос на отправку жалобы на организатора

CRM Gateway

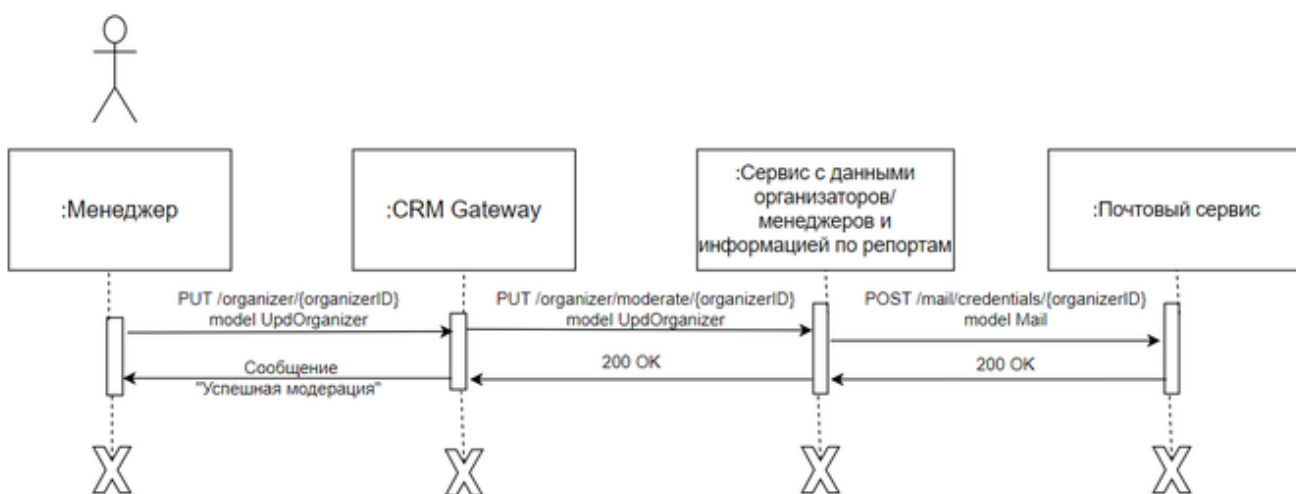
API спецификация

Функциональные схемы процессов

Регистрация организаторов

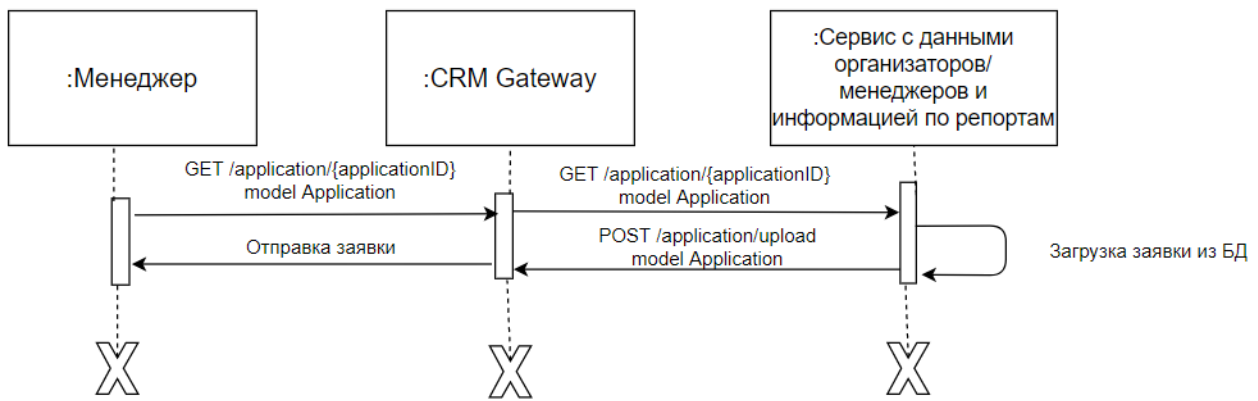


Модерация организаторов

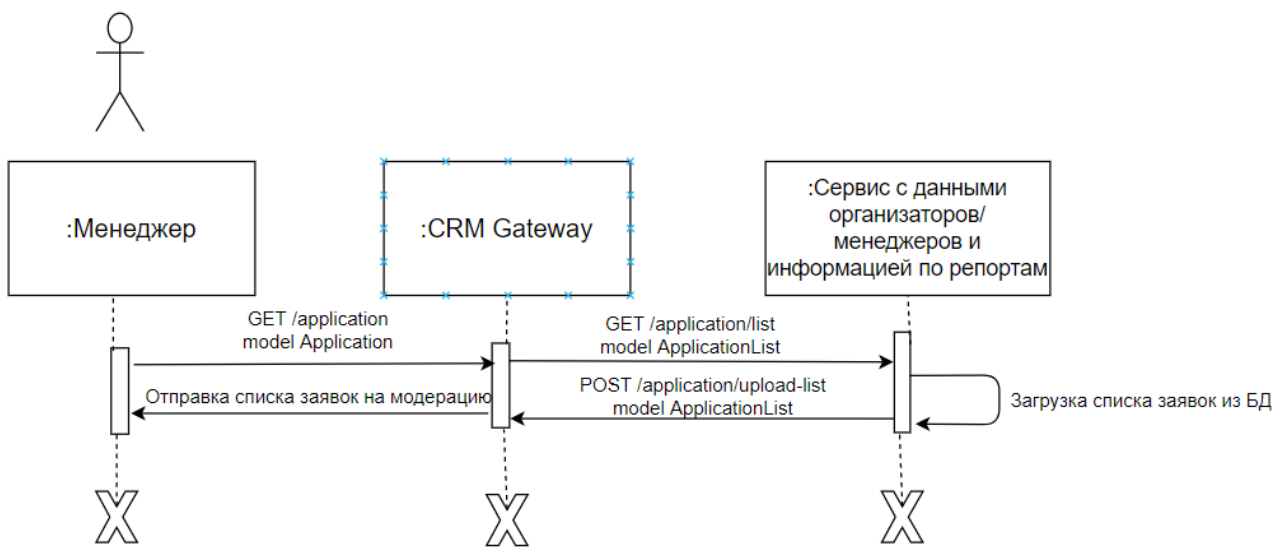


Получение заявки по id

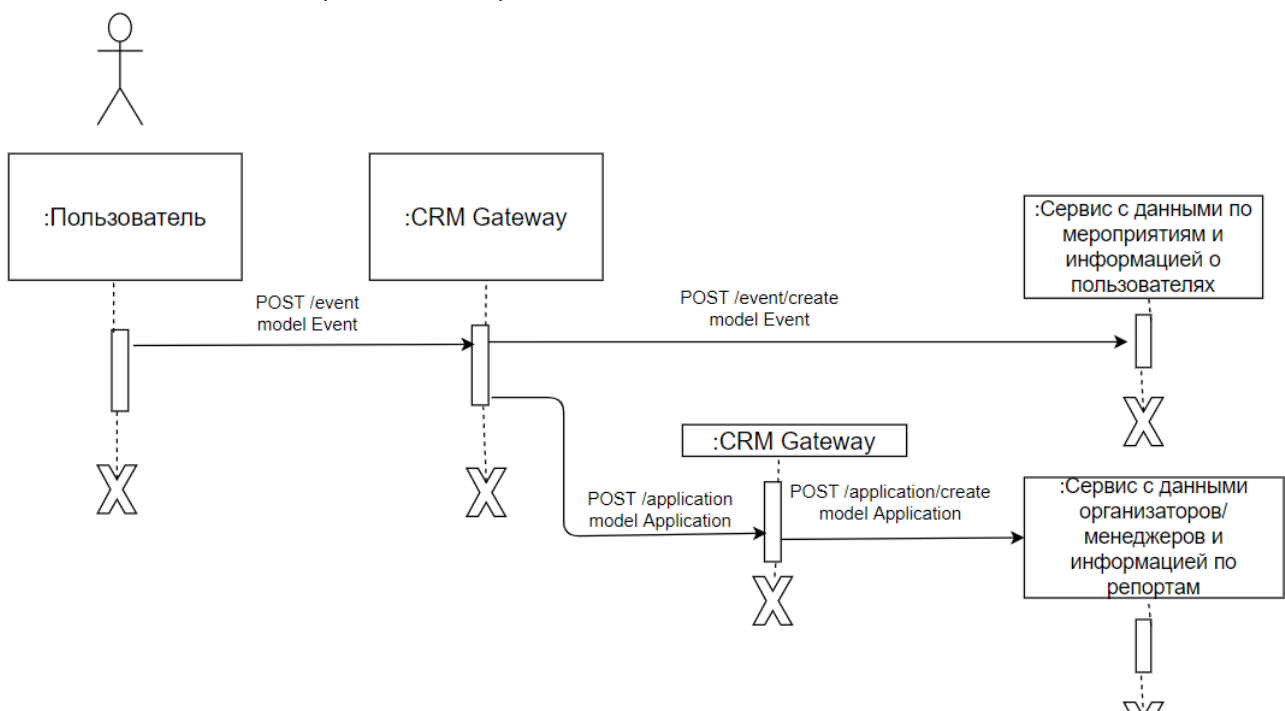




Получение списка заявок

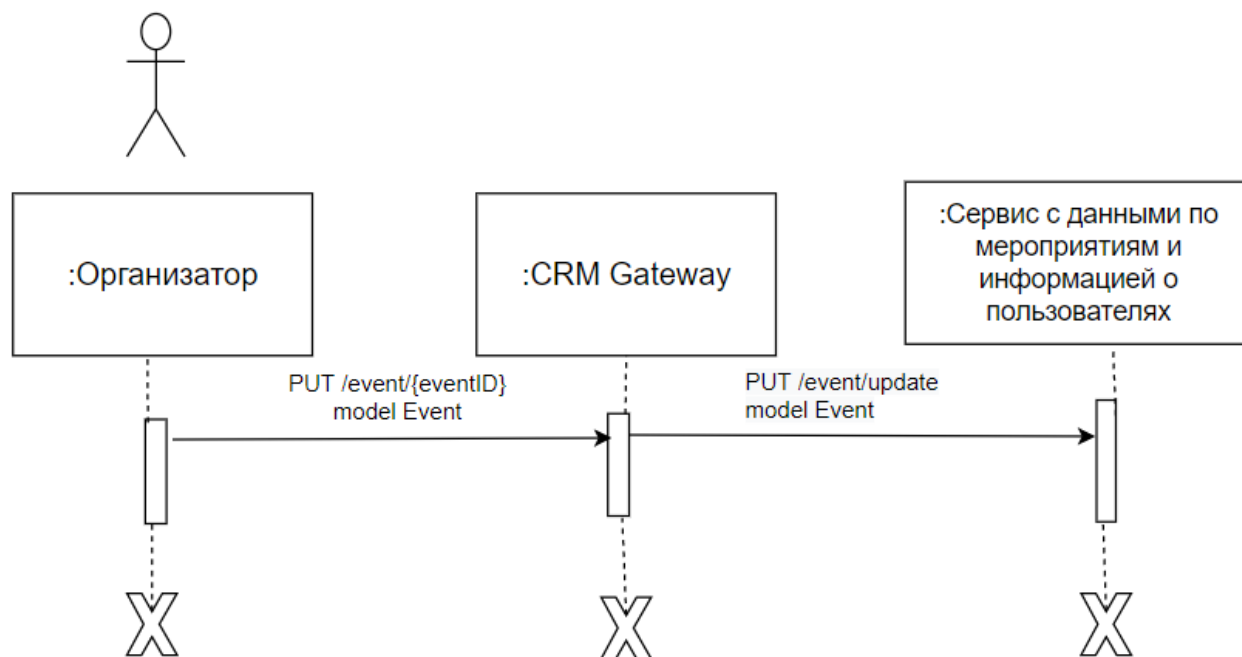


Создание события (с заявкой)

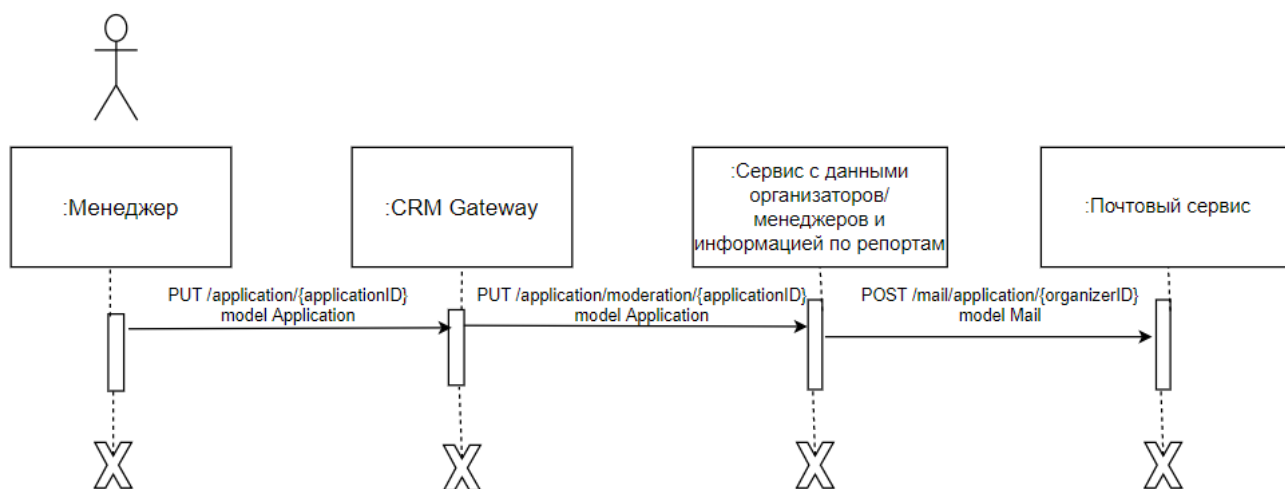




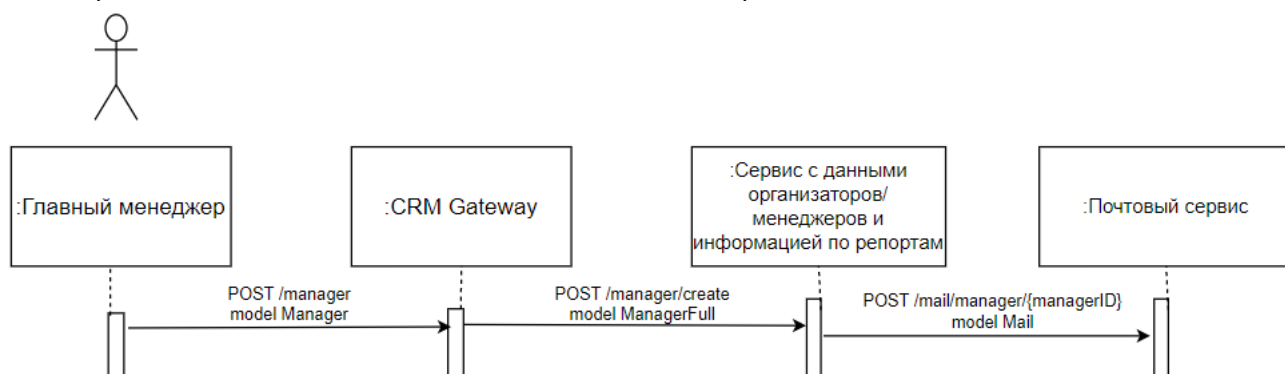
Редактирование события



Модерация заявки по id



Регистрация и добавление новых менеджеров



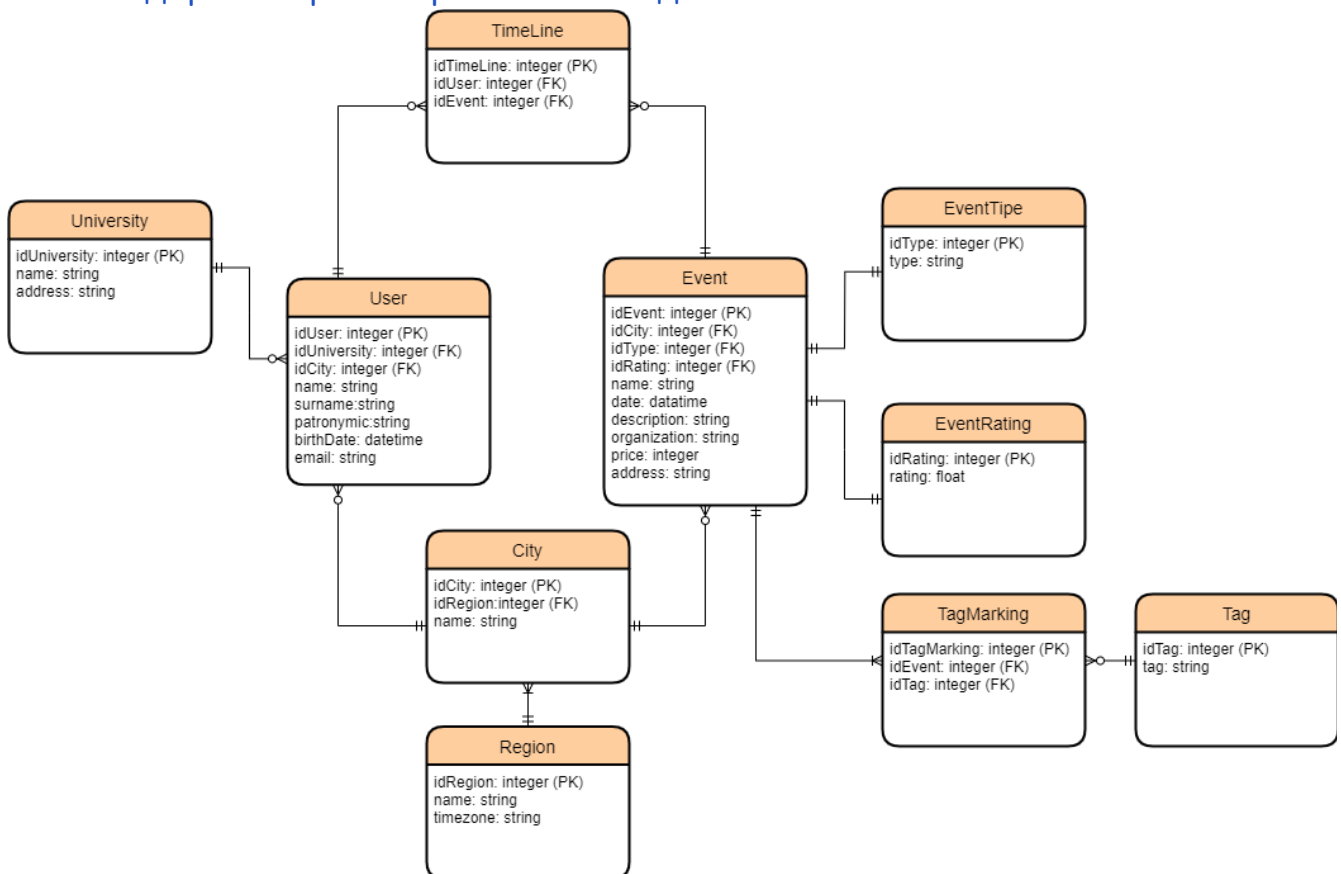


Events Backend. Сервис с данными о мероприятиях и информацией о пользователях

Схема данных

Схема данных сервиса была спроектирована в соответствии со следующими входными данными:

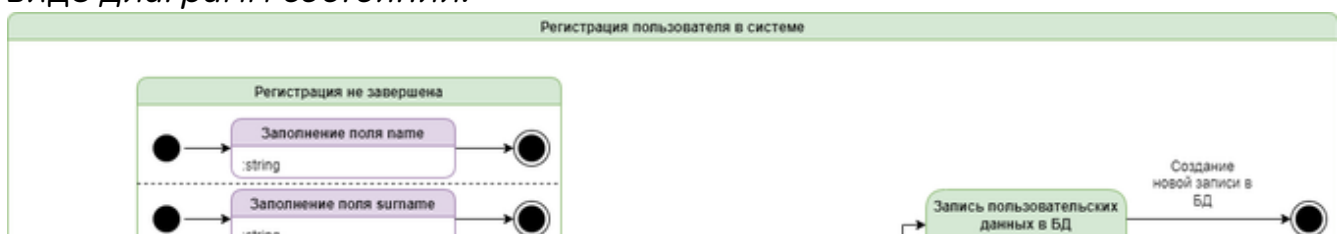
- [Архитектура ПО](#)
- [Системные требования](#)
- [Требования к ПО](#)
- [Диаграмма классов, участвующих в приложении](#)
- [Стандарт на проектирование баз данных](#)

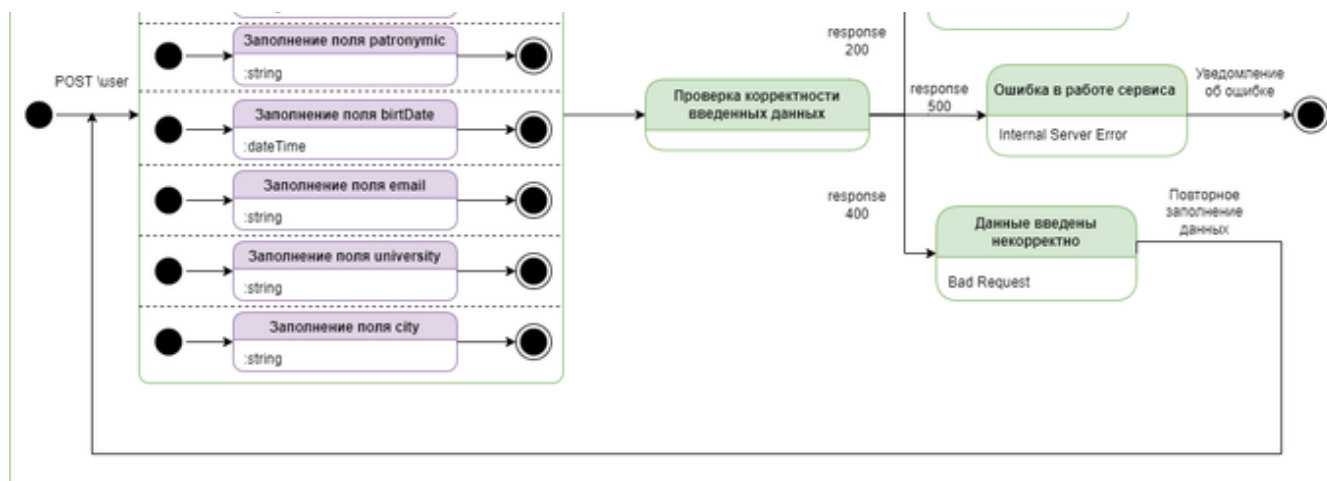


Спецификация API

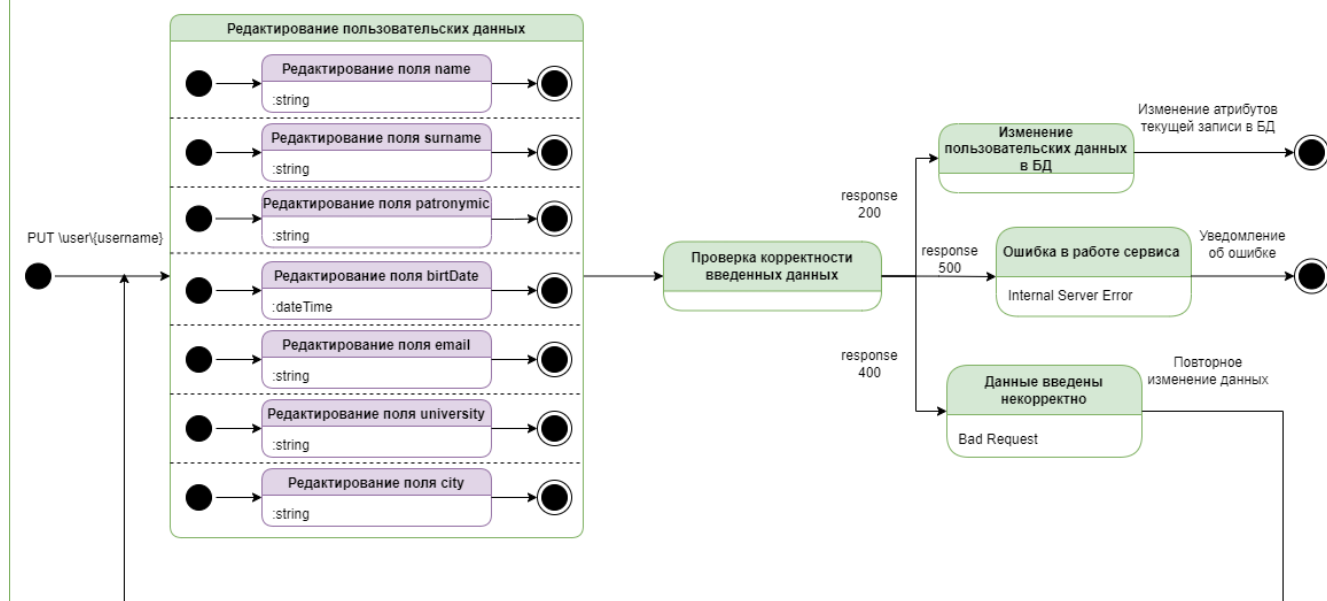
Бизнес-логика процессов внутри сервиса

Схемы процессов по обновлению данных внутри сервиса представлены в виде *диаграмм состояний*:

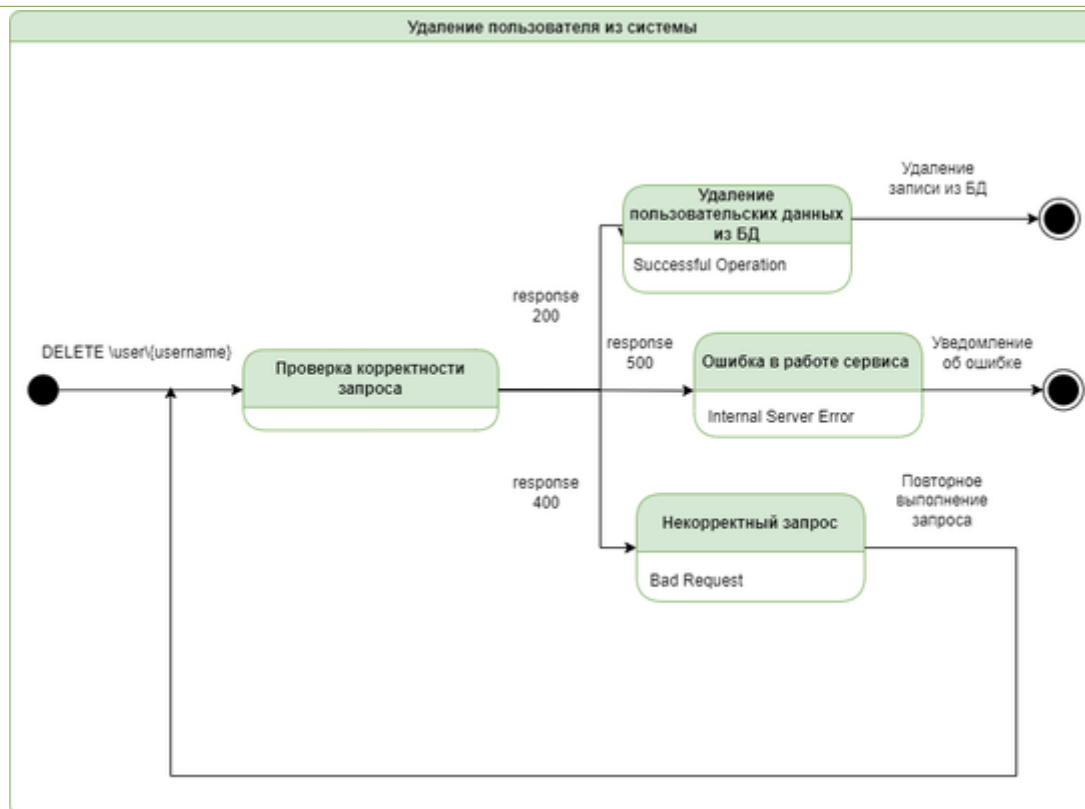




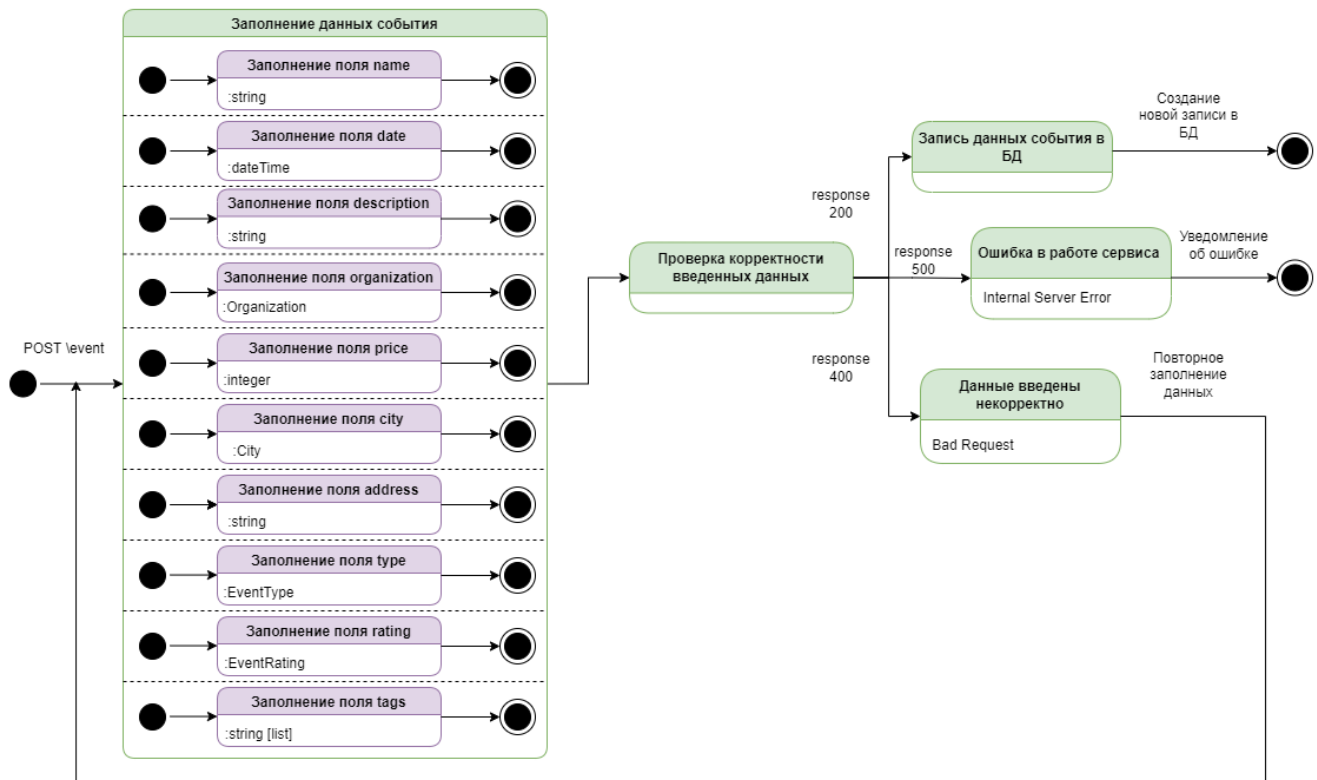
Изменение данных пользователя в системе



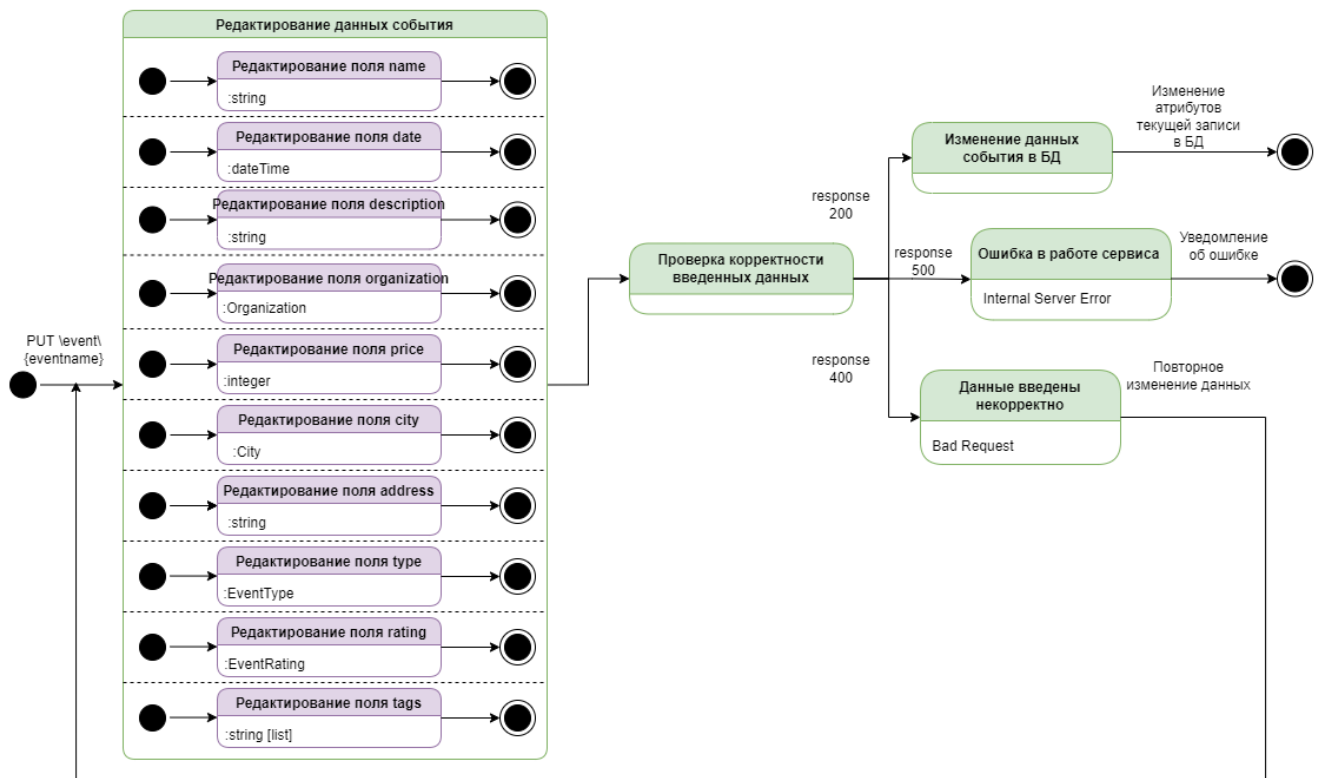
Удаление пользователя из системы



Создание события в системе

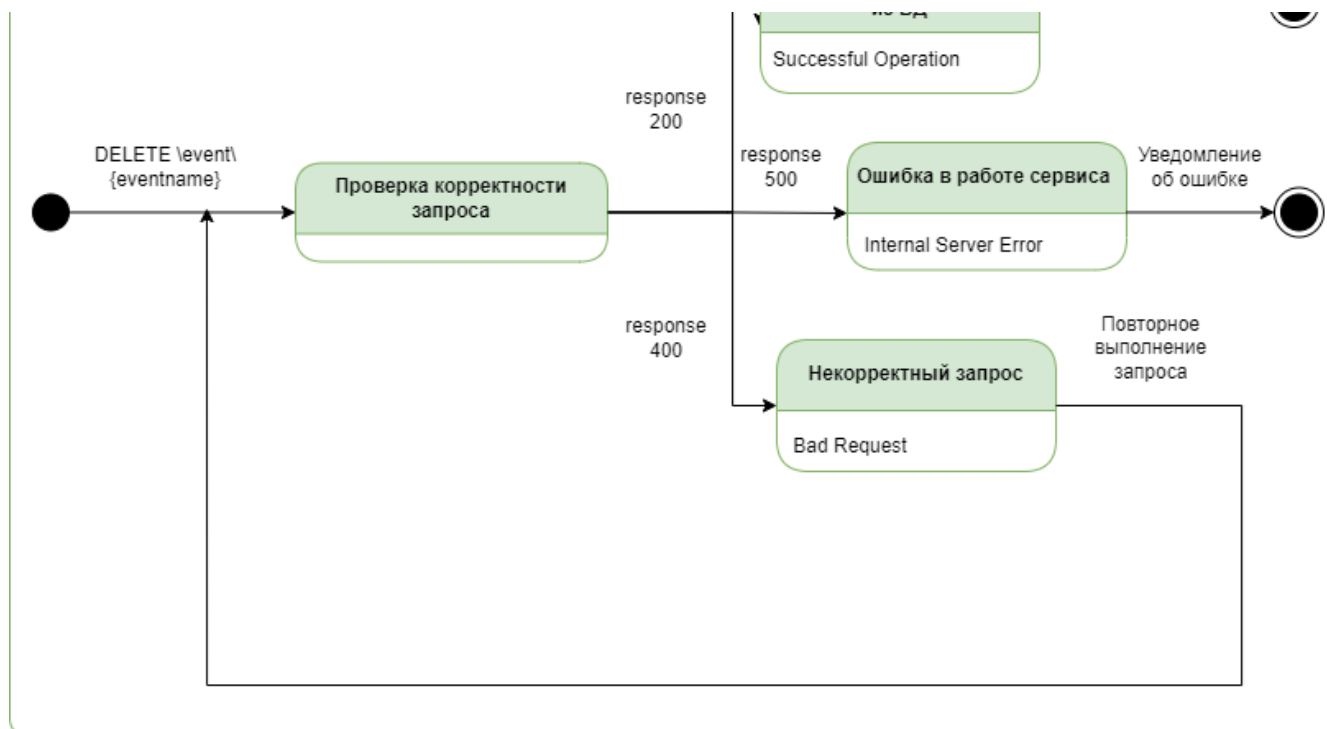


Изменение данных события в системе



Удаление события из системы



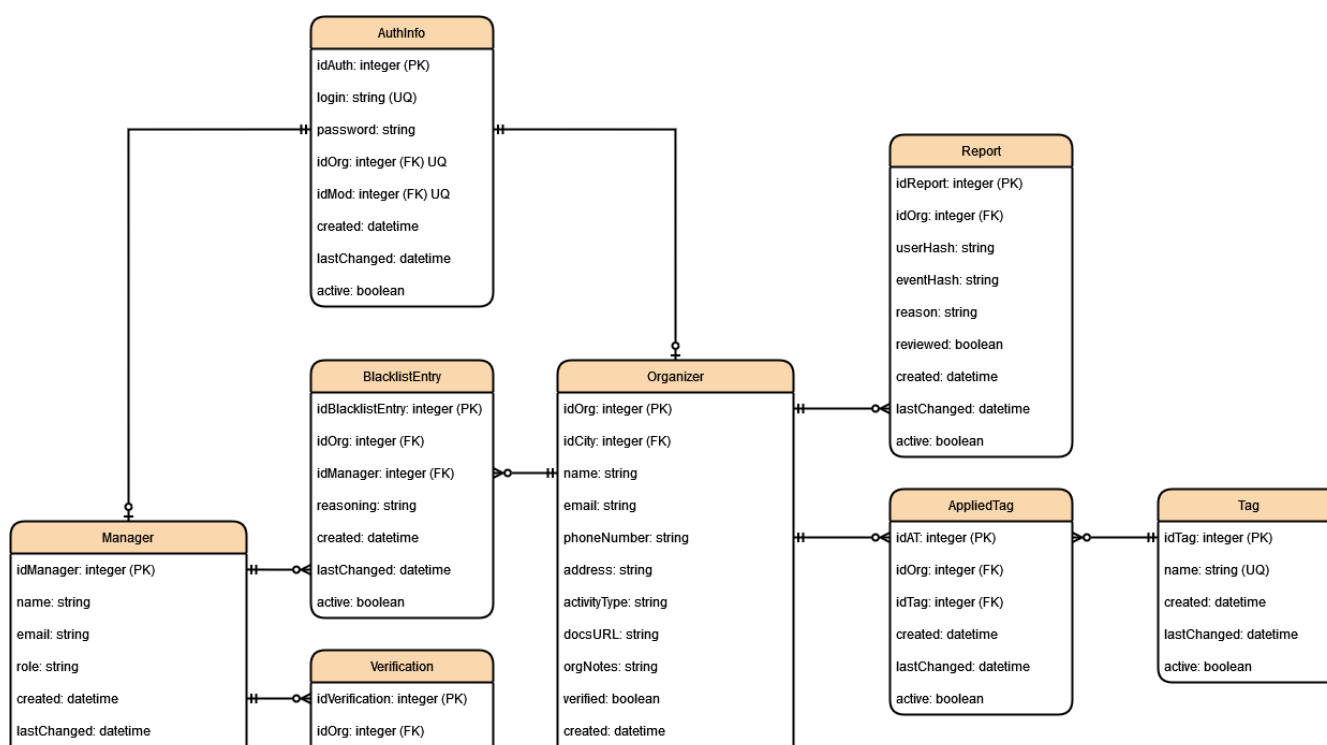


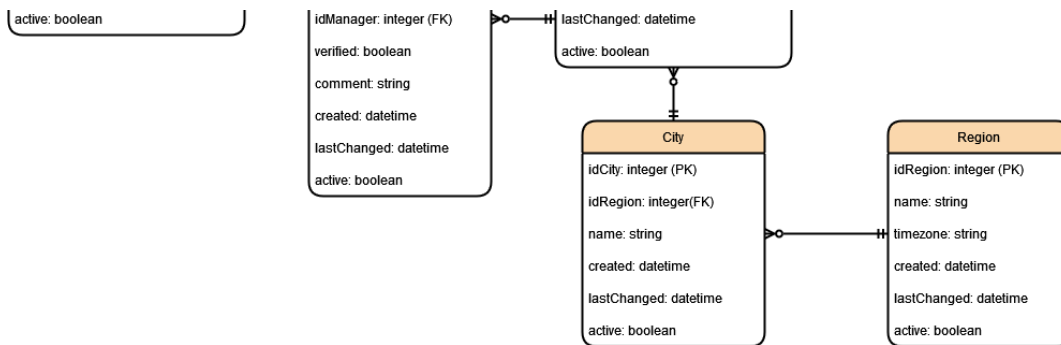
CRM Backend. Сервис с данными об организаторах и информацией о репортах.

Схема данных

Схема данных сервиса была спроектирована в соответствии со следующими входными данными:

- [Архитектура ПО](#)
- [Системные требования](#)
- [Требования к ПО](#)
- [Диаграмма классов, участвующих в приложении](#)
- [Стандарт на проектирование баз данных](#)





Спецификация API

Мобильное приложение

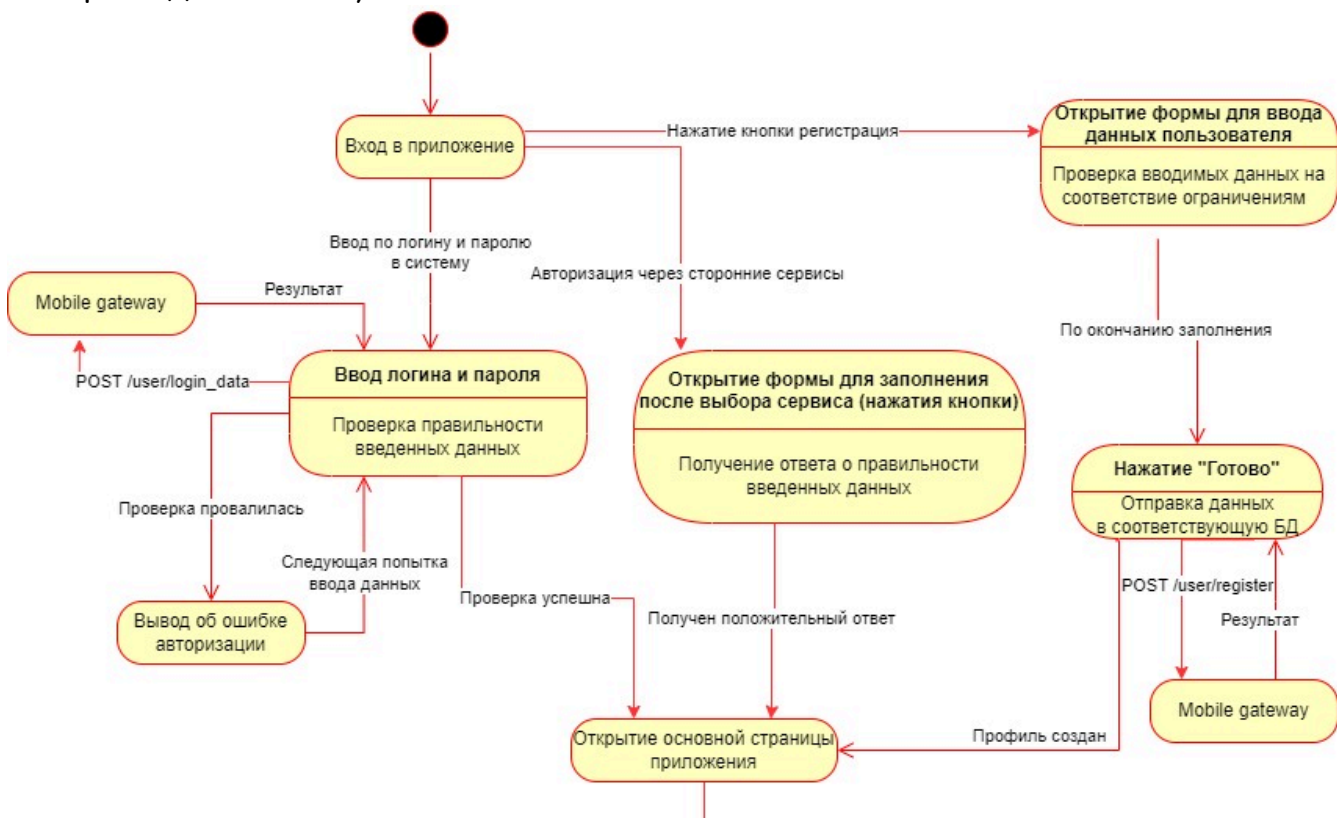
Работа со страницами приложения была спроектирована в соответствии со следующими входными данными:

- [Архитектура ПО](#)
- [Системные требования](#)
- [Требования к ПО](#)

Можно выделить страницы, с которыми пользователь будет взаимодействовать, как стартовую (с авторизацией и регистрацией) и основную (с персонализированной и общей лентами и личным кабинетом)

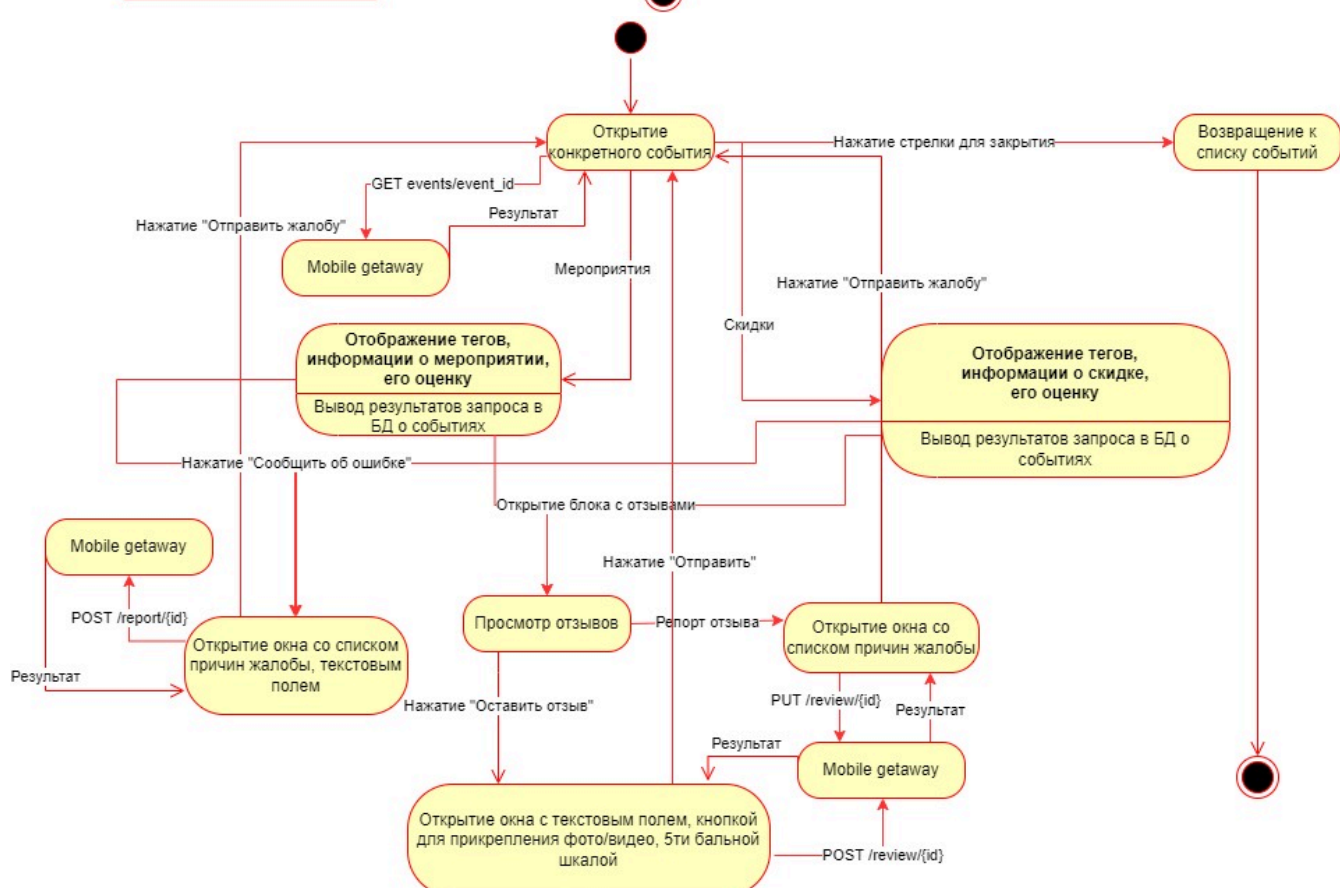
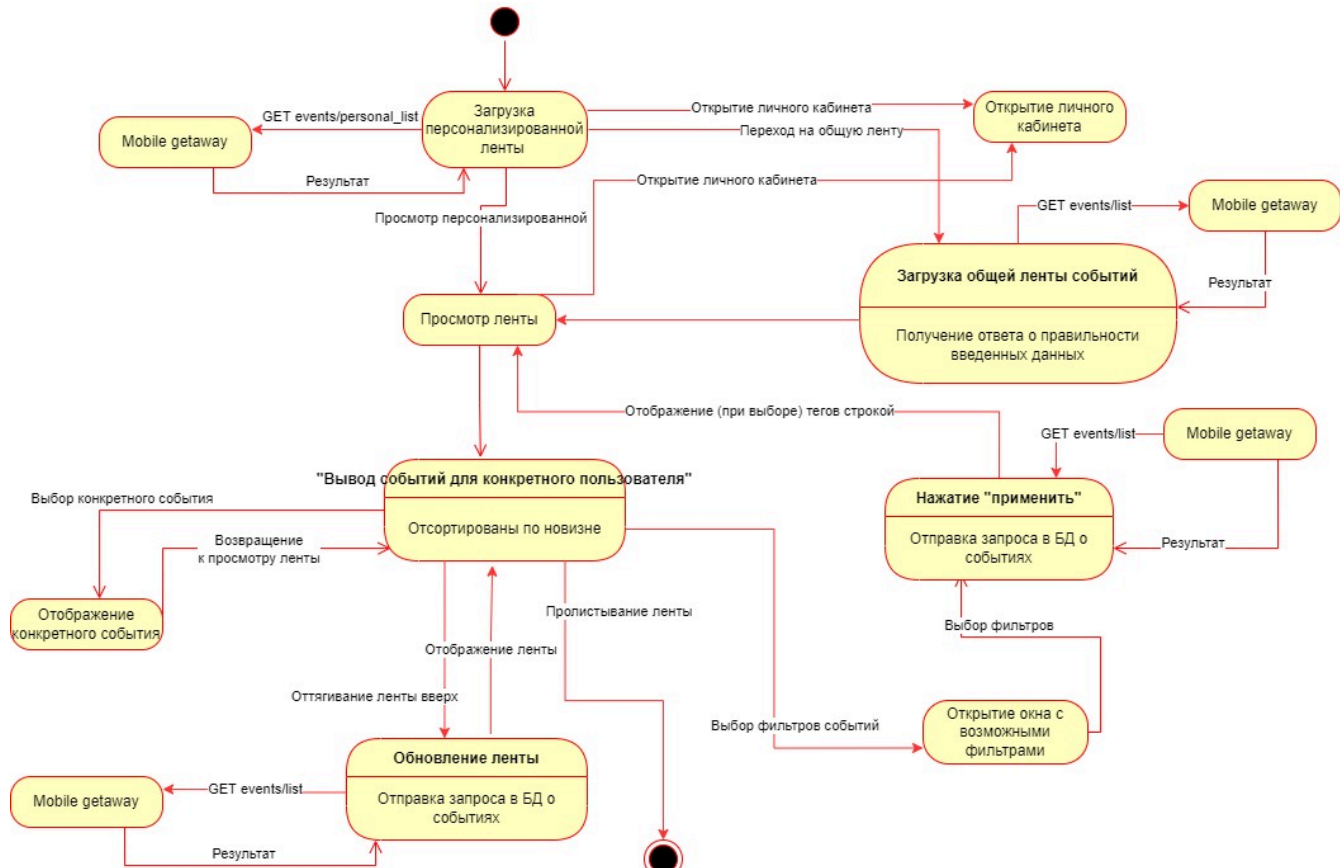
При входе в приложение

- для авторизованного пользователя сразу открывается персонализированная лента
- для неавторизованного - стартовая страница (описание которой приведено ниже)

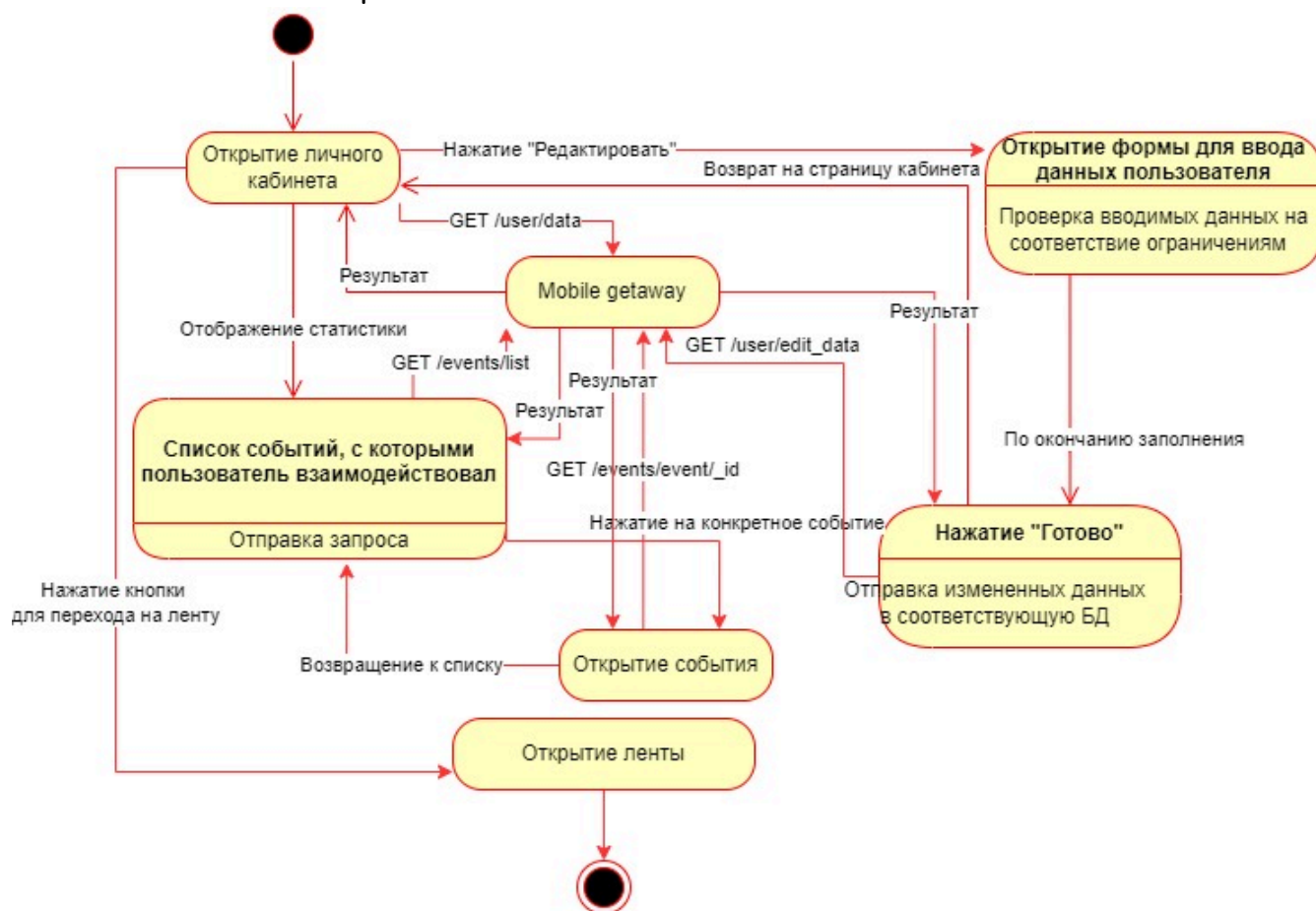




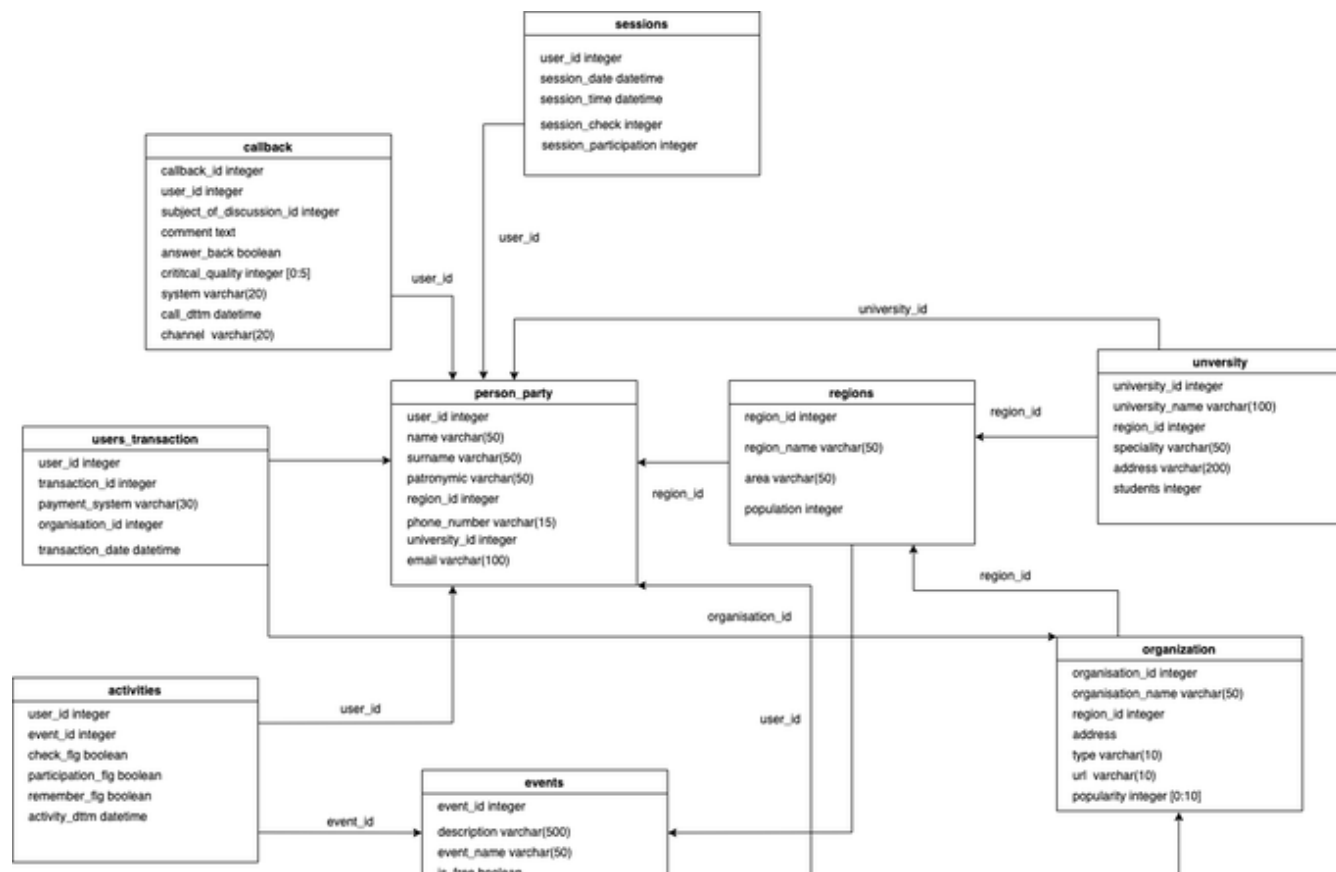
На основной странице расположены 3 раздела: персонализированная лента, общая лента и личный кабинет. Ниже описаны просмотр ленты и открытие событий, расположенных на ней.

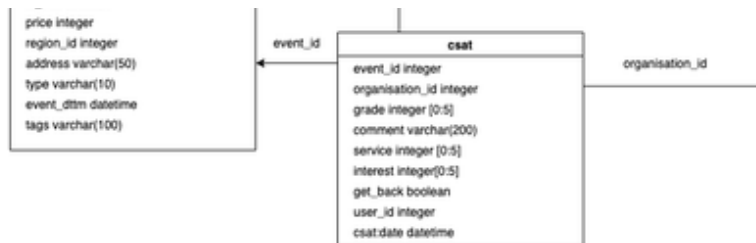


При желании пользователь может изменить свои данные и посмотреть свою активность в приложении в личном кабинете.



DWH





Описание таблиц

Таблица	Краткое описание
Person_party	Информация о клиенте
Regions	Информация о регионах
University	Информация об институтах
Organization	Информация об организациях, проводящие мероприятия
Csat	Информация об оценках клиентов мероприятий и приложения
Events	Информация о событиях
Activities	Информация о действиях клиентов в приложении
User_transactions	Информация о финансовых транзакций клиентов
Callback	Информация об обратной связи клиентов
Sessions	Информация о сессиях клиентов

Откуда берутся данные ?

1. CSAT/callback - из CRM системы
2. user_transaction,activities,sessions - логи приложения
3. person_party,events,regions,university,organization - бэкэнд системы

На основании данных, хранящихся в DWH можно выделить несколько метрик, которые позволят отслеживать необходимые бизнес-задачи:

1. Вовлеченность клиентов в приложении
2. Удовлетворенность клиентов
3. Активность событий

Собираются следующие категории данных :

1. Информация о клиенте, его действиях в приложении, операциях и т. д.
2. Информация о событиях, организациях, регионах и т.д.
3. Информация о обратной связи, оценках

Для определения метрики “вовлеченность клиента” стоит смотреть на следующие параметры :

1. Просмотренные(открытых) событий
2. События, в которых принял клиент участие
3. Время проведенного в приложении
4. Доля студентов в институте с этим приложением

Для определения метрики “удовлетворенность клиентов” необходимо ориентироваться на следующие параметры:

1. жалобы/обращения по отношению к приложению
2. жалобы/обращения по поводу мероприятий
3. Оценки CSAT и детализация

Для определения метрики “Активность событий” необходимо ориентироваться на следующие параметры:

1. транзакции клиентов
2. Воронка проникновения в событие
3. оценки событий
4. жалобы на событий

Примеры запросов для получения статистики и подведения результатов :

Подводится статистика о каждом клиенте, его активность, отзывчивость, участие в мероприятиях, транзакциях. По этим данным, можно отслеживать вовлеченность клиента, исследовать различные зависимости от института студентов и их вовлеченность, проверить различные гипотезы, провести кластеризацию и выдавать более релевантные для различных групп предложения.

```
with events_check as (  
  select count(event_id) as count_check,user_id  
  from activities  
  where check_flg = 1 and activity_dttm between (current_date-31,  
    current_date-1)  
  group by user_id  
),  
with events_part as (  
  select count(event_id) as count_check,user_id  
  from activities  
  where participation_flg = 1 and activity_dttm between (current_date-
```

```

31,current_date-1)
group by user_id
),
sessions as (
select sum(session_time) as session_time,count(session_time) as
session_dates,user_id
from sessions
where session_date::date between(current_date-31,current_date-1)
group by user_id
),
trans as (
select count(trans) as count_transactions,user_id
from user_transaction
where transaction_date::date between(current_date-31,current_date-1)
group by user_id
),
csat as (
select count(event_id) as count_csat_answers,user_id
from csat
where csat_date::date between(current_date-31,current_date-1)
group by user_id),
csat_back as (
select count(event_id) as count_want_back,user_id
from csat
where get_back = 1 and csat_date::date between(current_date-31,
current_date-1)
group by user_id)
select pp.user_id,ec.count_check,ep.count_part,s.session_time,s.
session_dates,t.count_transactions,c.count_csat_answers,cb.
count_want_back
from person_party pp
left join events_check ec on pp.user_id = ec.user_id
left join events_part ep on pp.user_id = ep.user_id
left join sessions s on pp.user_id = s.user_id
left join trans t on pp.user_id = t.user_id
left join csat c on pp.user_id = c.user_id
left join csat_back cb on pp.user_id = cb.user_id

```

Воронка по мероприятиям содержит в себе следующие результаты : количество просмотров, принявших участие, оставленных оценках, оценок “пять” и количество захотевших еще раз побывать на мероприятии. По этим данным можно отслеживать лучшие мероприятия, лучшие организации, провести сегментацию аудитории по интересам, провести тест конверсий в просмотр и принятие участие и т.д.

```

--
with count_event_check as (
select count(user_id) as count_check,event_id
from activities
where check_flg = 1 and activity_dttm between (current_date-31,

```

```

current_date-1)
  group by event_id
),
count_event_part as (
  select count(event_id) as count_part,user_id
  from activities
  where partition_flg = 1 and activity_dttm::date between
(current_date-31,current_date-1)
  group by user_id)
count_events as (
  select distinct event_id
  from events
  where event_dttm::date between (current_date-31,current_date-1)
),
count_csat as
(select count(user_id) as count_csat,event_id
from csat
where csat_date::date between (current_date-31,current_date-1)
group by event_id
),
count_csat_fives as
(select count(user_id) as count_fives_csat,event_id
from csat
where csat_date::date between (current_date-31,current_date-1) and
grade = 5
group by event_id
),
csat_back as (
  select count(user_id) as count_want_back,event_id
  from csat
  where get_back = 1 and csat_date::date between(current_date-31,
current_date-1)
  group by event_id)
select ce.event_id,cec.count_check, cep.count_part,cc.count_csat,ccf.
count_fives_csat,cb.count_want_back
from count_events ce
left join count_event_check cec on ce.event_id = cec.event_id
left join count_event_part cep on ce.event_id = cec.event_id
left join count_csat cc on cc.event_id = ce.event_id
left join count_csat_fives ccf on ccf.event_id = ce.event_id
left join csat_back cb on cb.event_id = ce.event_id

```

По этим метрикам, проводя исследования и тесты по повышению их показателей , можно построить монетизацию проекта за счет рекламы /комиссий и т.д.(метрики помогут показать денежный value от использования приложения).

Документация

Назначение раздела

В данном разделе собраны все документы, позволяющие организовать работу команды и разработку.

Содержание

- Глоссарий терминов
- Ответственность участников разработки
- План разработки
 - Модель жизненного цикла
- Стандарты
 - Стандарт на программирование backend-составляющей
 - Стандарт на программирование frontend-составляющей
 - Стандарт на программирование data science-составляющей
 - Стандарт написания пользовательских сценариев
 - Стандарт на проектирование баз данных

Глоссарий терминов

	Понятие	Определение	Аналог
1	Задача	Минимальная единица в Jira, используемая для описания необходимых составляющих работы в рамках одной цели	Тикет, задача
2	Спринт	Временной отрезок, составляющий одну или несколько недель (чаще всего 2), за который выполняется заранее установленный объём задач	Отрезок
3	Backend	Программно-аппаратная часть сервиса, отвечающая за функционирование его внутренней части. В случае веб-	Серверная часть приложения

		системы предполагается серверная часть	
4	Frontend	Клиентская сторона пользовательского интерфейса к программно-аппаратной части сервиса. В случае веб-системы предполагаются веб-сайты и мобильные приложения	Клиентская часть приложения
5	Автотест	Тест, реализующий какую-либо проверку автоматически, без прибегания к ручным действиям пользователя /разработчика	Тест
6	Деплой	Процесс “доставки” программного продукта пользователю	Выкатка
7	Релиз	Момент деплоя новой версии приложения	
8	СУБД	Система управления базами данных	БД
9	Continuous Integration	Непрерывная интеграция. Процесс непрерывного автоматизированного анализа написанного кода	CI
10	Continuous Delivery	Непрерывная доставка.	CD

		Автоматизированное развёртывание всех изменений в коде на тестовой среде	
11			

Ответственность участников разработки

В рабочем процессе выделяются несколько независимых друг от друга “направлений”, в которые направлена деятельность участников команды.

Они определяются следующими факторами:

- Должно производиться управление командой
- Должен производиться процесс документооборота и стандартизации
- Должна производиться программная разработка и создание дизайн-макетов

Выделяются следующие “направления” деятельности

Управление командой

Обязанности:

- Планирование работ по проекту
- Декомпозиция задач на подзадачи, определение плана работ на ближайший спринт
- Координация действий участников проекта
- Распределение задач между участникам проекта, определение компетенций и зон ответственности каждого из них
- Подготовка отчётов по спринту, разным этапам проекта

Ответственный:

- Лев Мардер

Осуществление процесса документооборота

Обязанности:

- Разработка и поддержка планов: разработки, управления конфигурацией, верификации, плана на программирование и др.
- Разработка и поддержка стандартов на: разработку требований, проектирования программной архитектуры, проектирования баз данных, программирования, тестирования и др.
- Написание и поддержка документов в системе Confluence

Ответственные:

- Лев Мардер
- Георгий Чистов
- Александр Тихонов
- Мария Морозевич
- Кирилл Воронин
- Павел Арсентьев
- Николай Пак

Разработка backend

Обязанности:

- Анализ плана необходимых работ
- Декомпозиция бизнес-требований на конкретные задачи на разработку
- Проектирование архитектуры backend-системы
- Проектирование способов взаимодействия между микросервисами
- Проектирование схем баз данных
- Написание кода, перенос бизнес-логики в программную логику
- Верификация написанного кода путём написания и прогона автотестов
- Верификация реализованных бизнес-требований, стандартов на программирование и исходного кода путём проведения процесса “Код-ревью”

Ответственные:

- Георгий Чистов
- Александр Тихонов

Разработка frontend

Обязанности:

- Анализ плана необходимых работ
- Декомпозиция бизнес-требований на конкретные задачи на разработку
- Создание дизайн-макетов конкретных приложений
- Переложение дизайн-макетов на программный код
- Написание кода, перенос бизнес-логики в программную логику, создание пользовательского интерфейса
- Создание интеграционных тестов без использования серверной части, верификация frontend-приложений
- Верификация реализованных бизнес-требований, стандартов на программирование и исходного кода путём проведения процесса “Код-ревью”

Ответственные:

- Николай Пак
- Кирилл Воронин

Разработка математической модели

Обязанности:

- Исследование способов реализации наукоёмких задач, которые ставит бизнес
- Создание математической модели, встраиваемой в систему в виде отдельного сервиса
- Разработка требований к верификации созданной модели

Ответственные:

- Павел Арсентьев

Функциональное и регрессионное тестирование

Обязанности:

- Написание тестовой документации, тест-кейсов
- Проведение тестирования frontend- и backend-составляющей на основе тест-кейсов

Ответственные:

- Мария Морозевич

Разработка средств поддержки разработки и администрирования

Обязанности:

- Составление требований к используемым технологиям, включая системы версионирования, системы деплоя, СУБД и т.д.
- Разработка и поддержка инструментов Continuous Integration и Continuous Delivery
- Подготовка приложений к релизу

Ответственные:

- Лев Мардер
- Александр Тихонов

План разработки

Назначение раздела

В данном разделе собраны планы и модели, касающиеся процесса разработки программной системы.

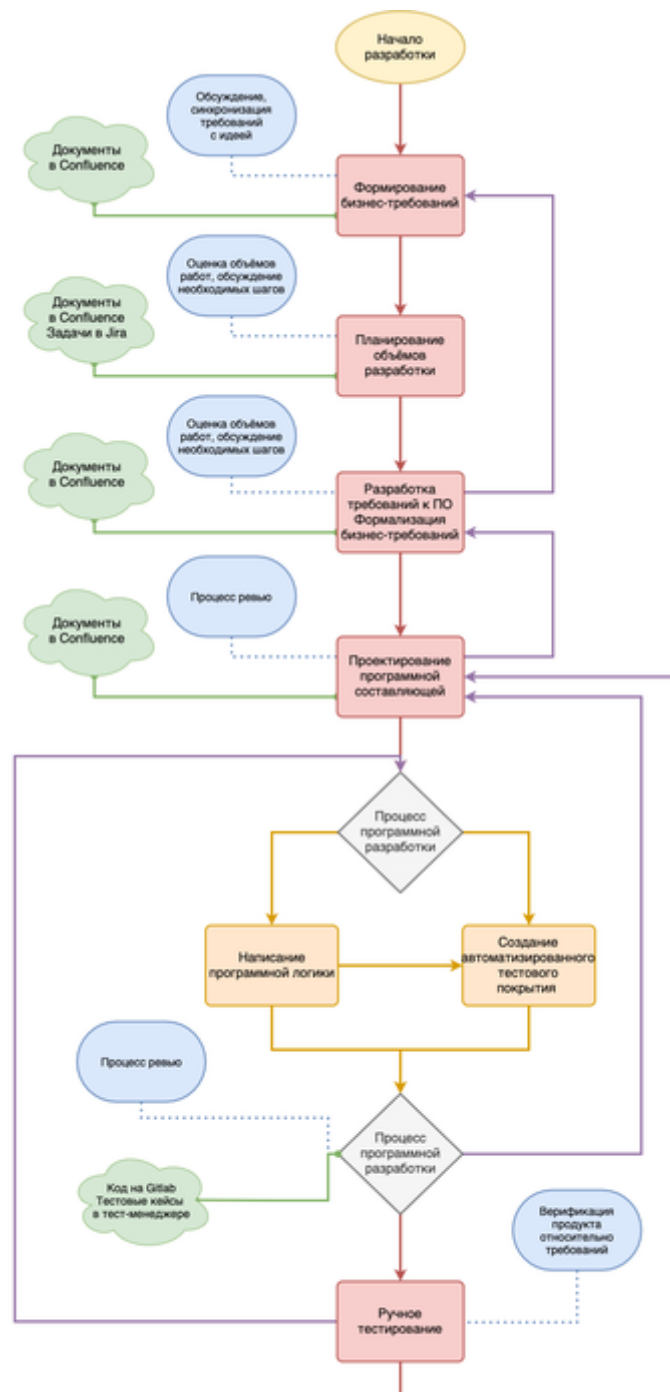
Содержание

- Модель жизненного цикла
- Использование Jira. Жизненный цикл задачи

Модель жизненного цикла

Модель жизненного цикла представлена в виде схемы, изображённой ниже. Она включает в себя два идейных блока:

- первичная разработка - формирование бизнес-требований, требований к ПО и планирование будущей разработки
- программная разработка - циклический процесс разработки ПО, связанный, в первую очередь, с программированием и верификацией программного кода, а также тестированием и доставкой продукта клиенту





Легенда схемы

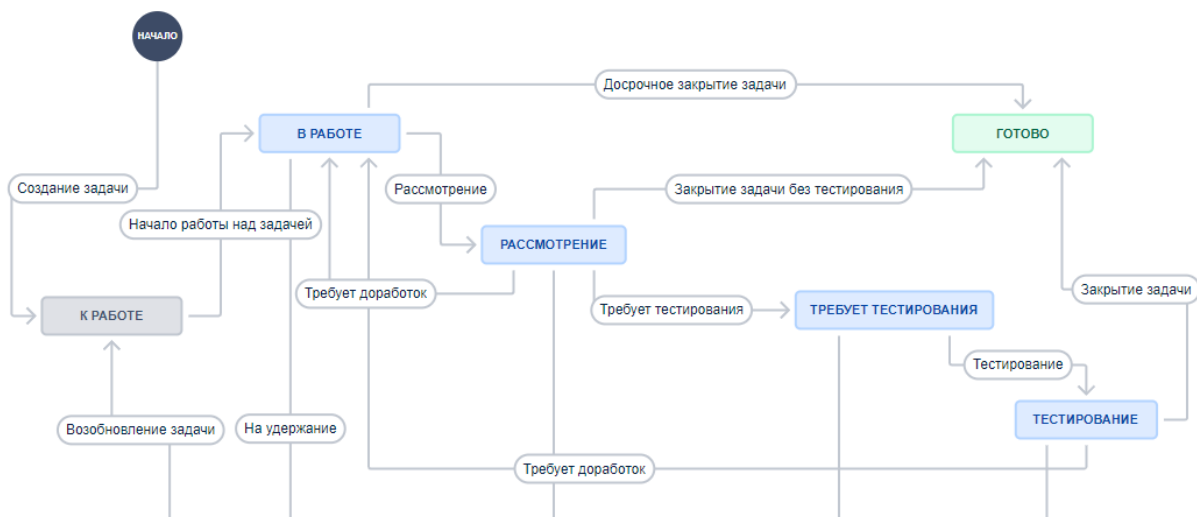
Цвета

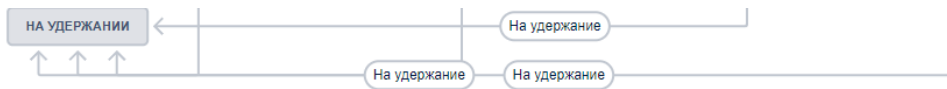
- **Жёлтый** - начальные и конечные точки разработки
- **Красный** - обязательные стадии при разработке
- **Серый** - обязательные задачи с возможностью пропуска внутренних составляющих
- **Оранжевый** - необязательные задачи, выполняемые асинхронно
- **Синий** - процесс верификации
- **Зелёный** - процесс управления конфигурацией, сохранение артефактов
- **Фиолетовый** - циклические процессы, резервные пути (применяются в случае заикливания разработки и/или в случае неудачи при верификации)

Связи

Связь	Описание
	Прямая связь между блоками в модели ЖЦ
	Связь между блоком в модели ЖЦ и процессом управления конфигурацией
	Связь между блоком в модели ЖЦ и процессом верификации

Использование Jira





Описание статусов задачи

- **К РАБОТЕ** - статус, в котором, ожидается начало работы над задачей в одном из ближайших спринтов
- **НА УДЕРЖАНИИ** - статус, в котором задача ожидает работы, но по тем или иным причинам она была отложена до возобновления
- **В РАБОТЕ** - статус, в котором, над задачей работает участник разработки (под работой понимается выполнение указаний, описанных в задаче)
- **РАССМОТРЕНИЕ** - статус, в котором работа над задачей завершена и ожидается (выполняется) рассмотрение результатов задачи на корректность и соответствие описанию
- **ТРЕБУЕТ ТЕСТИРОВАНИЯ** - статус, в котором ожидается проведение тестирования; данный сигнал является формальным и служит сигналом для тестировщика о том, что задачу можно брать в тест
- **ТЕСТИРОВАНИЕ** - статус, в котором проводится тестирование задачи (изменений в коде, нового функционала)
- **ГОТОВО** - статус, в котором все работы закончены, а задача считается выполненной

Описание жизненного цикла задачи

- **Создание задачи** - переход в статус **К РАБОТЕ**. В этом статусе предполагается, что в задаче указаны все нужные поля и её можно брать для работы, либо её необходимо дописать ответственному человеку и перевести в статус **В РАБОТЕ**.
- **Начало работы над задачей** - переход из статуса **К РАБОТЕ** в статус **В РАБОТЕ**. Задача переходит в работу, участник разработки работает над ней.
- **Досрочное закрытие задачи** - переход из статуса **В РАБОТЕ** в статус **ГОТОВО**. Закрытие задачи по причине неактуальности / устарелости.
- **Рассмотрение** - переход из статуса **В РАБОТЕ** в статус **РАССМОТРЕНИЕ**. Перевод сделанной задачи на процесс review (актуально для задач на программирование, где применяется code review и задач, требующих обсуждения команды)
- **Закрытие задачи без тестирования** - переход из статуса **РАССМОТРЕНИЕ** в статус **ГОТОВО**. Переход для задач, не

требующих тестирования (задач, не затрагивающих разработку, либо разработка которых полностью покрывается автотестами), либо для задач, которые требуется закрыть.

- **Требуется доработка** - переход из статусов **РАССМОТРЕНИЕ** и **ТЕСТИРОВАНИЕ** в статус **В РАБОТЕ**. Переход для задач, которые не прошли процесс ревью или тестирования и требуют доработок.
- **Требуется тестирования** - переход из статуса **РАССМОТРЕНИЕ** в статус **ТЕСТИРОВАНИЕ**.
- **Тестирование** - переход из статуса **ТРЕБУЕТ ТЕСТИРОВАНИЯ** в статус **ТЕСТИРОВАНИЕ**. Переход подразумевает, что ответственный тестировщик взял задачу на тест.
- **На удержании** - переход из любого промежуточного статуса в статус **НА УДЕРЖАНИИ**. Совершение этого перехода подразумеваются для задач, которые по тем или иным причинам оказываются не нужны / заблокированы и не будут выполнены в текущий спринт.

Процесс разработки требований к ПО

Процесс разработки требований к ПО является частью жизненного цикла ПО и взаимодействует со всеми остальными процессами, определенными в проекте. Упрощенное окружение процесса представлено на рисунке 1.

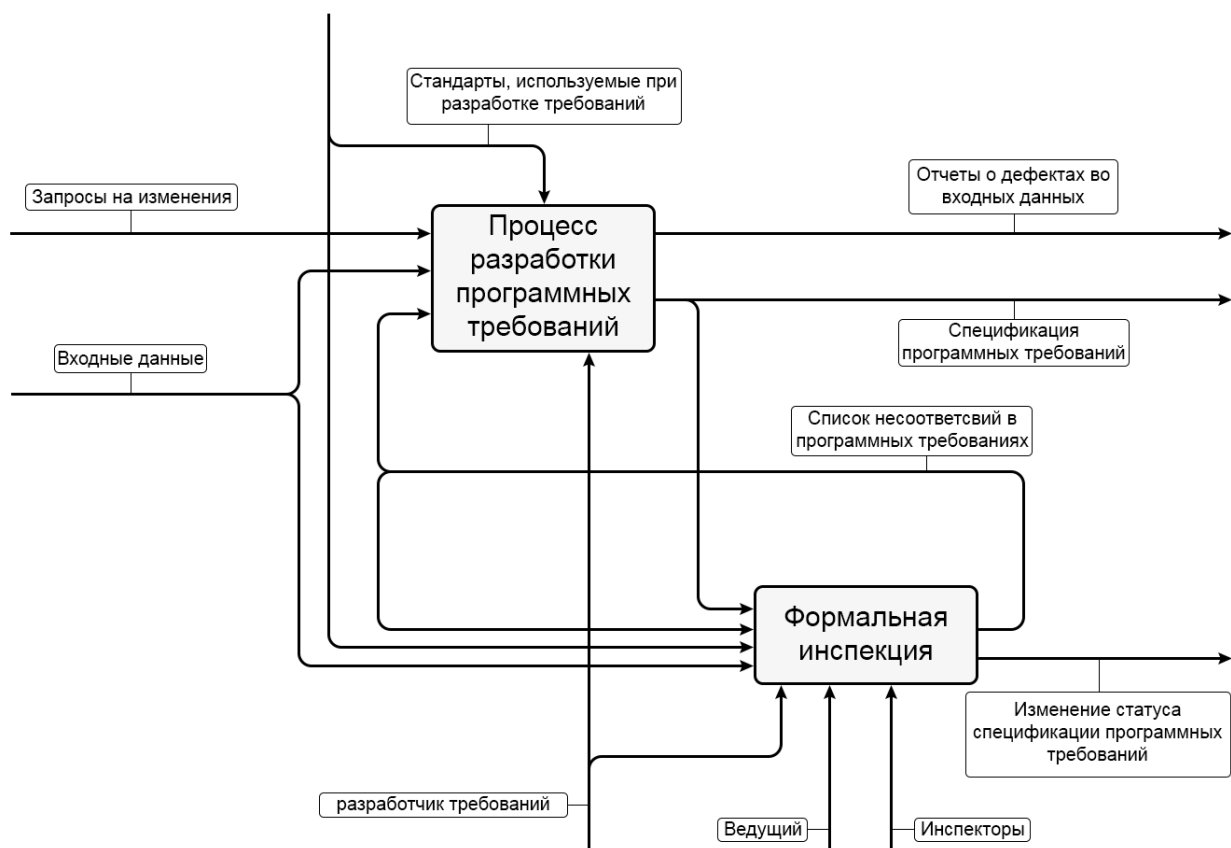


Рисунок 1. Окружение процесса разработки программных требований

Процесс разработки требований использует входные данные, предоставляемые заказчиком (например, спецификации системных требований и архитектуры) и данные, возникающие в ходе других процессов, связанных с данным процессом (например, результаты формальных инспекций). Результаты процесса разработки требований поступают на входы других процессов, выполняющихся в проекте.

Верификация разработанной спецификации программных требований проводится с помощью процедуры формальной инспекции, которая регламентируется стандартом проекта «Формальные инспекции» и планом верификации проекта. В случае обнаружения несоответствий процесс разработки программных требований повторяется.

Входные данные процесса разработки требований к ПО

Исходными входными данными являются бизнес-требования, функциональные требования, пользовательские требования, архитектурные ограничения и описание ее программно-аппаратных интерфейсов, стандарты проекта (в том числе и [стандарт на разработку требований](#)).

Кроме того, при повторении процесса дополнительным входом служат описания несоответствий в спецификации требований к ПО, выявленных в других процессах (например, при проектировании, кодировании или тестировании) или заказчиком.

Выходные данные процесса разработки требований к ПО

Список выходных данных, места их хранения и дальнейшее предназначение определяются в стандартах проекта. Выходные данные обычно включают в себя разработанные спецификации требований к ПО и сообщения о проблемах.

Условия начала процесса разработки требований к ПО

Условия начала процесса разработки требований определяются в стандартах проекта. Как минимум, для начала процесса необходимо наличие хотя бы части входных данных.

Структура процесса разработки требований к ПО

Процесс разработки требований к ПО включает в себя следующие виды деятельности:

- анализ информации и выделение программных требований из входных данных процесса, отделение функциональных требований от нефункциональных, бизнес-правил, предполагаемых решений и другой информации;
- разложение высокоуровневых требований до нужного уровня детализации;
- распределение требований по компонентам ПО;
- согласование приоритетов реализации;
- понимание относительной важности атрибутов качества;
- выявление пробелов в требованиях или излишних требований, не соответствующих заданным рамкам;
- документирование требований;
- утверждение требований.

Условия конца процесса разработки требований к ПО

Процесс разработки требований носит итерационный характер. Окончанием итерации является решение разработчика требований о том, что разработанные им документы готовы к формальной инспекции и регистрацией этого в соответствии со стандартами проекта. Пока в результате формальной инспекции не будет установлено соответствие разработанных требований входным данным, процесс будет повторяться.

Таким образом, условием конца процесса разработки требований к ПО является извещение руководителя проекта о завершении работ в соответствии с пунктом “Структура процесса разработки требований к ПО”.

Роли и квалификация участников процесса

На роль разработчика требований назначаются участники команды, считающиеся компетентными для выполнения этой роли и хорошо знающие предметную область (насколько это возможно).

План управления конфигурацией

Назначение раздела

В данном разделе собраны планы и модели, касающиеся процесса управления конфигурацией.

Содержание

- [Базовые версии](#)
- [Управление конфигурацией программного кода](#)
- [Идентификация единиц конфигурации](#)
- [Инструменты управления конфигурацией](#)

Базовые версии

Базовая версия необходима для того, чтобы на неё можно было ссылаться в процессе разработки, считать её эталонной на момент времени, а также, чтобы чётко отслеживать версию документа.

Требования к базовой версии

- Корректность, то есть базовая версия документа должна пройти ревью перед сборкой.
- Версионность, то есть каждая базовая версия документа должна иметь свой идентификационный номер, которым определяется её версия.

Процесс сборки базовой версии документа, описывающего проект

В качестве системы для хранения знаний по проекту используется Confluence. По своей сути он не является прямым источником информации для базовой версии, а лишь является редактором и метом накопления информации. Однако с его помощью может быть собрана базовая версия документа.

Состояние страницы-раздела Confluence

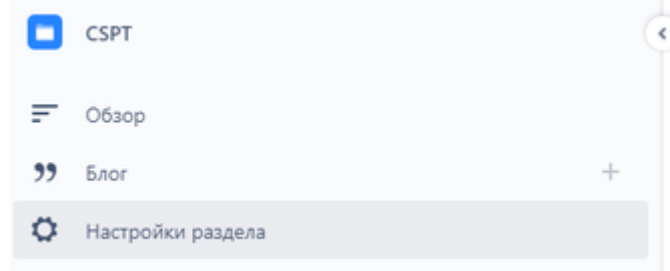
- Каждая страница редактируется её автором или ответственным разработчиком в процессе работы над соответствующей задачей / вносом правок.
- Каждая страница проходит процесс рассмотрения, в ходе которого определяется корректность внесённой информации, её соответствие цели задачи и её описанию, а также соответствие проекту.
- Если страница успешно проходит процесс рассмотрения, то она считается стабильной и блокируется для редактирования.
- В случае, если требуется доработка страницы после её блокирования, создаётся отдельная задача и у администратора Confluence запрашивается разблокировка страницы.

Действия перед сборкой базовой версии

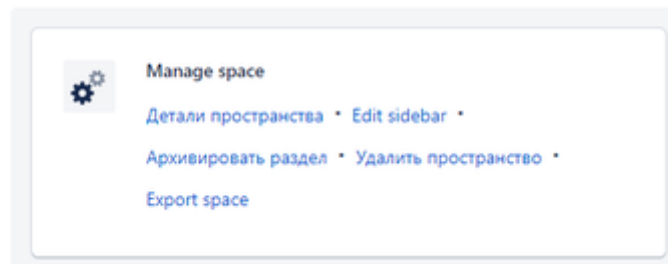
- Необходимо убедиться в том, что все необходимые Confluence страницы закрыты для редактирования.
 - Если какая-то из страниц открыта, то необходимо проверить, не используется ли она в какой-то открытой задаче из текущего спринта.
 - В случае, если она не задействована ни в одной из открытых задач, следует обратиться к ответственному разработчику.
- Необходимо убедиться в доступности репозитория, где хранятся базовые версии документов. Если он недоступен, то сборку следует отложить.

Сборка файла с базовой версией документа

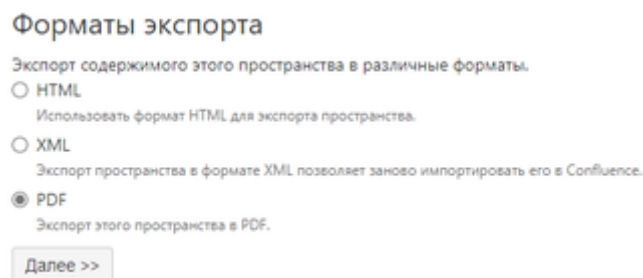
- Открыть **настройки раздела** в пространстве Confluence.



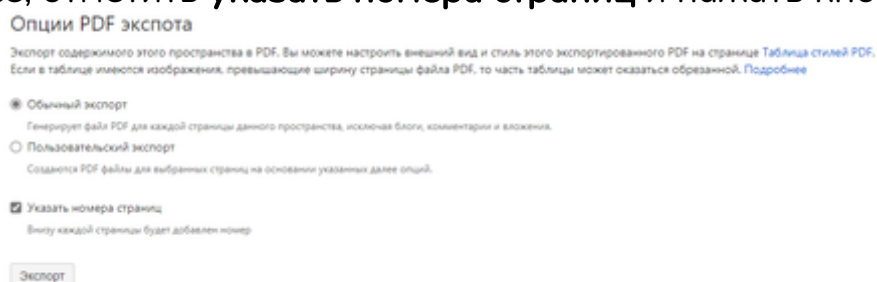
- Перейти в раздел **manage space export space**
Настройки раздела



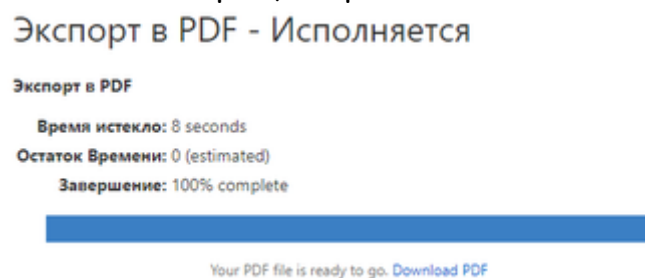
- Выбрать формат экспорта **pdf** и нажать кнопку **далее**



- Выбрать **пользовательский экспорт**, указать нужные разделы Confluence, отметить **указать номера страниц** и нажать кнопку **экспорт**



- Дождаться окончания генерации файла, нажать **download PDF**



- Переименовать файл в соответствии со стандартом по названию версии
 - В текущем виде номер версии формируется как “{день месяца}{месяц}{текущий год}-{время генерации}”
 - Например, документ, собранный 12 ноября 2021 года в 8:15 будет иметь версию “121121-0815”
 - Документ именуется как “CSPT-{версия документа}”
 - Например, документ, собранный 12 ноября 2021 года в 08:15 будет именоваться “CSPT-121121-0815”

Процесс загрузки и получения новой версии документа

Версионность базовой версии обеспечивается путём хранения документов в git-репозитории. С помощью такого git-репозитория можно загружать и получать новую версию документа.

Получение новой версии документа

- Перейти в репозиторий, где хранятся базовые версии. Проверить, что в корне проекта есть папка **documents**, выбрана ветка **main**

Name	Last commit	Last update
documents	Update README.md, documents/Документ ...	24 seconds ago
README.md	Update README.md, documents/Документ ...	24 seconds ago

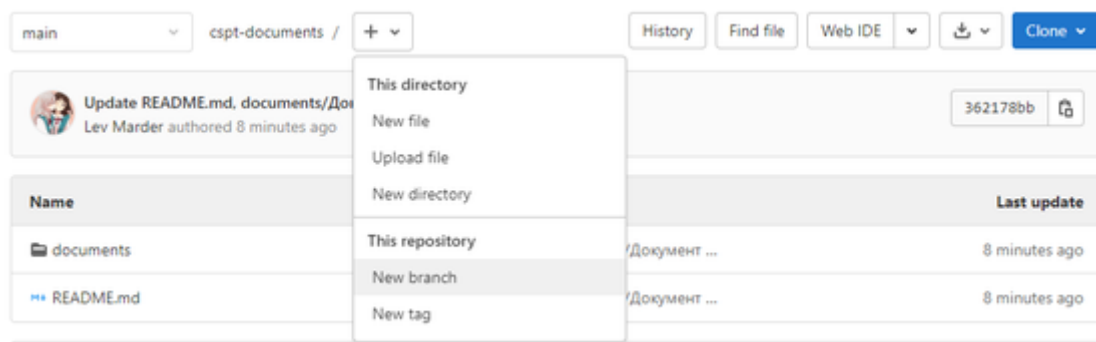
- Перейти в папку **documents**

Name	Last commit	Last update
Документ v.2021.455.pdf	Update README.md, documents/Документ v.2021.455.pdf	3 minutes ago

- Выбрать документ с ближайшим значением last update.
- Просмотреть документ можно прямо из браузера. Если необходимо его загрузить на компьютер, то можно нажать на кнопку скачивания.

Загрузка новой версии документа

- Перейти на главную страницу репозитория. В выпадающем меню нажать кнопку **New branch**.



- Создать новую ветку по названию новой базовой версии, которая будет загружаться.

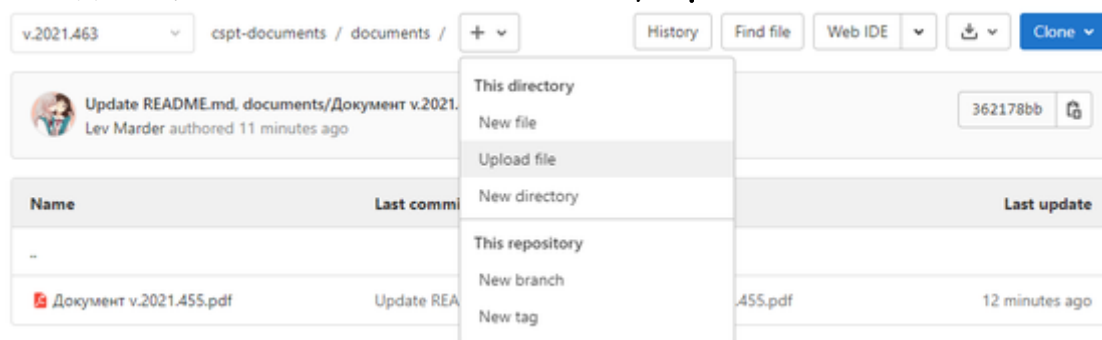
New Branch

Branch name

Create from

Existing branch name, tag, or commit SHA

- Убедиться, что репозиторий переключился на новую ветку.
- Перейти в директорию **documents**.
- В выпадающем меню нажать на кнопку **Upload file**.



- В появившееся окно добавить новый файл. В качестве commit message указать "version {название версии}". Нажать на кнопку **Upload file**.

Upload New File

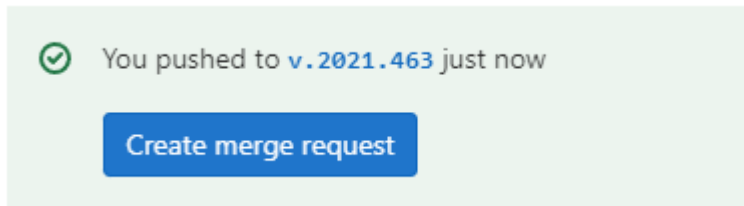
1.13 MiB
Документ v.2021.463.pdf
[Remove file](#)

Commit message

Target branch



- Появляется сообщение с кнопкой **create merge request**. Нажать на кнопку.



- В открывшемся меню меняются следующие поля
 - Assignee - указать себя
 - Reviewer - указать ответственное лицо, занимающееся администрированием репозитория с базовыми версиями
- Нажать кнопку **create merge request**.
- После получения подтверждения на загрузку новой базовой версии в текущем окне нажать кнопку **merge**.
- После этого новая базовая версия оказывается на ветке main. Загрузить её можно описанным способом.

Идентификация единиц конфигурации

Единицей конфигурации является единица передачи части системы между участниками процесса.

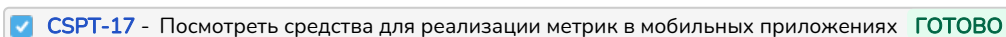
Единица конфигурации обладает следующими свойствами:

- Наличие одной и более версий
- Наличие уникального идентификатора

Список единиц конфигурации

Задача

Задача представлена в виде задачи в Jira. Пример:



Задача однозначно идентифицируется по составному идентификатору, составленному из инкрементального номера задачи и названия проекта по шаблону "{название проекта}-{номер}".

Также задача имеет название, не обязательно уникальное и тип. Предполагаются следующие типы:

- **Эпик (epic)** — большая задача, на решение которой команде нужно несколько временных отрезков.
- **История (story)** — часть большой задачи (эпика), которую команда может решить за 1 временной отрезок.

- **Задача (task)** — техническая задача, которую делает один из членов команды
- **Под-задача (sub-task)** — часть истории / задачи, которая описывает минимальный объем работы члена команды.
- **Баг (bug)** — задача, которая описывает ошибку в системе.

Версионность задачи реализована в виде истории изменений.

Репозиторий

Репозиторий - это Gitlab-хранилище, где содержится исходный код какого-либо проекта. Стоит отметить, что в репозитории также могут храниться файлы конфигурации приложения, документы и прочее.

Пример:

[Lev Marder / go-events-atomhack-summer-2021 · GitLab](#)

Репозиторий идентифицируется по названию, которое обязательно является уникальным.

Версионность реализована с помощью Git, а именно:

- В качестве основной ветки используется ветка master. Различные версии основной ветки могут быть просмотрены по коммитам.
- Основное ответвление от master - develop. Тут находятся версии репозитория, подготовленные к релизу.
- Также может создаваться бесчисленное количество ответвлений от ветки develop, каждое из которых идентифицируется уникальным названием, зачастую совпадающим с названием задачи, с которой эта ветка связана.

Merge Request

Merge request является запросом на слияние веток в Gitlab. Пример:

[Develop \(!1\) · Merge requests · Lev Marder / go-articles-sbercode-spring-2021 · GitLab](#)

Merge request идентифицируется по названию репозитория и инкрементальному номеру, подсчитываемому для каждого репозитория отдельно. Шаблон идентификатора: "{репозиторий}!{номер}".

Merge request также имеет название, которое может быть неуникальным и служит для описания цели merge request'a.

Версионность реализована в виде истории обсуждений и истории коммитов в ветку, для которой создан merge request.

Страница Confluence

Страница Confluence является составной частью базовой версии документа об организации разработки. Пример: текущая страница.

Страница Confluence идентифицируется по её полному пути от начала пространства Confluence.

Версионность обеспечивается за счёт ограничения доступа к редактированию страницы между выпуском базовой версии. По сути версией страницы является вид страницы на тот момент, когда её редактирование блокируется.

Инструменты управления конфигурацией

Для осуществления управления конфигурацией ПО требуется среда, в которой можно было бы эффективно и удобно осуществлять такое управление.

В качестве инструментов управления конфигурацией были выбраны:

- Gitlab - управление репозиториями.
- Jira - управление страницами.
- Confluence - управление документами и среда для их редактирования.

Gitlab

В качестве среды для работы с репозиториями используется Gitlab. Gitlab - это веб-сервис для хостинга и совместной разработки проектов, использующий Git в качестве системы контроля версий.

Единицы конфигурации хранятся в git-репозиториях, представляющих собой каталоги файловой системы, содержащие файлы конфигурации репозитория, файлы журналов, хранящие операции, выполняемые над репозиторием, индекс, описывающий расположение файлов, и хранилище отслеживаемых файлов. Участники разработки могут вносить свой вклад в файлы, находящиеся в репозитории, тем самым изменяя конфигурацию проекта.

Gitlab хранит полную историю изменений проекта в виде наборов состояний - коммитов. В случае необходимости любое из состояний проекта может быть восстановлено. Состояние может быть выбрано в качестве очередной базовой версии разрабатываемого продукта. Версия может только увеличиваться при движении вперед по истории состояний.

Gitlab, будучи основанным на Git, поддерживает систему с ветками - отдельными над-состояниями репозитория, характеризующимися своим набором коммитов, возможностью ведения параллельной разработки и т. д. Ветки позволяют организовать разработку в команде с большим числом участников, удобно сравнивать версии проекта и, в случае необходимости, совершать откат последней базовой версии проекта.

Jira

В качестве среды для управления задачами используется Jira. Jira – это программный инструмент для управления проектами. Jira используется для формирования списка задач, отслеживания общего прогресса команды и решения возникающих по ходу разработки продукта проблем.

Приложение построено по принципам канбан- и скрам-досок. Эти принципы дополняются массой вспомогательных механизмов, которые добавлялись в приложение исключительно с целью упростить создание новых приложений, добавить в них функции, исправить ошибки и т.п. Также эта система управления проектами исповедует Agile-методику разработки.

Основные сценарии использования Jira и сценарии, в которых удобно использование задач:

- Организация списка задач в виде канбан-доски с целью наглядного отображения текущей истории/эпика и статуса проекта.
- Управление проектом и командой, организация рабочего процесса.
- Работа с жизненным циклом задачи.
- Отслеживание ошибок, баг-трекинг, формирование списка задач на внесение изменений в проект.

Confluence

В качестве среды для создания, редактирования документов и управления ими используется Confluence. Confluence — это wiki движок и среда для редактирования текста. Он позволяет организовать внутренний портал/пространство, дать доступ к нему участникам разработки и создавать страницы будущих документов.

Confluence позволяет пользоваться широким набором инструментов для редактирования текста. Возможности среды позволяют интегрироваться с Jira и Gitlab, автоматически подтягивать изменения из связанных документов и не только.

В случае текущего проекта Confluence используется в первую очередь как среда редактирования текста и сбора информации о проекте. Каждая страница Confluence подлежит версионированию (версия фиксируется в момент блокировки страницы для редактирования). Функционал Confluence позволяет генерировать pdf-документ на основе страниц, который и служит базовой версией документа об организации разработки.

Стандарты

Назначение раздела

В данном разделе собраны стандарты на разработку различных компонентов программной системы.

Содержание

- Стандарт на программирование backend-составляющей
- Стандарт на программирование frontend-составляющей
- Стандарт на программирование data science-составляющей
- Стандарт написания пользовательских сценариев
- Стандарт на проектирование баз данных

Стандарт на кодирование backend-составляющей

Используемый стек технологий

- C# - язык программирования, используемый для разработки проху-сервиса;
- Go - язык программирования, используемый для разработки микросервисов;
- HTTP - протокол передачи данных, используемый для связи “проху приложение” по стандарту REST API;
- gRPC - фреймворк для удаленного вызова процедур на основе HTTP. 2, используемый для связи “проху микросервис”;
- Protocol Buffers (proto3) - протокол сериализации данных, используемый в рамках gRPC для связи “проху микросервис”

Используемые инструменты

- IDE:
 - Visual Studio 2019 или VS Code 1.61 - для разработки на C#
 - JetBrains GoLand 2021.2.3 - для разработки на Go
- Линтеры:
 - Microsoft.CodeAnalysis.NetAnalyzers v5.0.3 - анализатор кода для C#/.NET
 - Для Go:
 - golangci-lint v1.42.1 - агрегатор линтеров для Go, позволяющий запускать большое количество линтеров параллельно друг-другу
 - линтеры deadcode, errcheck, gosimple, govet, ineffassign, staticcheck, structcheck, typecheck, unused, varcheck, errorlint, goconst, gocyclo, goheader, goimports, revive (см. [подробнее](#))

Стандарт оформления кода для C#

Стандарт оформления кода для C# основан на общепринятых конвенциях ([C# Coding Conventions](#)):

- Для наименования классов, записей, структур, а также публичных полей, свойств, событий, методов и локальных функций используется PascalCase:

```
public class DataService
{
}
```

- Для наименования интерфейсов используется PascalCase с добавлением префикса I:

```
public interface IWorkerQueue
{
}
```

- Для наименования внутренних и приватных полей, используется camelCase с добавлением префикса _:

```
public class DataService
{
    private IWorkerQueue _workerQueue;
}
```

- Для приватных полей, являющихся статическими, используется camelCase с добавлением префикса s_:

```
private static IWorkerQueue s_workerQueue;
```

- Для наименования аргументов методов используется camelCase.
- Используются настройки редактора кода по умолчанию: умный отступ, четырехсимвольные отступы, табуляции как пробелы.
- Не более одного утверждения или объявления в строке.
- Хотя бы одна пустая строка между определениями методов и свойств.

Стандарт оформления кода для Go

Язык программирования Go отличается существованием предусмотренных его разработчиками [единых правил оформления кода](#), которые и будут использоваться в проекте. Форматирование кода и система наименований определяются использованием линтеров.

- Форматирование кода осуществляется с помощью линтера goimports с настройками по умолчанию.
- Соответствие стандартам наименования проверяется с помощью линтера revive с настройками по умолчанию.

Стандарт на кодирование data science-составляющей

Используемый стек технологий

- Python - язык программирования, используемый для data science исследований и разработки сервисов
- SQL - язык запросов в БД, используемый для получения данных о пользователях, событиях и т.д.; данные нужны для обучения математических моделей

Используемые инструменты

- IDE:
 - JetBrains PyCharm/Jupyter Notebook - для разработки на Python
 - JetBrains DataGrip - для разработки на SQL
- Библиотеки:
 - pandas - для загрузки, выгрузки, первичной обработки табличных данных, визуализации
 - pickle - для сериализации и десериализации объектов Python
 - matplotlib - для построения и визуализации сложных графиков
 - seaborn - для построения и визуализации сложных графиков
 - scikit-learn - для data science и machine learning
 - tensorflow - для data science и machine learning
 - keras - для data science и machine learning
- Линтеры:
 - flake8/pylint - анализатор кода для Python
 - встроенный линтер в IDE для SQL

Структура data science-проекта

- **data_science** - Содержит все файлы для работы с данными
 - models
 - operators
- **sql** - Содержит все файлы для представления данных из базы данных
 - metrics
 - data_storefronts

Идентификация элементов конфигурации

При использовании Python выделяются следующие элементы.

Элемент	Определение	Способ именования
пакет	каталог со сценарием реализации <code>__init__.py</code>	полное название по пути к файлу от папки <i>data_science</i> , названия файлов или папок отделяются точкой

класс	описание пользовательского типа данных, на основе которого создаются однородные объекты	полное название по пути к файлу от папки <i>data_science</i> , названия файлов или папок отделяются точкой, после чего ставится имя класса
функция	объект, принимающий аргументы и возвращающий значение	полное название по пути до класса или пакета, названия отделяются точкой, после чего ставится имя функции

При использовании SQL выделяются следующие элементы.

Элемент	Определение	Способ именования
скрипт	набор sql команд, сохраненных в файл	полное название по пути к файлу от папки <i>sql</i> , названия файлов или папок отделяются точкой, после чего ставится имя скрипта

Стандарты на форму представления исходного кода

Стандарт, используемый для языка Python

В качестве стандарта для представления кода на языке Python используется Стандарт PEP8.

1. В качестве отступов используются пробелы, не используется табуляция.
2. Для обозначения отступов используются 4 пробела.
3. Ограничение длины строки - максимум 79 символов.
4. Отделение определений функций и определений классов двумя пустыми строками.
5. Переменные при объявлении всегда должны быть инициализированы.
6. Используется кодировка UTF8.
7. Бинарные операторы окружаются пробелами с обеих сторон (Это операторы присваивания (=, +=, -= и другие), сравнения (==, <, >, !=, <>, <=, >=, in, not in, is, is not), логические (and, or, not)).
8. Комментарии должны являться законченными предложениями. Если комментарий — фраза или предложение, первое слово должно быть написано с большой буквы, если только это не имя переменной, которая начинается с маленькой буквы (не следует менять регистр переменной, т.к. это меняет смысл).

9. Многострочные комментарии пишутся при помощи тройных кавычек.
10. Блок комментариев с документацией кода, идущий после объявления функции/класса должен иметь тот же отступ, что и сам код. Каждая строка такого блока должна начинаться с символа # и одного пробела после него (если только сам текст комментария не имеет отступа).
11. Абзацы внутри блока комментариев разделяются строкой, состоящей из одного символа #.
12. Импорты всегда помещаются в начале файла, сразу после комментариев к модулю и строк документации, и перед объявлением констант.
13. Каждый импорт, как правило, должен быть на отдельной строке.
14. Для имен функций используется нижний регистр. Если необходимо повысить читаемость, то можно разделять слова символом нижнего подчеркивания. (snake_case нотация)
15. Имена переменных выбираются по тем же правилам, что и имена функций.
16. Константы определяются на уровне подключения модулей и пишутся в верхнем регистре с разделением слова символами подчеркивания.
17. Можно использовать как одинарные, так и двойные кавычки. Однако, если в строке используются двойные кавычки, то следует выделять внутреннюю строку одинарными и наоборот.

Стандарт, используемый для языка SQL

1. Идентификаторы и имена выбираются осмысленными и в едином стиле (поддерживается одна нотация, выбираемая командой разработки/аналитики).
2. Имена полей должны быть уникальны и не должны быть в зарезервированном ключевым словом.
3. Пробелы и отступы расставляются индивидуально для лучшей читаемости кода.
4. Дата и время должны соответствовать стандарту ISO 8601: YYYY-MM-DD HH:MM:SS.SSSSS.
5. Функции SQL используются только стандартные (входящие в СУБД).
6. Комментарии пишутся предпочтительно в стиле C — /* (начало) и *, (конец). Либо - - перед комментарием, тогда окончанием будет новая строка.
7. Ограничение длины имени - 30 символов.
8. Имена начинаются с буквы и заканчиваются буквой.

9. В именах используются только буквы, цифры и символ подчёркивания.
10. Запрещено использование нескольких подряд идущих символов подчёркивания.
11. Символ подчёркивания используется там, где в реальной жизни стоит пробел (например, `first name` станет `first_name`).
12. Следует избегать сокращений. Если их всё же нужно использовать, следует убедиться в том, что они общепонятны.
13. Следует использовать собирательные имена или, что менее предпочтительно, форму множественного числа. Например, `staff` и `employees` (в порядке убывания предпочтения).
14. Не допускается совпадение названия таблицы с названием любого из её столбцов.
15. По возможности следует избегать объединения названий двух таблиц для построения таблицы отношений. Например, вместо названия `cars_mechanics` лучше подойдёт `services`.
16. Пробелы используются для выравнивания основных ключевых слов.
17. Всегда делайте перенос строки:
 - перед AND или OR
 - после точки с запятой (для разделения запросов)
 - после каждого основного ключевого слова
 - после запятой (при выделении логических групп столбцов)
18. У каждой таблицы должен быть хотя бы один первичный ключ.

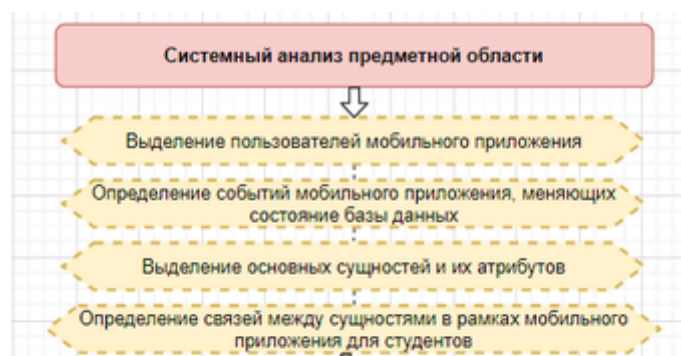
Стандарт написания пользовательских сценариев

Стандарт на проектирование баз данных

Проектируемая база данных должна обеспечивать:

- хранение всей необходимой информации;
- возможность получения данных по всем необходимым запросам;
- сокращение избыточности и дублирования данных;
- целостность данных (правильность их содержания): исключение противоречий в содержании данных, исключение их потери и т.д.

Этапы проектирования баз данных





1) Концептуальное (инфологическое) проектирование

Цель инфологического проектирования состоит в получении моделей, отражающих предметную область и информационные потребности пользователей. В качестве инструмента для построения модели данных на этапе проектирования можно выбрать **модель «сущность–связь»** (Entity–Relationship) - метод **«ER– диаграммы»**.

Основные элементы ER-моделей:

- объекты (сущности);
- атрибуты объектов;
- связи между объектами.

Связь между сущностями характеризуется:

- типом связи (1:1, 1:N, N:M);
- классом принадлежности (может быть обязательным и необязательным). Если каждый экземпляр сущности участвует в связи, то класс принадлежности — обязательный, иначе — необязательный.

Типы атрибутов:

- *Идентифицирующие* (имеют уникальное значение для сущностей данного типа) и *описательные* (заклюают в себе интересующие свойства сущности). Для каждой сущности должен быть указан первичный ключ;
- *Составные* (состоят из комбинации нескольких компонентов) и *простые* (состоят из одного компонента);

- *Однозначные* (имеют одно значение для каждого экземпляра сущности) и *многозначные* (имеют много значений для каждого экземпляра сущности);
- *Основные* (значение не зависит от других атрибутов) и *производные* (значение вычисляется на основе значений других атрибутов);
- *Обязательные* (значение всегда устанавливается при помещении данных в БД) и *необязательные* (значение необязательного атрибута может быть пропущено).

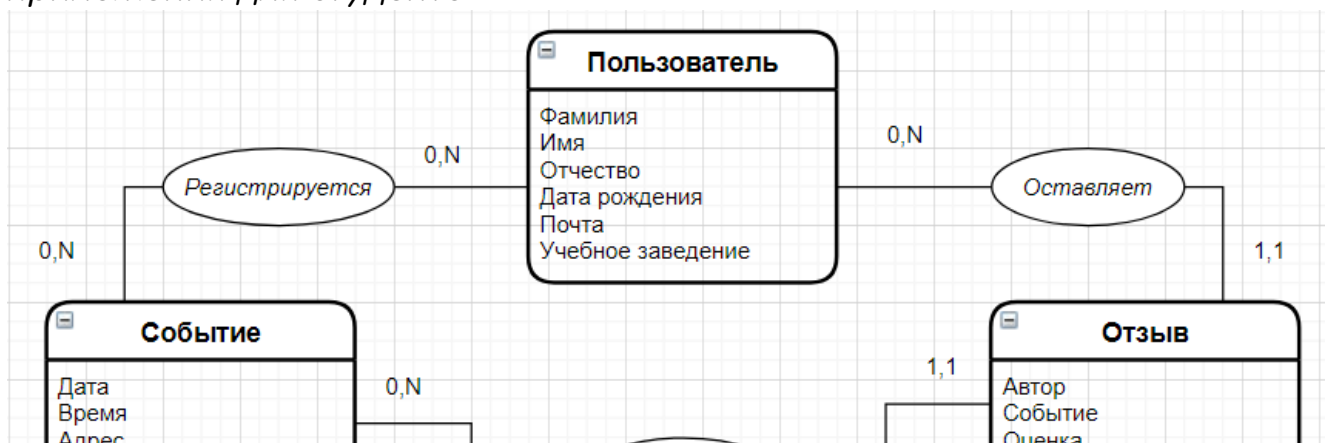
Типы сущностей:

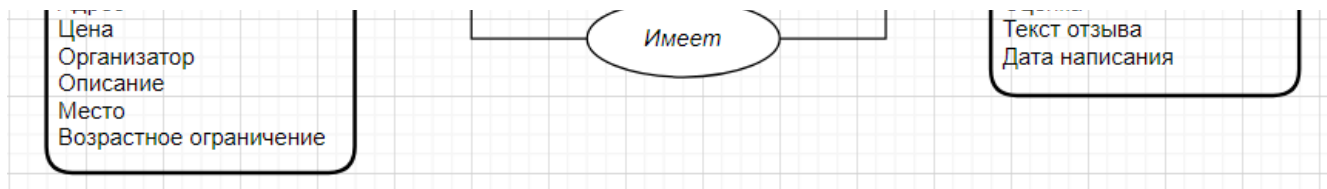
- *Стержневая сущность* - является независимой сущностью;
- *Ассоциативная сущность* - связана с несколькими родительскими сущностями и содержит информацию о связях сущностей;
- *Именующая сущность* – частный случай ассоциативной сущности, не имеющей собственных атрибутов (только атрибуты родительских сущностей, мигрировавших в качестве внешнего ключа);
- *Характеристическая сущность* – сущность, связанная только с одной родительской и хранящая информацию о характеристиках родительской сущности;
- *Категориальная сущность* – дочерняя сущность в иерархии наследования.

Типы связей:

- *Тип связи 1:1* - примером является связь между сущностями ГРАЖДАНИН РФ и ПАСПОРТ РФ, каждый гражданин Российской Федерации может иметь только один паспорт РФ;
- *Тип связи 1:M* - примером является связь между сущностями ГРУППА и СТУДЕНТ (в одной группе обучается много студентов);
- *Тип связи M:N* - примером является связь между сущностями ПРЕПОДАВАТЕЛЬ и ДИСЦИПЛИНА. Каждый преподаватель может вести несколько дисциплин, и одну и ту же дисциплину могут вести несколько преподавателей.

Пример фрагмента концептуальной модели базы данных мобильного приложения для студентов:





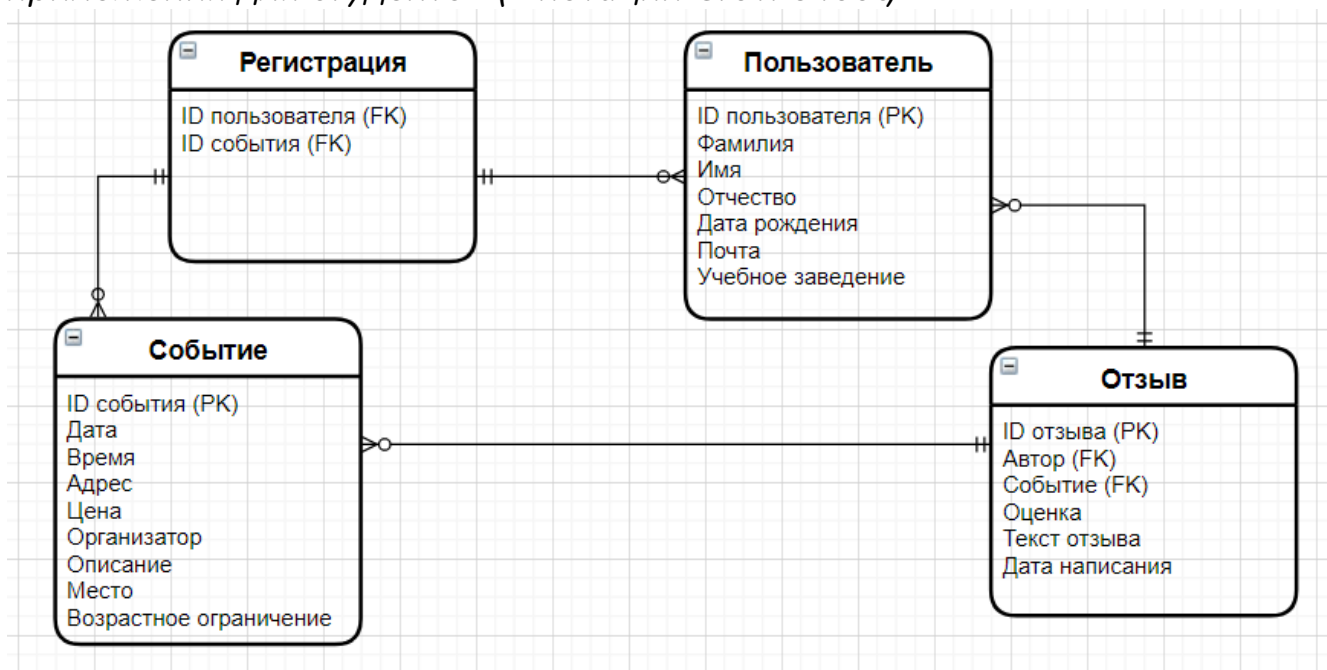
2) Логическое проектирование

Цель логического проектирования заключается в определении числа и структуры таблиц, формировании запросов к БД, определении типов отчетных документов, разработке алгоритмов обработки информации, создании форм для ввода и редактирования данных в базе и решении ряда других задач. Наиболее важной является проблема структуризации данных.

Внутренние ограничения целостности:

- *ограничения на значения ключевых атрибутов*: уникальность значений атрибутов первичных и альтернативных ключей определяется понятием ключа. Эти ограничения задаются соответствующим выделением ключей в концептуальной схеме;
- *ограничения на допустимые значения атрибутов*: все допустимые значения атрибутов должны удовлетворять условию принадлежности соответствующему домену. Эти ограничения задаются соответствующим описанием доменов и атрибутов;
- *ограничения на существующие значения (ссылочные ограничения)*: существование одних сущностей (дочерних, типа категория) ставится в зависимость от существования других (родительских, родовых).

Пример фрагмента логической модели базы данных мобильного приложения для студентов (в нотации Crow's foot):



3) Физическое проектирование

Цель физического проектирования заключается в описании способа физической реализации логического проекта базы данных. Для реляционной базы данных на этом этапе сущности преобразуются в таблицы, атрибуты – в столбцы. Связи между сущностями преобразуются в физическую схему БД.

Решается задача грамотного размещения объектов базы данных в пространстве памяти, строятся индексы, определяется целесообразность использования хеширования и кластеризации, учитывается специфика выбранной СУБД.

Нормализация

При проектировании реляционных баз данных выполняется *нормализация* - процесс преобразования отношений базы данных к виду, отвечающему нормальным формам.

Нормальные формы:

- **Первая нормальная форма (1NF):** переменная отношения находится в первой нормальной форме (1НФ) тогда и только тогда, когда в любом допустимом значении отношения каждый его кортеж содержит только одно значение для каждого из атрибутов;
- **Вторая нормальная форма (2NF):** переменная отношения находится во второй нормальной форме тогда и только тогда, когда она находится в первой нормальной форме и каждый неключевой атрибут неприводимо (функционально полно) зависит от её потенциального ключа;
- **Третья нормальная форма (3NF):** переменная отношения находится в третьей нормальной форме тогда и только тогда, когда она находится во второй нормальной форме, и отсутствуют транзитивные функциональные зависимости неключевых атрибутов от ключевых.
- **Нормальная форма Бойса — Кодда (BCNF):** переменная отношения находится в нормальной форме Бойса — Кодда (иначе — в усиленной третьей нормальной форме) тогда и только тогда, когда каждая её нетривиальная и неприводимая слева функциональная зависимость имеет в качестве своего детерминанта некоторый потенциальный ключ.
- **Четвёртая нормальная форма (4NF):** переменная отношения находится в четвёртой нормальной форме, если она находится в нормальной форме Бойса — Кодда и не содержит нетривиальных многозначных зависимостей.
- **Пятая нормальная форма (5NF):** переменная отношения находится в пятой нормальной форме (иначе — в проекционно-соединительной

нормальной форме) тогда и только тогда, когда каждая нетривиальная зависимость соединения в ней определяется потенциальным ключом (ключами) этого отношения.

- **Доменно-ключевая нормальная форма (DKNF):** переменная отношения находится в ДКНФ тогда и только тогда, когда каждое наложенное на неё ограничение является логическим следствием ограничений доменов и ограничений ключей, наложенных на данную переменную отношения.
- **Шестая нормальная форма (6NF):** переменная отношения находится в шестой нормальной форме тогда и только тогда, когда она удовлетворяет всем нетривиальным зависимостям соединения. Из определения следует, что переменная находится в 6НФ тогда и только тогда, когда она неприводима, то есть не может быть подвергнута дальнейшей декомпозиции без потерь.

Графическая нотация

В качестве стандарта графической нотации используется **нотация Crow's Foot**.

Согласно данной нотации:

Сущность изображается в виде прямоугольника, содержащего её имя, выражаемое существительным. Имя сущности должно быть уникальным в рамках одной модели;

Связь изображается линией, которая связывает две сущности, участвующие в отношении. Степень конца связи указывается графически, множественность связи изображается в виде «вилки» на конце связи. Модальность связи так же изображается графически — необязательность связи помечается кружком на конце связи. Выражается глаголом в изъявительном наклонении настоящего времени или глаголом с поясняющими словами;

Атрибуты сущности записываются внутри прямоугольника, изображающего сущность, и выражаются существительным в единственном числе. Среди атрибутов выделяется ключ сущности.

Описание	Символ
кружок и тире: минимум 0, максимум один (необязательно)	
Тире и тире: минимум один, максимум один (обязательный)	

	..
Кружок и вилка: минимум 0, максимум много (необязательно)	
Тире и вилка: минимум один, максимум много (обязательно)	

Стандарт на кодирование frontend-составляющей

Используемые языки программирования

- Языки программирования Javascript, Typescript -- для кодирования frontend

Структура проекта

- **node_modules** - Содержит все установленные зависимости проекта.
- **public** - Содержит результат сборки и статические файлы хранилища.
 - static
 - icons
 - index.html
- **src** - Содержит исходники проекта
 - assets
 - components
 - layouts
 - middleware
 - mixins
 - services
 - store
 - utils
 - views
 - App.vue
 - main.js
 - router.js
- vue.config.js
- package.json

Идентификация элементов конфигурации

При использовании Typescript, Javascript выделяются следующие элементы.

Корень проекта

Элемент	Определение	Способ именования
Файл контроля используемых зависимостей и их версий	package.json	Файл контроля используемых зависимостей в корневом каталоге проекта
Файл конфигурации проекта	vue.config.js	файл конфигурации, в корневом каталоге проекта

Каталог src

Элемент	Определение	Способ именования
Компонент интерфейса	файл с расширением *.vue	полное название по пути к файлу от папки <i>frontend</i> , названия файлов или папок отделяются слэшем, после чего ставится имя компонента интерфейса через точку.
Файл стилей и препроцессоров	файл с расширением *.scss	полное название по пути к файлу от папки <i>frontend</i> , названия файлов или папок отделяются слэшем, после чего ставится имя файла стилей через точку.
App.vue	Экземпляр приложения	полное название по пути к файлу от папки <i>frontend</i> , названия файлов или папок отделяются слэшем
main.js	Входная точка для приложения	полное название по пути к файлу от папки

		<i>frontend</i> , названия файлов или папок отделяются слэшем
router.js	Маршруты приложения	полное название по пути к файлу от папки <i>frontend</i> , названия файлов или папок отделяются слэшем

Каталог public (статичные файлы frontend)

Элемент	Определение	Способ именования
Любой статический файл	файл статического контента	полное название по пути к файлу от папки <i>frontend</i> , названия файлов или папок отделяются слэшем

Стандарты на форму представления исходного кода

Стандарт, используемый для языков Typescript, Javascript

В качестве стандарта для представления кода на языках Typescript, Javascript используются модифицированные ESLint-правила. В модификации входят следующие правила:

1. В качестве отступов используются пробелы.
2. Для обозначения отступов используются 4 пробела.
3. Пробелы в конце строки обрезаются.
4. В конце любого файла с кодом добавляется пустая строка.
5. Фигурные скобки вокруг тела стрелочных функций.
6. CamelCase нотация написания кода.
7. В конце каждой непустой строки должна стоять точка с запятой.
8. В функциях именованные переменные стоят после неименованных.
9. Количество строчек кода в функции не более 25.
10. Максимум 5 переменных в функции.
11. Переменные при объявлении всегда должны быть инициализированы.
12. Максимум два класса могут быть описаны в одном файле.
13. Многострочные комментарии пишутся при помощи звездочек и слешей.
14. Запрет на использование пустых блоков кода (if, for...).
15. Запрет на использование eval метода.
16. Запрет на использование "мультистрок".

17. Нельзя создавать неиспользуемые переменные.
18. Использование констант для переменных, которые не перезаписываются.
19. Объявление переменных всегда должно быть наверху области видимости.
20. Отступы вокруг бинарных операторов.
21. Две строчки отступа между классами, одна между функциями.

Среда разработки

Для разработки на Typescript, Javascript используется Sublime Text3 v. 3.2.2 (Build 3211)

Стандарт на представление аналитики

Используемый стек технологий

- Python - язык программирования, используемый для анализа данных.
- SQL - язык запросов в БД, используемый для получения данных о пользователях, событиях и т.д.; нужны для подсчета основных целевых метрик (например, CSAT (удовлетворенность пользователей, конверсия, утилизация и т.д.)
- Tableau - это система интерактивной аналитики, позволяющая в кратчайшие сроки проводить глубокий и разносторонний анализ больших массивов информации и не требующая обучения бизнес-пользователей и дорогостоящего внедрения.

Используемые инструменты

- Excel - это программа для работы с таблицами, помощник для всех, кто имеет дело с большим объемом цифровых данных. Используется в качестве средства для первичной обработки данных.
- Think-sell - высокоэффективное программное обеспечение для создания диаграмм и верстки слайдов. Оно автоматизирует работу в PowerPoint, повышая таким образом эффективность и качество создания слайдов. Используется для создания интерактивных презентаций.
- Zeppelin - многопользовательское интерактивное браузерное программное средство для анализа и визуализации данных, а также совместной работы над данными. Используется для написания Python /SQL-запросов.
- Greenplum - массово-параллельная СУБД для хранилищ данных на основе PostgreSQL. Используется в качестве хранилища данных. Главным образом используются витрины данных.

Стандарт, используемый для языка Python

В качестве стандарта для представления кода на языке Python используется Стандарт PEP8.

1. В качестве отступов используются пробелы, не используется табуляция.
2. Для обозначения отступов используются 4 пробела.
3. Ограничение длины строки - максимум 79 символов.
4. Отделение определений функций и определений классов двумя пустыми строками.
5. Переменные при объявлении всегда должны быть инициализированы.
6. Используется кодировка UTF8.
7. Бинарные операторы окружаются пробелами с обеих сторон (Это операторы присваивания (=, +=, -= и другие), сравнения (==, <, >, !=, <>, <=, >=, in, not in, is, is not), логические (and, or, not)).
8. Комментарии должны являться законченными предложениями. Если комментарий — фраза или предложение, первое слово должно быть написано с большой буквы, если только это не имя переменной, которая начинается с маленькой буквы (не следует менять регистр переменной, т.к. это меняет смысл).
9. Многострочные комментарии пишутся при помощи тройных кавычек.
10. Блок комментариев с документацией кода, идущий после объявления функции/класса должен иметь тот же отступ, что и сам код. Каждая строчка такого блока должна начинаться с символа # и одного пробела после него (если только сам текст комментария не имеет отступа).
11. Абзацы внутри блока комментариев разделяются строкой, состоящей из одного символа #.
12. Импорты всегда помещаются в начале файла, сразу после комментариев к модулю и строк документации, и перед объявлением констант.
13. Каждый импорт, как правило, должен быть на отдельной строке.
14. Для имен функций используется нижний регистр. Если необходимо повысить читаемость, то можно разделять слова символом нижнего подчеркивания. (snake_case нотация)
15. Имена переменных выбираются по тем же правилам, что и имена функций.
16. Константы определяются на уровне подключения модулей и пишутся в верхнем регистре с разделением слова символами подчеркивания.
17. Можно использовать как одинарные, так и двойные кавычки. Однако, если в строке используются двойные кавычки, то следует выделять внутреннюю строку одинарными и наоборот.

Стандарт, используемый для языка SQL

1. Идентификаторы и имена выбираются осмысленными и в едином стиле (поддерживается одна нотация, выбираемая командой разработки/аналитики).
2. Имена полей должны быть уникальны и не должны быть в зарезервированном ключевым словом.
3. Пробелы и отступы расставляются индивидуально для лучшей читаемости кода.
4. Дата и время должны соответствовать стандарту ISO 8601: YYYY-MM-DD HH:MM:SS.SSSSS.
5. Функции SQL используются только стандартные (входящие в СУБД).
6. Комментарии пишутся предпочтительно в стиле C — /* (начало) и *, (конец). Либо - - перед комментарием, тогда окончанием будет новая строка.
7. Ограничение длины имени - 30 символов.
8. Имена начинаются с буквы и заканчиваются буквой.
9. В именах используются только буквы, цифры и символ подчёркивания.
10. Запрещено использование нескольких подряд идущих символов подчёркивания.
11. Символ подчёркивания используется там, где в реальной жизни стоит пробел (например, `first name` станет `first_name`).
12. Следует избегать сокращений. Если их всё же нужно использовать, следует убедиться в том, что они общепонятны.
13. Следует использовать собирательные имена или, что менее предпочтительно, форму множественного числа. Например, `staff` и `employees` (в порядке убывания предпочтения).
14. Не допускается совпадение названия таблицы с названием любого из её столбцов.
15. По возможности следует избегать объединения названий двух таблиц для построения таблицы отношений. Например, вместо названия `cars_mechanics` лучше подойдёт `services`.
16. Пробелы используются для выравнивания основных ключевых слов.
17. Всегда делайте перенос строки:
 - перед AND или OR
 - после точки с запятой (для разделения запросов)
 - после каждого основного ключевого слова
 - после запятой (при выделении логических групп столбцов)
18. У каждой таблицы должен быть хотя бы один первичный ключ.

Стандарт на визуализацию данных в TABLEAU.

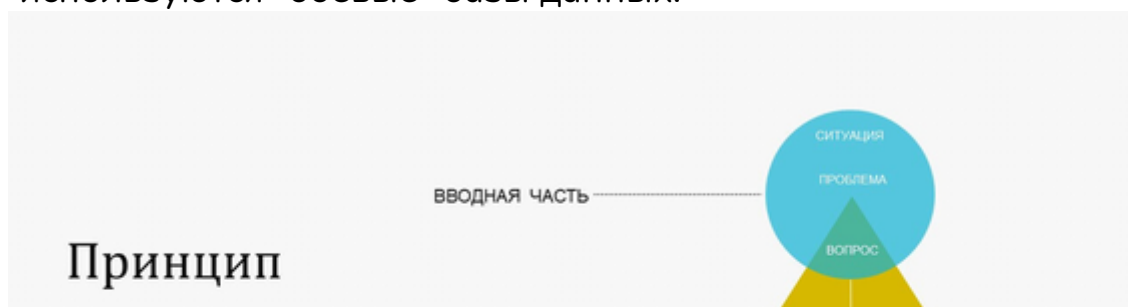
1. Визуализация данных должна быть читаема и понятна пользователям.
2. Использование бóльшего места для визуализации, подписи очевидны и не пересекаются друг с другом.
3. Цвета графиков должны быть различимы друг от друга, не должны нарушать интуитивное понимание пользователя. Пример: **красный**, **зеленый** - не подходит.
4. Всегда должны использоваться актуальные данные.
5. Используемые данные должны быть обработаны, подготовлены к визуализации.
6. Следует использовать технику представления более подробной информации с помощью «наведения курсора».
7. Необходимо иметь большое количество настроек графиков и отчетов для конечного пользователя (фильтры и т.д.).

Стандарт на предоставление аналитики (Создание презентаций).

1. Презентация должна быть структурирована (слайды должны логически переходить от одного к другому). Используется принцип Пирамиды Минто (приложение N°1).
2. В заголовке слайда помещается ровно 1 основной вывод по слайду.
3. Предоставляются выводы из результатов аналитики, целевые метрики и данные предоставляются только с использованием визуализации.
4. Отступ текста заголовков должен быть одинаков на всех слайдах.
5. Основной текст выравнивается по левому краю заголовка.
6. Используются несколько цветов на графиках для улучшения читаемости.

Рекомендация к процессу выгрузки данных.

1. В качестве частно используемых источников данных для аналитики используются аналитические базы данных, для того, чтобы не грузить исходные данные в оперативную память.
2. Для различных задач могут создаваться свои частно используемые локальные таблицы, с целью не использовать лишний раз общие таблицы без надобности.
3. Для построения отчетов, требующих оперативных данных, используются “боевые” базы данных.



пирамиды Минто

ГЛАВНОЕ
УТВЕРЖДЕНИЕ

АРГУМЕНТЫ



Стандарт на разработку требований

Правила представления спецификации требований к ПО

Формат спецификации требований к ПО должен быть согласован с заказчиком. В случае, когда заказчик явно не указал свои пожелания к формату спецификации требований к ПО, возможно использовать шаблоны, определенные в стандартах проекта. Ниже описаны требования к содержанию спецификаций программных требований, создаваемых в ходе [процесса разработки программных требований](#).

Содержание спецификации требований к ПО

Спецификация требований к ПО должна содержать:

- функциональные требования, включающие определение предполагаемых функций ПО и его внешних интерфейсов;
- актуальные нефункциональные требования, включающие архитектурные ограничения;
- системные требования.

Свойства спецификации требований к ПО

Спецификация требований к ПО должна обладать следующими свойствами:

- корректность (спецификация требований к ПО должна соответствовать входным данным процесса разработки требований к ПО);
- однозначность (каждое требование в спецификации требований к ПО должно иметь единственную интерпретацию);
- полнота (спецификация требований к ПО должна включать все требования, необходимые с точки зрения реализации функционала, ограничений и внешних интерфейсов системы);
- непротиворечивость (в спецификации требований к ПО не должно быть ни одного требования, противоречащего другому);

- избыточность (спецификация требований к ПО не должна содержать ненужных свойств/функций системы);
- неделимость (каждое требование в спецификации требований к ПО не может быть разделено на два или более независимых составляющих);
- модифицируемость (любое изменение в требованиях в спецификации требований к ПО может быть сделано полностью и последовательно, без изменений в структуре документа);
- проверяемость (каждое требование в спецификации требований к ПО может быть проверено на реализуемость в ПО);
- трассируемость (каждое требование в спецификации требований к ПО должно быть уникально идентифицировано).

Для обеспечения трассируемости требований внутри проекта используется система уникальных номеров, которые присваиваются каждому требованию с целью классификации их относительно системных требований и элементов архитектуры разрабатываемого ПО.

Присвоенный требованию номер содержит информацию, необходимую для соотнесения его с более общими требованиями и затрагиваемыми ими составными частями проекта.

Все идентификационные номера соответствуют следующему формату:

V-W-XYZ-NNNNN

Где:

V	
Число	Номер сценария использования системы, к которому относится требование
A	Требование относится ко всем сценариям

W	
Число	Номер действия в сценарии V, к которому относится требование
A	Требование относится ко всем действиям в сценарии
Пропуск	Если V = A

X	
A	Требование относится ко всем элементам архитектуры в целом
B	Требование относится к какому-то конкретному элементу архитектуры
C	Требование относится к связи между двумя элементами архитектуры

Y	
Пропуск	Если X = A
F	Если требование относится к frontend-составляющей или связи с frontend-составляющей
B	Если требование относится к backend-составляющей или связи с backend-составляющей
D	Если требование относится к data science-составляющей или связи с data science-составляющей
X	Если требование относится к внешним сервисам или связи с внешними сервисами

Z	
Пропуск	Если X = A или X = B
F	Если требование относится к связи элемента Y с frontend-составляющей

B	Если требование относится к связи элемента Y с backend-составляющей
D	Если требование относится к связи элемента Y с data science-составляющей
X	Если требование относится к связи элемента Y с внешними сервисами

NNNNN – случайно генерируемый пятизначный номер, уникальный для данной комбинации идентификаторов.

План верификации

План верификации определяет средства, с помощью которых будут достигаться цели процесса верификации.

Под верификацией понимается техническая оценка результатов как процессов разработки ПО, так и собственно процесса верификации.

Цель процесса верификации - обеспечение

- а) соответствия требований высокого и низкого уровней программной реализации;
- б) соответствия всех единиц конфигурации отвечающим им стандартам;
- в) полного покрытия тестами всех требований к ПО.

Среда верификации

Среда верификации - общая характеристика процесса верификации, связанные с ними компоненты разработки.

Входные и выходные данные

Входными данными в процесс верификации являются:

- Требования к ПО
- Описание проекта
- Исполняемый код сервисов (при наличии)
- Исполняемый код тестов (при наличии)
- Документы об организации процессов разработки

Выходными данными являются:

- Результаты формальной инспекции

- Артефакты сборки исполняемых файлов
- Артефакты прогона тестовых файлов
- Записи о покрытии

Инструменты верификации

Для проведения мероприятий верификации используются специальные инструменты, необходимые для проведения специфических процессов верификации. Предполагается, что их верификация не требуется (присутствуют соответствующие сертификаты и/или квалификационные пакеты).

Выделяются следующие инструменты:

Название инструмента	Предназначение
Библиотеки для модульного тестирования для языков Go, JavaScript и C#	Модульное и функциональное тестирование исходного кода
Сборный инструмент, состоящий из пакетов pytest, selenium и пр.	Автоматизированное интеграционное тестирование
Gitlab	Проведение рассмотрения документов и исходного кода сервисов, тестов
Jira	Управление запросами на изменение и сообщениями о проблемах
Jenkins	Среда непрерывной интеграции / непрерывной доставки, используемая для прогона тестов
Excel + Drawio	Составление и визуализация трассировки

Мероприятия верификации

Процесс верификации ПО подразумевает собой наличие верификационных мероприятий, которые служат для регистрации и определения ошибок. Мероприятия выделяются следующие:

- [Формальная инспекция](#)
- [Тестирование ПО](#)
- [Анализ покрытия](#)

Формальная инспекция

Формальная инспекция - одно из мероприятий процесса верификации ПО, направленное на обеспечение качества ПО посредством “осмотра” единиц конфигурации проекта.

Цель

Целью формальной инспекции является проверка каждой единицы конфигурации проекта на соответствие требованиям и стандартам проекта путём осмотра её заинтересованными участниками разработки.

Одной из важных целей формальной инспекции является обеспечение *не зависимости* в оценке единиц конфигурации. Это необходимо для достижения наивысшего качества итогового ПО.

Участники формальной инспекции

В процессе формальной инспекции принимают участие несколько участников, каждый из которых имеет свою роль

Роли:

- Инспектор - участник формальной инспекции, оценивающий единицу конфигурации в соответствии с некоторым чеклистом.
- Координатор формальной инспекции - главный инспектор, назначающий остальных инспекторов, назначающий начало инспекции и координирующий действия остальных участников ФИ.
- Автор - автор инспектируемой единицы конфигурации,

Важно отметить, что для достижения свойства независимости в формальной инспекции, инспекторами должны выбираться участники разработки, не связанные с инспектируемой единицей конфигурации.

Чеклисты

Характеристика

Каждая формальная инспекция проходит в соответствии с некоторым чеклистом, содержание которого определяется непосредственно инспектируемой единицей конфигурации проекта.

Например, в чеклисте для проведения формальной инспекции требований к ПО будут содержаться стандарты, выполнение которых является обязательным для прохождения ФИ.

Структура

Независимо от того, для какого конкретно типа единиц конфигурации применяется чеклист, он имеет шаблонную структуру.

- Проверка именования
- Проверка соответствия стандартам
 - Стандарт 1
 - Стандарт 2
- Проверка на соответствия требованиям
 - Требование 1
 - Требование 2
- Проверка на адекватность

При выполнении каждый из пунктов отмечается галочкой, при несоответствии - крестиком. Для прохождения ФИ требуется выполнение всех пунктов чеклиста

Процесс формальной инспекции

В зависимости от типа единицы конфигурации процесс формальной инспекции может отличаться, но отличия связаны, в первую очередь, с используемым инструментом для проведения формальной инспекции.

Общий случай

- Инициализируется процесс формальной инспекции. Собираются все единицы конфигурации, требующие инспекции.
- Собирается комиссия в соответствии с правилом независимости (влияет на подбор инспекторов).
- Генерируются чеклисты для проверки каждой единицы конфигурации.
- Обсуждение и проведение формальной инспекции с использованием одного из инструментов (Confluence или Gitlab).
- Завершение. Получение протокола формальной инспекции, определение единиц конфигурации, прошедших инспекцию или требующих доработки.

Использование Gitlab

Gitlab используется для проведения формальной инспекции единиц кода, например, функций, модулей или тестов.

Перед началом ФИ в рабочей ветке создаётся Merge Request, где в качестве Assignee указывается автор Merge Request'a, в качестве Reviewer указывается координатор формальной инспекции.

Далее проводится формальная инспекция в соответствии с общим случаем.

В процессе ФИ, если есть такая необходимость, оставляются комментарии к конкретным строчкам с указанием конкретных правок/недочётов.

В конце, в дополнение к протоколу формальной инспекции, добавляется ещё и запись о проведении ФИ на странице Merge Request'a.

Confluence используется для проведения формальной инспекции документов, например, стандартов, различных планов и т.д.

Перед началом ФИ каждый из нужных документов блокируется для редактирования.

В процессе ФИ, если есть такая необходимость, оставляются комментарии к конкретным словам/строкам/предложениям/абзацам, содержащим недочёты. Это позволяет автору, впоследствии, быстро и удобно просмотреть все места, требующие правок, учесть комментарии.

В конце, в дополнение к протоколу формальной инспекции, добавляются ещё и комментарии после ФИ на странице с документом.

Если ФИ пройдена успешно, то документ остаётся заблокированным, в противном случае он открывается на редактирование для проведения последующих правок.

Использование Jira

Формальная инспекция и статусы задач в Jira тесно связаны друг с другом.

Работа над любой задачей сопровождается задачей в Jira. В момент, когда создаваемая в рамках задачи единица конфигурации готова к формальной инспекции, сама задача переводится в статус

РАССМОТРЕНИЕ. Если в процессе ФИ не находятся недочёты, то соответствующая задача переводится в статус **ГОТОВО**. Если в процессе ФИ находятся недочёты, то соответствующая задача переводится обратно в статус **В РАБОТЕ**, а в комментарии к ней приводится протокол ФИ инспекции и указания к доработкам.

Тестирование ПО

Тестирование ПО - одно из мероприятий процесса верификации.

Цели тестирования:

- Проверка исходного кода разрабатываемых сервисов на соответствие требованиям низкого уровня
- Проверка “сложного функционала” на соответствие требованиям высокого уровня

Тестированию подвергаются все программные компоненты проекта, участвующие в процессе управления конфигурацией.

Все тесты пишутся/проводятся согласно принципу “чёрного ящика”: ответственный участник разработки использует только функциональные требования к ПО для проведения тестирования, без знания исходного кода сервисов.

Данные

Входы

- Стандарты на написание тестов
- Требования к ПО
- Спецификация сервисов

Выходы

- Тесты ПО по требованиям
- Записи о покрытии модульными тестами кода

Виды тестирования

Модульное тестирование

Суть

Модульное тестирование - это тестирование отдельных функций в исходном коде сервисов. Это необходимо для проверки соответствия написанных функций поставленным требованиям нижнего уровня.

Каждый модульный тест содержит набор входных параметров функции и ожидаемый результат её выполнения. Для разных функций набор тестовых сценариев может быть разным, в зависимости от количества входных параметров и сложности функции.

Отличия

Модульное тестирование является сугубо автоматизированным, любое изменение в исходном коде должно сопровождаться их прогоном для того, чтобы убедиться в том, что такие изменения не затрагивают поставленный в требованиях функционал.

Также стоит отметить, что модульное тестирование не использует компоненты, интегрируемые в сервис (БД, брокеры сообщений и т.д.). Если есть необходимость в обращении к таковым, то используются специальные заглушки.

Модульное тестирование также осуществляет сбор покрытия функций в коде тестами. Это осуществляется с помощью статического анализа кода, который проводится специальными библиотеками. Таким образом, на выходе получается документ о покрытии кода.

Библиотеки

В таблице представлены библиотеки, используемые для проведения модульного тестирования для используемых языков программирования.

Язык программирования	
-----------------------	--

	Библиотека для модульного тестирования и сбора покрытия
JavaScript	Jest 27.5
Go	go test 1.17
C#	Microsoft.VisualStudio.TestTools.UnitTesting

Функциональное тестирование

Суть

Функциональное тестирование - это тестирование отдельных компонент сервиса без включения внешних интеграций с другими сервисами. Это необходимо для проверки соответствия написанного функционала требованиям высокого уровня.

В функциональных тестах не проверяются отдельные функции, а сразу целые компоненты. В связи с этим такие тесты являются более сложными, чем модульные тесты, покрывают сразу несколько требований низкого уровня. Говоря в общем, один сценарий функционального теста описывает один процесс, происходящий в сервисе и захватывает в себя одну или более функцию, для которой существует модульный тест.

Отличия

В текущем проекте, в зависимости от сервиса, конечные цели функционального тестирования отличаются, т.к. компоненты разных сервисов отличаются. Так, выделяются следующие типы:

- Тесты API. Это функциональные тесты для backend-сервисов, в которых проверяется спецификация API и его функционал.
- Тесты воркеров/кronов. Это функциональные тесты асинхронных задач, где, в первую очередь проверяется корректность их выполнения в фоне. Такие тесты также могут проверять требования низкого уровня, говорящие о временных характеристиках работы сервисов.
- Тесты интерфейса. Проверяется корректная работа кнопок интерфейса.

Также в функциональных тестах могут использоваться интегрированные в сервис компоненты, такие как БД, брокеры сообщений и пр. Это бывает необходимо, т.к. со стороны теста не должно быть вторжения в данные /события, проверяются конкретные взаимодействия сервиса с ними.

Если существует необходимость производить интеграцию с другими сервисами, то используются заглушки, имитирующие их работу в случаях “позитивных” и “негативных” сценариев.

Библиотеки

В таблице представлены библиотеки, используемые для осуществления функционального тестирования для используемых языков программирования.

Язык программирования	Библиотека для функционального тестирования
JavaScript	Cypress 9.5.0
Go	go test 1.17 / testify / gonkey (разные для разных целей)
C#	xUnit 2.1

Интеграционное тестирование

Суть

Интеграционное тестирование служит для проверки взаимодействия двух и более сервисов друг с другом. Это наиболее сложный вид тестирования, т.к. он затрагивает большое количество связанных друг с другом компонент и сложен в реализации.

Каждый тестовый сценарий представляет собой процесс, который затрагивает два и более сервиса, описывает поведение каждого из них, те изменения, которые происходят с данными и событиями. Всё это трассируется на исходные системные требования, которые такой процесс и описывают.

Отличия

Интеграционное тестирование может быть как автоматизированным, так и ручным, оба типа допустимы и применяются в разных ситуациях.

Для проведения ручного интеграционного тестирования требуется тестовый стенд, на который могут быть загружены исполняемые файлы необходимых сервисов и выведена их конфигурация, схожая с “боевой”. Составляется набор тестовых сценариев, которые описывают межсервисное взаимодействие, указанное в системных требованиях к проекту. Затем тестировщик проводит руками действия, необходимые для воспроизведения того или иного сценария с целью убедиться, что полученное поведение соответствует ожидаемому (соответствует требованиям).

Библиотеки

В случае автоматизированного интеграционного тестирования используются специально разработанные под конкретный проект инструменты, т.к. единого подхода не существует. В случае текущего проекта используется язык программирования Python и несколько библиотек.

Название библиотеки	Применение
Requests	Отправка http-запросов на API
Selenium	Взаимодействие с интерфейсом
Psycopg2 / PyMysql	Взаимодействие с базами данных
Pytest	Проведение проверок

Анализ покрытия

Тестирование

gonkey. Функциональное тестирование backend

gonkey - фреймворк для проведения функционального тестирования средствами языка Go.

Основная особенность gonkey - написание тестовых кейсов с использованием языка разметки yaml. По сути описание теста получается декларативным, легко читаемым и понятным.

Шаблонный репозиторий с gonkey-тестами - <https://gitlab.com/ikramanop/city-for-students-tests-example/-/tree/main/>

Более подробно про gonkey:

- <https://github.com/lamoda/gonkey>
- <https://habr.com/ru/company/lamoda/blog/539168/>

Как использовать?

Для того, чтобы использовать gonkey в своём проекте, необходимо скопировать папку tests в корень своего проекта, связать исполняемый файл сервиса с файлом test_runner.go, указать необходимые переменные окружения для внешних хостов и подключений к базам данных.

Что значит каждый из файлов в директории tests?

- **/cases** - директория с тестовыми файлами в формате .yaml
- **/fixtures** - директория с фикстурами для БД в формате .yaml

- **/mocks** - директория с моками внешних сервисов в формате .json или .xml
- **/logs** - директория, в которую складываются результаты тестирования в виде allure-отчётов
- **test_runner.go** - файл, в котором определяется структура для запуска тестов
- *****_test.go** - файл, в котором вызывается сам тест

Как написать тест по существующей спецификации сервиса?

Для написания теста нам необходимо несколько вещей:

- Спецификация тестируемой ручки API
- Схема внешних взаимодействий (для шлюзов и *редко* для backend-сервисов)
- Схема данных (для backend-сервисов)
- Схема бизнес-логики (для backend-сервисов)

Из спецификации мы берём название ручек, модели запросов/ответов.

По схеме внешних взаимодействий можно понять какие требуются заглушки для тестов.

Схема данных определяет тестовые фикстуры.

Схема бизнес-логики определяет, как изменяются данные в разных бизнес-процессах в разных ручках API.

Пример gonkey-теста с комментариями в формате .yaml

```
- name:
    ,
    event -
    method: PUT -
    path: /v2/event/1 -

    fixtures: -
      - empty_db
      - event

    variables: -
      event_id: 1

    request: | -
      {
        "name": "    ",
        "date": "26.03.2022",
        "description": "    .    ",
        "organization_id": 1,
```

```

    "price": 0,
    "city": {
      "name": "",
      "region": {
        "name": "",
        "timezone": "+03:00"
      }
    },
    "address": " . 1",
    "type": {
      "type": ""
    },
    "rating": {
      "rating": 5
    },
    "tags": [
      {
        "tag": ""
      },
      {
        "tag": ""
      }
    ]
  }

response:
200: | -
    {
      "code": 0,
      "status": true,
      "message": ""
    }

dbQuery: | -
  SELECT name, date, description FROM event
  WHERE id = {{ $event_id }}

dbResponse: -
- '{
  "name": " ",
  "date": "26.03.2022",
  "description": " . "
}'

```